

无线电干扰源的快速自动定位与清除

摘 要

针对目标圆域内无线电干扰源的快速定位与清除，本文将测向误差写成有界集合，而不是随机噪声。各检测点对应一个张角 2° 的误差锥，定位区域就是这些锥的公共交集。四问共用同一套几何对象：问题一给出交会多边形的直径与覆盖判定，问题二在此之上选择第二检测点，问题三用覆盖网搜索全向源，问题四把覆盖改成环抱证书以检出定向源。程序可在官方模拟器上直接运行。

问题一把定位区域看成凸多边形。直径 d 是全部顶点连线（边和对角线）里最长的一条；以 d 为直径的圆，圆心只能取这条线段的中点，再检验各顶点到中点是否都不超过半径。等边三角形给出反例：第三顶点到中点为 $\sqrt{3}d/2 > d/2$ 。两站交会算例直径 $d = 70.419 \text{ m}$ ，直径圆可以覆盖；三站环绕时直径圆盖不住，但最小包围圆半径 16.288 m ，仍小于 20 m 的清除半径。能否一次清除由最小包围圆半径是否不超过 20 m 决定，与直径圆是否覆盖无关。

问题二的关键是：干扰源到 S_1 的距离未知，第二点只能落在某个候选区域里，再按假设从中选出一个点。候选区由交会角约束给出，分居示向度两侧，呈蝶形。四种假设对应四个点，均由同一张网格上的明确准则给出；时间流派 (500,250) 的成功样本期望约 268 s ，用于问题三。问题四因定向波束改用小侧偏。两次仍压不进 20 m 时，沿当前长轴的法向再切一刀，远目标直径可由 164.5 m 压到 33.1 m 。计入平均时间的，只包括最小包围圆半径已经不超过 20 m 的成功样本。

问题三把未知源数变成有限点判空：按接收半径下界 1000 m 铺七点覆盖网（原点加 1150 m 六等分），最坏间隙 988.5 m 、余量 11.5 m 。机器狗原点扫频、按时间赌注 (500,250) 追击、环网补漏。正式测试三局均 100% 清除，平均定位清除时间 285.34 s/个 ($259.48\text{--}323.51 \text{ s}$)。同一策略在官方模拟器 19 局演练中 19/19 全清，平均 $352 \pm 31 \text{ s/个}$ ($290\text{--}402 \text{ s}$)。关掉覆盖网补漏后清除率降至 56.3% 。

问题四定向源使无信号失去距离含义。判空改为方位环绕：探针绕候选点的最大间隙不超过 180° 。采用含圆外停点的 26 点骨架 ($400 \text{ m} \times 6$ 、 $1300 \text{ m} \times 8$ 、 $1900 \text{ m} \times 12$)，粗网格最坏间隙恰为 180.0° ；第二点改小侧偏。正式测试三局均 100% 清除，平均 600.26 s/个 ($518.78\text{--}698.52 \text{ s}$)。同一策略官方 17 局演练 17/17 全清，平均 $593 \pm 95 \text{ s/个}$ ($411\text{--}740 \text{ s}$)。把证书换成七点网或圆内环绕，清除率分别降至 94.1% 与 96.0% 。

关键词： 交会定位；误差锥；最小包围圆；第二检测点；覆盖网；环抱证书；定向干扰源

一、问题重述

目标区域是半径 1800 m 的圆盘。无人机器狗可在圆内直线移动，速度 5 m/s；停下后对指定频道测向，单次 5 s。测向误差有界：真实方位角与示向度之差不超过 $\delta = 1^\circ$ 。有效接收半径 $R \in [1000, 1500]$ m。机器狗走到距源不超过 20 m 处，即可光学定位并激光清除；不超过 5 m 时信号过强，无法测向，但可直接清除。

问题一给出若干检测点及其示向度，要求计算交会得到的多边形定位区域的直径，并判断以该直径为直径的圆能否覆盖该区域。这里的定位区域只由示向度误差锥交会得到，不叠加目标圆域或接收半径圆盘。

问题二已知某全向干扰源已在第一检测点测得示向度，要求给出第二个检测点的选择策略及其候选区域。所谓较好的定位，是指两次（必要时三次）测向后，定位区域的最小包围圆半径不超过 20 m，从而一次进圈即可清除，同时尽量缩短移动与测量时间。

问题三的圆内有 10–16 个全向干扰源，分别占用频道 1–20 中的若干个，每频道至多一个源。机器狗从原点、频道 1 出发，要在虚拟时间与约 20 分钟现实时限内，尽量提高清除比例、缩短平均定位清除时间。

问题四在问题三基础上引入定向干扰源。定向源的个数与朝向均未知，只在定向方向两侧各 90° 的闭半平面内辐射。硬约束是清除比例 100%，并在此前提下缩短平均定位清除时间。

二、问题分析

2.1 问题一

问题一要求根据若干检测点及其示向度，确定交会得到的多边形定位区域 P ，计算直径 d ，并判断以该直径为直径的圆能否覆盖 P 。示向度存在有界误差 $\pm 1^\circ$ ，单次测向只能将干扰源限制在以检测点为顶点、张角为 2° 的误差锥内。各锥取公共交集，即得到 P 。当各锥对向交会时， P 为凸多边形。

凸集的直径在极点处取得，因而 d 等于全部顶点两两距离的最大值。题面进一步要求判断以该直径为直径的圆能否覆盖 P 。圆盘内最远点对只能是对径点，故若这样的覆盖圆存在，圆心唯一确定为直径端点的中点；再检验各顶点是否落入该圆即可。一般凸多边形并不保证落进直径圆，等边三角形给出反例，因此覆盖判定必须逐点进行。半平面交会用于求出顶点；单站或示向度近乎同向时公共区域无界，直径没有有限值。

2.2 问题二

第一次测向之后，干扰源的方位已知而距离未知，可行域收缩为以 S_1 为顶点的细长楔形。因此第二检测点不存在唯一最优坐标，而应落在由交会角约束确定的候选区域内；再根据对未知距离的假设，从该区域内选出具体的测点。交会角 γ 决定定位区域的形状；一阶平行四边形近似给出直径关于 γ 的闭式，从而说明 $\gamma = 90^\circ$ 时定位效果最好。距离未知时，为使各种可能距离仍保持可用交会角，第二点必须离开示向度射线足够远，候选区因此分居两侧，呈蝶形。四种准则在同一张网格上各给一个采用点；问题三采用时间赌注 (500, 250)，问题四因定向波束改用小侧偏。两次不够，就沿长轴法向再测。

2.3 问题三

问题三的难点不是定位，而是不知道有几个源、占用哪些频道时如何保证一个不漏。全向源无信号只有两种可能：该频道已无未清源，或检测点已超出接收半径。一组监听点若能用半径 1000 m 的接收圆盘盖住整个 1800 m 圆域，某频道在这些点全无信号即可判空。覆盖网最坏间隙必须小于 1000 m；4–6 点不够，原点加 1150 m 六等分的七点网最坏间隙 988.5 m。定位清除复用问题一、二：原点扫频、追击、环网补漏，第二点用时间赌注 (500, 250)。

2.4 问题四

定向源只在发射方向两侧各 90° 辐射，“无信号”不再等于超距。问题三的圆盘覆盖因此失效。判空改为方位环绕：绕任意候选点，落在 1000 m 内的探针最大方位间隙不超过 180° ，则任意朝向必被听到。圆缘外向源使圆内探针间隙达到 360° ，证书必须含圆外停点。构造 26 点骨架后网格核验最坏间隙 180° 。追击改小侧偏，避免沿示向度走过源点掉进背向阴影。

三、模型假设

- H1 示向度误差有界 $[-\delta, \delta]$ ， $\delta = 1^\circ$ ；同一地点重复测量误差不变（题面附录）。
- H2 测向机与干扰源视为同一平面；问题一至三中干扰源全向，接收只取决于距离（题面）。
- H3 示向度是检测点指向干扰源的方位角，可行域只取检测点前方的误差锥（题面定义）。
- H4 问题一的定位区域只由误差锥交会得到，不叠加目标圆域与有效接收半径圆盘（与附录多边形交会区一致）。
- H5 问题二计时：移动按 5 m/s 直线，单次测量、清除各 5 s；题面未给转向时间，故不另计。问题一不涉及移动（题面附录）。
- H6 有效接收半径 $R \in [1000, 1500]$ m，未知但对该源固定；问题三覆盖判空按最坏 1000 m（题面附录）。
- H7 问题二表 2 的 $E[T]$ ：假定源在 1800 m 圆内均匀出现，于是距离 D 的密度正比于 D 。题目没有别的距离先验。问题一的直径与覆盖不用这条。
- H8 问题二称两步成功：两次测向后 $R_{\text{MEC}} \leq 20$ m；问题三第二点采用时间赌注 (500, 250)。
- H9 定向源可检区域为张角 180° 的闭半平面与接收圆盘的交；朝向固定但未知（问题四）。
- H10 检测点允许在圆域外；环抱证书外环取 1900 m（问题四，题面坐标范围）。

四、符号说明

符号	含义	单位	首次出现
$S_i = (x_i, y_i)$	第 i 个检测点	m	§5.1
N	检测点个数	—	§5.1
θ_i	在 S_i 处测得的示向度	度	§5.1
$\delta = 1^\circ$	测向误差限	度	§2.1
G	干扰源位置	m	§5.1

符号	含义	单位	首次出现
C_i	第 i 个检测点的误差锥	—	§5.1
$P = \bigcap_i C_i$	交会定位区域	—	§5.1
$V(P)$	P 的顶点集, 边数至多 $2N$	—	§5.1
$d = d(P)$	P 的直径 (题面符号)	m	§5.1
R_{MEC}	P 的最小包围圆半径	m	§5.1
C_{MEC}	最小包围圆圆心	m	§5.1
$\mathbf{n}_i$	从检测点看、指向误差锥内侧的单位法向	—	§5.1
$D = \ G - S_1\ $	干扰源到第一检测点的距离	m	§6.1
$d_i = \ G - S_i\ $	第 i 个检测点到源的距离	m	§6.2
γ	交会角 $\angle S_1 G S_2$	度	§6.2
J	测向关于源坐标的雅可比	1/m	§6.3
D_{loc}	一阶平行四边形的直径	m	§6.3
t, h	第二点局部坐标: 沿示向度的赌注与侧偏	m	§6.1
$s_{\min}$	可用交会角的正弦下界, 取 $1/2$	—	§6.5
K	两步成功常数, $K = 40/(2\tan 1^\circ) \approx 1146$	m	§6.4
κ	最小包围圆半径与直径之半的比	—	§6.3
W	第一次测向后干扰源的候选集	—	§6.1
$\mathcal{C}$	第二检测点的候选区域	—	§6.5
ρ	救援测点相对当前最小包围圆圆心的偏移	m	§6.7
$E[T]$	单目标定位清除总时间的期望	s	§6.6
R	有效接收半径, $R \in [1000, 1500]$	m	§7.1
r	问题三环网监听半径	m	§7.1
$\max\text{gap}(G)$	探针绕候选点 G 的最大方位间隙	度	§8.1
$\mathcal{S}$	问题四 26 点环抱骨架	—	§8.1

五、问题一的模型建立与求解

5.1 模型建立

将 N 次观测记为 $\{(\theta_i, S_i)\}_{i=1}^N$ 。真实源的可行位置集合由各次观测的误差锥给出:

$$C_i = \{X: \text{atan2}(X - S_i) \in [\theta_i - \delta, \theta_i + \delta]\}, P = \bigcap_{i=1}^N C_i.$$

定义 1 (误差锥). 检测点 S_i 的示向度为 θ_i , 测向误差限 $\delta = 1^\circ$, 真实源 G 位于以 S_i 为顶点、张角 2δ 的锥内:

$$C_i = \{X: \theta_i - \delta \leq \text{atan2}(X - S_i) \leq \theta_i + \delta\}.$$

定义 2 (交会定位区域). 各误差锥的公共交集

$$P = \bigcap_{i=1}^N C_i.$$

C_i 的两条边界直线过 S_i ，方向分别为 $\theta_i - \delta$ 与 $\theta_i + \delta$ 。取指向锥内的单位法向： $\theta_i - \delta$ 边为方向向量逆时针旋转 90° ， $\theta_i + \delta$ 边为顺时针旋转 90° 。记 $\mathbf{n}_{ij}$ 为第 j 条边界的内法向、 $\mathbf{o}_{ij}$ 为该直线上一点，则

$$P = \{X: (X - \mathbf{o}_{ij}) \cdot \mathbf{n}_{ij} \geq 0, j = 1, 2, i = 1, \dots, N\}.$$

共 $2N$ 个闭半平面，因而 P 凸。顶点取相邻边界直线的交点，仅保留同时落在全部半平面内的交点；该顶点集是算法 1 第 1 步的输入。当 $N \geq 2$ 且各锥对向交会时 P 有界，边数不超过 $2N$ ，见图 1。

模型准备. 题面只给出误差界 $[-\delta, \delta]$ ，不给出误差分布。最小二乘将测向差视为随机噪声，输出概率椭圆，需要分布假设；本题将可行域取为半平面的交，只使用该误差界。

方法	所需信息	输出
最小二乘 / 概率椭圆	误差分布	置信椭圆
集合估计 / 半平面交	仅 $[-\delta, \delta]$	可行凸集 P

图 1(a) 画出两锥如何交出 P 。为使交会可见，图中将 $\pm 1^\circ$ 放大为 $\pm 8^\circ$ ；图 1(b) 与后文算例改回真实 $\pm 1^\circ$ 。两个示向度的圆周距离若不超过 2° ，公共区域无界，直径没有定义，判别方法见定理 2。

定义 3 (直径). 有界凸区域 P 的直径 (题面符号 d) 为

$$d = d(P) = \max_{u, v \in V(P)} \|u - v\| = \max_{X, Y \in P} \|X - Y\|.$$

凸集上的最远点对在顶点处取得，故在顶点集上枚举即可：计算全部顶点两两间的距离，即边和对角线，最长的一条就是直径，见图 1(b)。

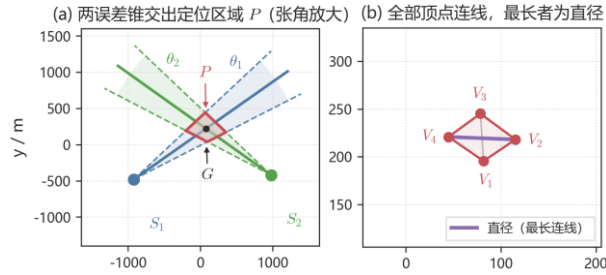


图 1 交会得到定位区域，再取全部顶点连线中的最长者

图 1 (a) S_1, S_2 的误差锥交出 P (张角放大)；**(b)** 真实 $\pm 1^\circ$ 下的凸四边形，粗线为全部顶点连线中的最长者，即直径。

第一问第二小问 (直径圆覆盖). 设 A, B 达到直径， $\|A - B\| = d$ 。若覆盖圆即为直径圆，它必须盖住 A 和 B ；圆内最远的两点只能是对径点，故圆心只能取

$$O = \frac{A + B}{2}, R = \frac{d}{2}.$$

再检查各顶点到 O 的距离是否都不超过 R ：圆盘是凸集，顶点都在圆内则 P 在圆内，图 2(a) 画出这一检验。反例：等边三角形以底边为直径作圆，第三顶点到圆心的距离为 $\sqrt{3}d/2 > d/2$ ，见图 2(b)。

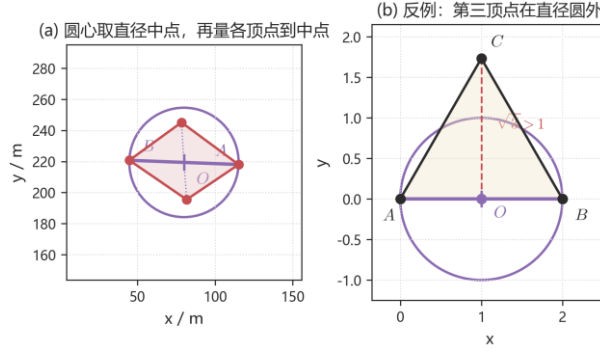


图2 中点检验覆盖，以及必须检验的理由

图2 (a) 圆心取直径中点 O ，虚线为各顶点到 O 的距离；(b) 等边三角形反例：第三顶点在直径圆外。

直径圆盖不住，只说明以 d 为直径、以直径中点为圆心的那个圆不够大； P 仍可能被其它圆心的圆盖住。盖住 P 的最小圆是最小包围圆，圆心记为 C_{MEC} ，半径记为 R_{MEC} 。

引理1 (Jung 定理, 见文献 [5]) . 直径为 d 的平面紧集, 其最小包围圆半径满足

$$\frac{d}{2} \leq R_{\text{MEC}} \leq \frac{d}{\sqrt{3}} \approx 0.577d.$$

最小包围圆一般不是外接圆。下界取等：直径圆已经盖住该集，最小包围圆就是这个两点圆。上界取等：集合是等边三角形，这时最小包围圆碰巧是三顶点的外接圆，半径 $2/\sqrt{3}$ （边长 2 时约 1.155），见图 3(a)。证明见文献 [5]。

定理1 (最小包围圆的结构) . 最小包围圆由 P 的顶点确定，只有两类：① 某两个顶点为直径端点的圆（圆心即中点）；② 某三个顶点的外接圆，且圆心落在这三点构成的三角形内部（锐角三角形）。钝角三角形的外接圆心在形外，半径大于直径之半，不会是最小包围圆。因此不是“凡三角形就取外接圆”；枚举上述两类即可。

算法1 (直径、直径圆覆盖与最小包围圆)

输入：顶点集 $V(P)$, $n = |V(P)| \leq 2N \leq 6$ 。输出：直径 d 、直径圆是否覆盖、 $(C_{\text{MEC}}, R_{\text{MEC}})$ 。

1. 枚举顶点对，取最远的两点 A, B ，令 $d \leftarrow \|A - B\|$ （直径定义）；2. 取 $O \leftarrow (A + B)/2$ 、 $R \leftarrow d/2$ ；若所有顶点 V_k 满足 $\|V_k - O\| \leq R$ 则判“覆盖”，否则“不覆盖”；3. 枚举顶点对：在覆盖全部顶点、以这两点为直径端点的圆中取半径最小者；4. 枚举顶点三元组：在覆盖全部顶点的三点外接圆中取半径最小者；5. 返回第 3、4 步中半径较小者，作为 $C_{\text{MEC}}, R_{\text{MEC}}$ 。

由定理 1，最小包围圆必在第 3、4 步的候选之中；复杂度 $O(n^3)$ 。

清除判据. 一次进圈保证清除，当且仅当存在点 C 使 $P \subset B(C, 20)$ ，即

$$P \subset B(C, 20) \Leftrightarrow R_{\text{MEC}} \leq 20.$$

取 $C = C_{\text{MEC}}$ 即得充分性；若 $R_{\text{MEC}} > 20$ ，则 P 中必有一点落在 20 m 之外， G 可能正是该点，故必要性成立。该判据只用到本问的 P ；本问第一问第二小问只回答直径圆能否覆盖，能否一次清除看 $R_{\text{MEC}} \leq 20$ 。

图 3 对照两个判据。算例 G 的直径圆不覆盖，但 $R_{\text{MEC}} = 16.288 < 20$ m，可以清除；算例 B 的直径圆覆盖，但 $R_{\text{MEC}} = 35.209 > 20$ m，两次测向不足以清除；等边三角形给出覆盖缺口的上界 $d/\sqrt{3}$ 。

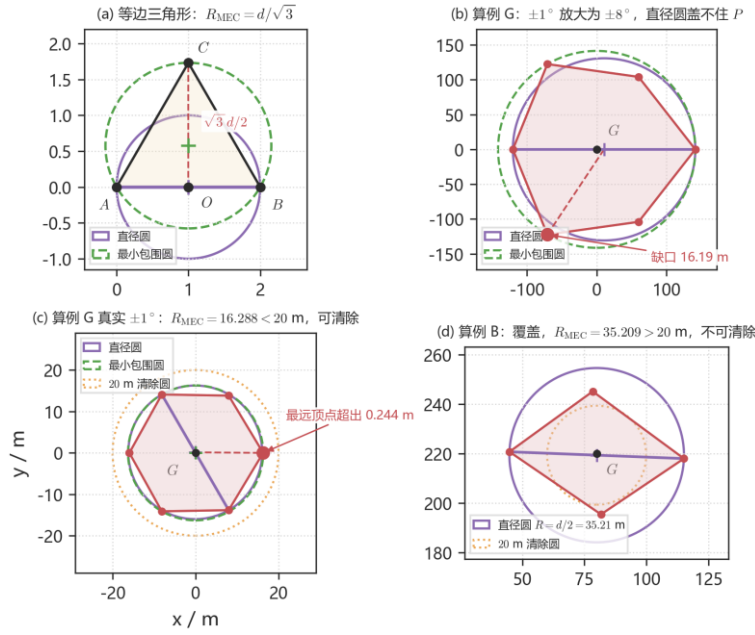


图3 直径圆与最小包围圆

图3 (a) 等边三角形（算例 E）：直径圆圆心 $(1, 0)$ 、半径 1，最小包围圆即外接圆，圆心 $(1, 1/\sqrt{3})$ 、半径 $2/\sqrt{3}$ ， $R_{MEC} = d/\sqrt{3}$ 。(b) 算例 G 把 $\pm 1^\circ$ 放大为 $\pm 8^\circ$ （与图 1 相同的示意惯例）： $d = 261.37$ m， $R_{MEC} = 141.29$ m，直径圆盖不住 P ，最远顶点缺口 16.19 m。(c) 算例 G 真实 $\pm 1^\circ$ ： $d = 32.252$ m，直径圆 $R = d/2 = 16.126$ m，最远顶点超出 0.244 m， $R_{MEC} = 16.288 < 20$ m，可清除。(d) 算例 B： $d = 70.419$ m，直径圆覆盖， $R_{MEC} = 35.209 > 20$ m，不可清除。

定理 2（有界性判据）. 每个半平面记为 $\{X: (X - o_i) \cdot \mathbf{n}_i \geq 0\}$ ， $\mathbf{n}_i$ 指向可行域一侧。令 $\alpha_i = \text{atan2}(n_{i,y}, n_{i,x})$ ，按角度排序列为 $\alpha_{(1)} \leq \dots \leq \alpha_{(m)}$ ，相邻两个法向（含首尾闭合）的夹角为

$$g_k = \alpha_{(k+1)} - \alpha_{(k)} \quad (k < m), g_m = \alpha_{(1)} + 2\pi - \alpha_{(m)}, g_{\max} = \max_k g_k.$$

则 P 有界当且仅当 $g_{\max} < \pi$ 。若 $g_{\max} \geq \pi$ ，取最大间隙的角平分方向 α^* （跨越 0 时右端按 $\alpha_{(1)} + 2\pi$ 计）的反向

$$\mathbf{u} = -(\cos \alpha^*, \sin \alpha^*), \alpha^* = \frac{1}{2}(\alpha_{(k^*)} + \alpha_{(k^*+1)}), k^* = \arg \max_k g_k,$$

则 $\mathbf{n}_i \cdot \mathbf{u} \geq 0$ 对所有 i 成立，沿 $\mathbf{u}$ 可走到无穷。两个示向度圆周距离不超过 2° 时即属此情形（算例 F）。几何解释见附图 1。

5.2 模型求解

算法流程见附图 2： $N < 2$ 或法向角间隙 $\geq 180^\circ$ （定理 2）判为无界；否则边界直线两两求交得顶点，再按算法 1 求直径、直径圆覆盖与最小包围圆。无界区域讨论直径无意义，需要增加或调整检测点。

表 1 汇总七个算例。

表 1 问题一算例汇总（“可清除”一列按 $R_{MEC} \leq 20$ m 判定；直径列为题面符号 d ）

算例	构型	状态	顶点数	d (m)	直径圆覆盖	R_{MEC} (m)	可清除
A	单站，单个误差锥	UNBOUND ED	—	—	—	—	—

算例	构型	状态	顶点数	d (m)	直径圆覆盖	R_{MEC} (m)	可清除
B	两站对向， 交会四边形	OK	4	70.419	是	35.209	否
C	三站切入， 交会六边形	OK	6	49.800	是	24.900	否
D	两站示向度 相差 180° ，锥 不相交	EMPTY	—	—	—	—	—
E	边长 2 的 等边三角形 (几何核 验)	OK	3	2.000	否	1.155	—
F	两站示向度 相同	UNBOUND ED	1 (有限顶 点)	—	—	—	—
G	三站均匀环 绕原点， 800 m	OK	6	32.252	否	16.288	是

表中“覆盖”只回答第一问的第二小问：以 d 为直径、以直径中点为圆心的圆能否盖住 P 。B、C 两行说明“覆盖”并不意味着能一次清除 (R_{MEC} 为 35.209、24.900 m，都超过 20 m)；G 行说明清除也不要求“覆盖”。

算例 B 四个顶点为 (81.714, 195.462)、(115.159, 218.014)、(78.375, 245.150)、(44.794, 220.758)，直径端点为第二、第四顶点，中点到各顶点均不超过 35.209 m，故直径圆覆盖。算例 G 的构型是：源在原点，三站为 (800, 0)、 $(-400, \pm 400\sqrt{3})$ ，示向度指向原点。直径圆盖不住两个顶点，见表中“否”；但 $R_{\text{MEC}} = 16.288 < 20$ ，圆心取 C_{MEC} 后 P 整体落入清除圆，见图 3。表中“直径圆覆盖=否”只回答问题一的第二问，不表示该源不能清除；能否清除只看 R_{MEC} 。各算例的站址、示向度与复现入口见附录 C。

模型检验. ① 与暴力搜索对照：对算例 B、C、G，以 2 cm 步长网格枚举最小包围圆圆心、以 1 cm 步长采样多边形边界枚举直径， R_{MEC} 与 d 的最大偏差分别为 0.37 cm 与 0.00 cm，属网格分辨率量级；② 退化与边界：单站 (A) 与同向两站 (F) 判为无界，相背两站 (D) 判为空集，单点退化的直径取 0，共线三点的直径等于两端距离，均与人工判断一致；③ 量纲与相似性：输入只有长度 (m) 与角度 (度)，将算例 B 整体放大 $k = 0.5, 2, 3$ 倍后 d 与 R_{MEC} 精确放大同一倍数 (相对误差 $< 10^{-12}$)，算法与尺度无关。

六、问题二的模型建立与求解

6.1 第一次测向之后

第一次测向之后，方向已知、距离未知。干扰源落在 S_1 示向度的楔形里，见图 4。因此第二点不可能有唯一最优坐标：它应当落在某个候选区域内，再按对未知距离的假设选出一个具体的点。

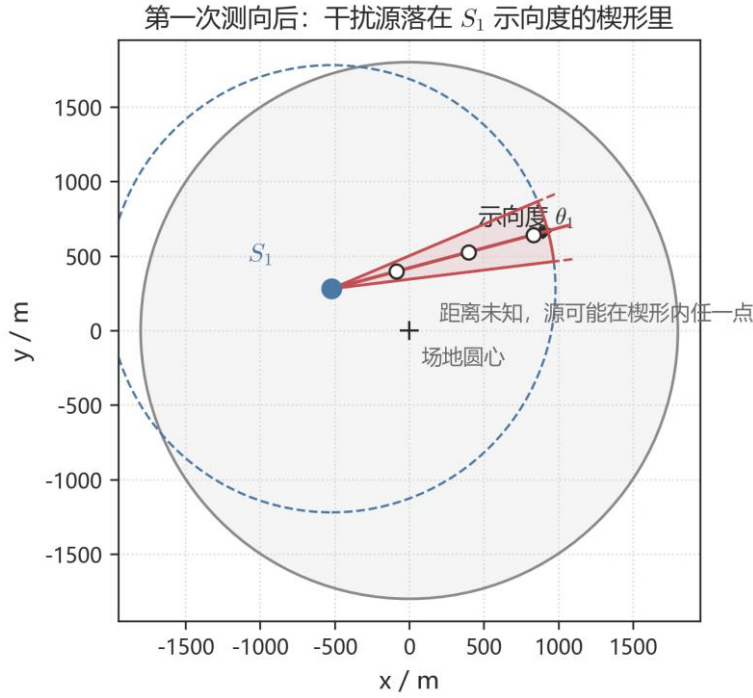


图4 第一次测向后：源落在 S_1 示向度的楔形里

图4 标出 S_1 与示向度 θ_1 。真实张角仅 $\pm 1^\circ$ ，图中放大至 $\pm 8^\circ$ 。空心圆表示距离未知。

以 S_1 为原点、测得示向度为极轴，第二点写成 $S_2 = S_1 + t\mathbf{e}_\theta + h\mathbf{e}_\theta^\perp$ 。 t 是对未知 $D = \|G - S_1\|$ 的纵向猜测， h 是侧偏。沿示向度走则两次共线，交会无界，所以必须有侧偏。

6.2 交会角

记 ε 为真实方向相对测得轴的偏角。 $\varepsilon = 0$ 时， S_1, G 在极轴上， $S_2 = (t, h)$ ，在 $\triangle S_1GS_2$ 中对边 h 对应 $\gamma = \angle S_1GS_2$ ，故

$$\sin\gamma = \frac{|h|}{d_2}, d_2 = \sqrt{(D-t)^2 + h^2}.$$

一般情形视线转过 ε ，有 $\sin\gamma = |h\cos\varepsilon - t\sin\varepsilon|/d_2$ 。图5 标出 t, h, d_1, d_2, γ 。侧偏越大、第二点越靠近源，交会角越接近直角。

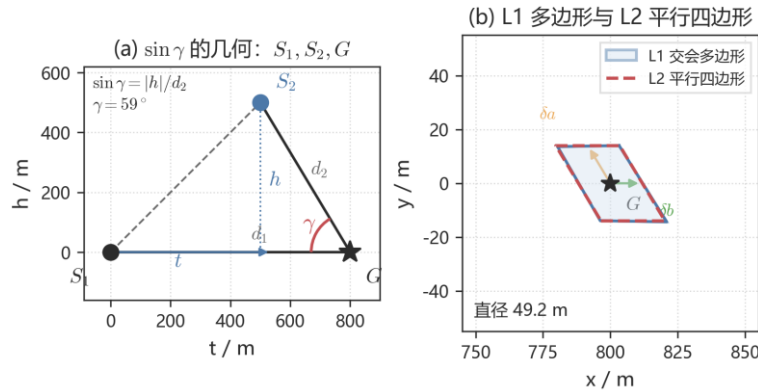


图5 交会角 $\sin\gamma = |h|/d_2$

图5 $\varepsilon = 0$ 时的直角三角形关系。本图 $t = h = 500$ m， $D = 800$ m， $\gamma = 59^\circ$ 。

6.3 一阶平行四边形与 D_{loc}

设计阶段需要能直接看出 (t, h, D) 如何影响直径。精确区域仍是问题一的凸多边形；这里做一次一阶展开，得到只看 d_1, d_2, γ 的闭式用于选点，程序终判仍回到问题一的多边形算法。推导分四步。

(1) 示向度的梯度。第 i 站示向度 $\varphi_i = \text{atan2}(y - S_{iy}, x - S_{ix})$ 对源坐标的梯度是视线法向除以距离：

$$\nabla \varphi_i = \frac{n_i}{d_i}, n_i \perp (G - S_i), \|n_i\| = 1.$$

沿视线方向走，方位角不变；垂直于视线走，变化最快，速率是 $1/d_i$ 。

(2) 两站合并。把两站写在一起： $\delta\varphi = J\delta x$ ，其中

$$J = \begin{pmatrix} n_1^T/d_1 \\ n_2^T/d_2 \end{pmatrix}, \det J = \frac{\sin\gamma}{d_1 d_2}.$$

共线时 $\sin\gamma = 0$ ， J 不可逆，与交会无界一致；有界时 $\delta x = J^{-1}\delta\varphi$ 。

(3) 误差盒的像。测向误差有界 $\delta\varphi_i \in [-\delta, \delta]$ ，故 $\delta\varphi$ 落在边长 2δ 的方块内，见附图 3(b)。线性映射把方块的四顶点 $(\pm\delta, \pm\delta)$ 送到

$$G \pm \delta a \pm \delta b,$$

其中 a, b 是 J^{-1} 的两列，见附图 3(c)。像集是以 G 为中心的平行四边形，两条对角线长 $2\delta\|a \pm b\|$ ，直径取较长者。

(4) 闭式。把 $\|a \pm b\|$ 展开 ($|a| = d_2/\sin\gamma$ 、 $|b| = d_1/\sin\gamma$ ，两列夹角与 γ 互补)，得

$$D_{\text{loc}} = 2\delta \max(\|a + b\|, \|a - b\|) = 2\delta \sqrt{\frac{d_1^2 + d_2^2 + 2d_1 d_2 |\cos\gamma|}{\sin^2\gamma}}.$$

平行四边形关于 G 中心对称，最小包围圆半径等于直径之半。设计阶段用 $D_{\text{loc}} \leq 40 \text{ m}$ 代替 $R_{\text{MEC}} \leq 20 \text{ m}$ ；程序终判仍走问题一的多边形。

$\delta = 1^\circ$ 时线性化误差很小，多边形与平行四边形几乎重合，看不出映射关系；图 6(a) 改用更极端的站址 ($d_1 = 800 \text{ m}$ 、 $d_2 = 280 \text{ m}$ 、 $\gamma = 38^\circ$ 一类) 并把张角放大到 5° ，两者才分开。为把“几乎重合”量化，先给口径：在 $d_1 \in \{300, 500, 800, 1100, 1400\}$ 、 $d_2/d_1 \in \{0.4, 0.7, 1.0, 1.4, 1.8\}$ 、 γ 步长 5° 的网格上逐点比较闭式直径与多边形直径的相对差——工作角域 $\gamma \in [30^\circ, 150^\circ]$ 内共 600 组，最大 0.43%、平均 0.09% (最坏出现在 $d_1 = d_2 = 500 \text{ m}$ 、 $\gamma = 30^\circ$)；把角域放宽到 25° – 155° (648 组) 最大也只有 0.63%。图 6(b) 画出相对差随 γ 的变化。

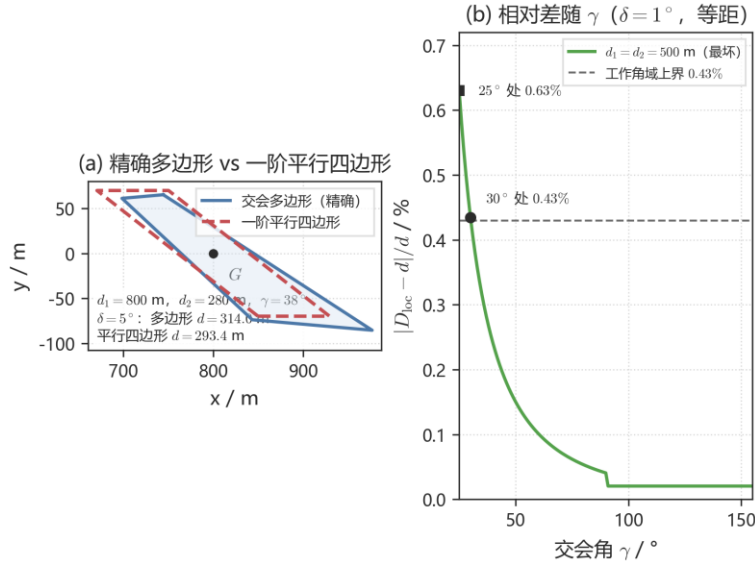


图 6 精确多边形与一阶平行四边形

图 6 (a) 极端站址下两图形可分开 (δ 放大到 5°)；(b) 等距时相对差几乎不随距离变化，故只画最坏的 $d_1 = d_2 = 500$ m。工作角域 $\gamma \geq 30^\circ$ 内最大 0.43%。

6.4 等距切片、800 m 与 $\inf D_{\text{loc}} = 2\delta D$

闭式含三个量 d_1, d_2, γ 。为单独看交会角，做一次切片：在分析里把真实距离 D 当作参数、令 $d_1 = d_2 = D$ 。机器狗并不知道 D ，切片只是“固定 D 再看 γ ”，不是假设。等距时

$$D_{\text{loc}} = 2\delta D \sqrt{\frac{2(1 + |\cos\gamma|)}{\sin^2\gamma}}.$$

$\gamma \in (0, 90^\circ]$ 时该式在 $\gamma = 90^\circ$ 取最小 $2\sqrt{2}\delta D$ 。图 7 令 $D = 800$ m， S_2 绕 G 走等距圆周： 90° 时 $D_{\text{loc}} = 39.5$ m，刚进入 40 m 筛选线； 30° 与 150° 升到 107.9 m。取 800 m 是因为等距直角交会碰到 40 m 门槛的距离是

$$D_{90} = \frac{40}{2\sqrt{2}\delta} \approx 810 \text{ m},$$

800 m 正好压在这条设计门槛上。机器狗不知道 D ， $d_1 = d_2$ 只是切片，用来单独说明 γ 的作用。

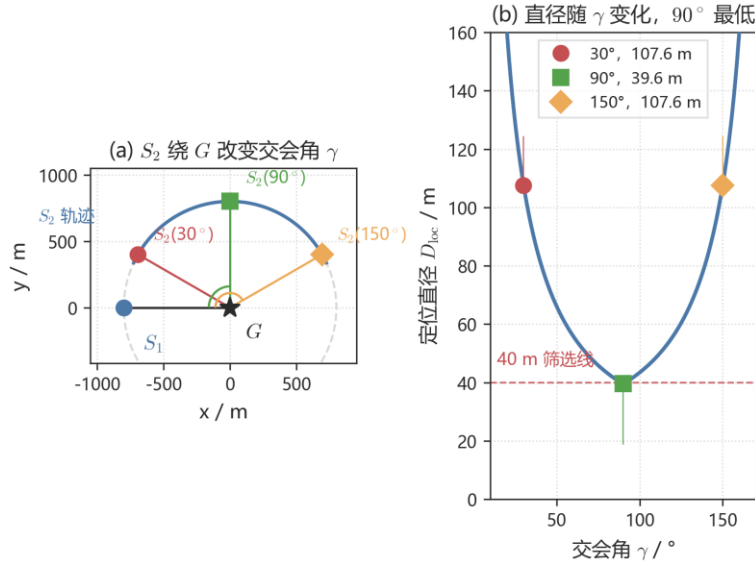


图 7 等距条件下直径随 γ 变化

图 7 $d_1 = d_2 = 800$ m。 (a) S_2 的等距轨迹； (b) 闭式曲线， 90° 最低。

等距只是切片。若允许 d_2 自由变化，直角交会下 $D_{loc} = 2\delta\sqrt{d_1^2 + d_2^2}$ ，令 $d_2 \rightarrow 0$ 得到

$$\inf_{S_2} D_{loc} = 2\delta D.$$

几何含义见附图 4(a)：第一锥在 G 处的横向全宽是 $2D\tan\delta \approx 2\delta D$ ，第二站无论怎么摆都削不掉这一条（ $\delta = 1^\circ$ 时 $2D\tan\delta$ 与 $2\delta D$ 相差约 0.01%，数值上可互换；下文界限一律用精确的 $\tan$ 写）。因此两步要把直径压到 40 m，必须

$$2D\tan\delta \leq 40 \Leftrightarrow D \leq D^* = \frac{20}{\tan 1^\circ} \approx 1146 \text{ m}.$$

更远的源两次测向不可能进入 20 m，必须准备第三次测量。附图 4(b) 在 $d_1 = 800$ m、 $\gamma = 90^\circ$ 上画出这条下界与等距点。

6.5 由 $\sin\gamma \geq 1/2$ 推出蝶形

距离未知，不能只对某一个 D 去摆 90° 。先看阈值为什么取 $1/2$ ：等距曲线上 $\gamma = 30^\circ$ 时直径已约 108 m， $\gamma = 90^\circ$ 只有 39.5 m， 30° 以下曲线更陡（图 7）；把“交会角还不太差”的门槛定在 30° ，意味着允许第二点之后的交会在 $[30^\circ, 150^\circ]$ 内摆动，超出这个范围就交给第三次切割。于是写成

$$\sin\gamma \geq s_{min} = \frac{1}{2} (\gamma \in [30^\circ, 150^\circ]).$$

$\varepsilon = 0$ 时 $\sin\gamma = |h|/d_2$ ，不等式成为 $|h| \geq \frac{1}{2}\sqrt{(D-t)^2 + h^2}$ 。两边平方、整理：

$$h^2 \geq \frac{1}{4}[(D-t)^2 + h^2] \Leftrightarrow |h| \geq \frac{s_{min}}{\sqrt{1-s_{min}^2}}|D-t| = \frac{1}{\sqrt{3}}|D-t|.$$

固定 D 时，这是 (t, h) 平面上以 $(D, 0)$ 为顶点、半张角 30° 的对顶锥的**外部**，见图 8(a)。 D 未知、在 $[D_{near}, D_{far}] = [5, 1500]$ 上取值，要求对每个 D 都成立，就是对一族对顶锥取交集，等价于侧偏盖住最远的那条边界：

$$|h| \geq \frac{s_{min}}{\sqrt{1 - s_{min}^2}} \max(|D_{far} - t|, |t - D_{near}|).$$

这就是杠杆条件，见图 8(b) 的包络构造。它在示向度两侧各给出一条斜带，两条带合起来像蝴蝶的两翼，故称蝶形候选区，见图 8(c)。再叠加上节的 $D_{loc} \leq 40$ m (K-条件)、接收约束和 $\|S_2\| \leq 1800$ ，得到候选区域 $\mathcal{C}$ 。图 9 画在场地坐标系里。空心点是关于示向度的对称点：最优解是等价类，所以题面问的是区域而不是一个点。

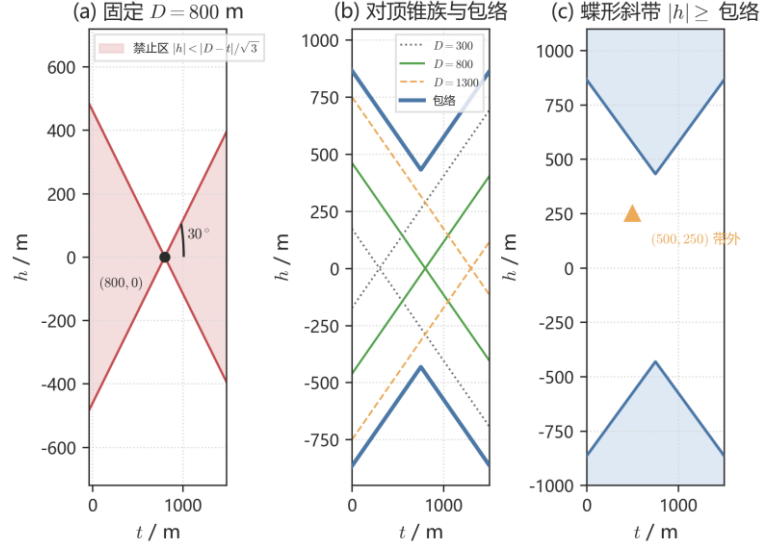


图 8 由 $\sin \gamma \geq 1/2$ 推出杠杆条件

图 8 (a) 固定 $D = 800$ m 时侧偏的禁止区（对顶锥）；(b) 一族对顶锥与包络；(c) 两条斜带=蝶形。时间点 (500, 250) 在杠杆带外。

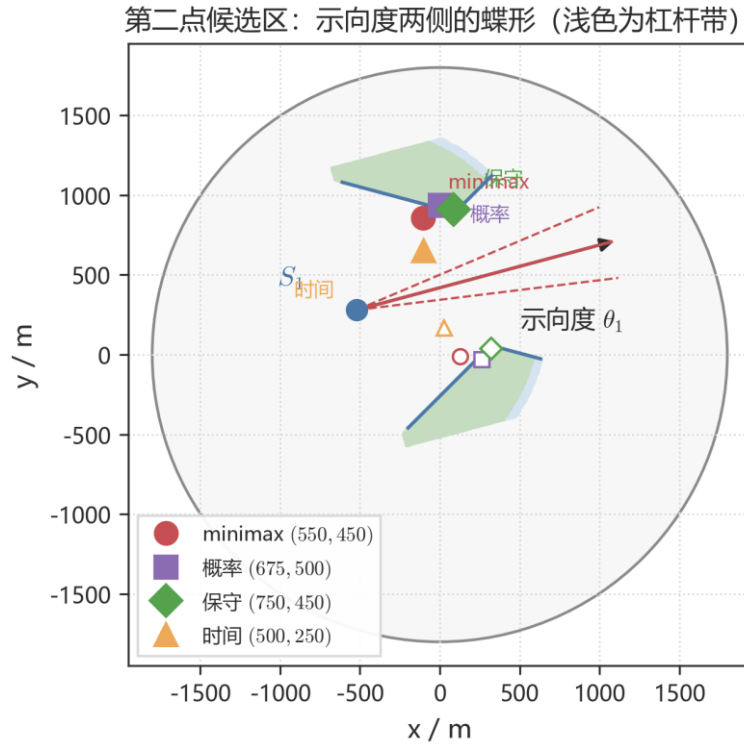


图 9 第二检测点的蝶形候选区

图 9 与图 4 同一组 S_1 、示向度。两侧翼为杠杆带；四个实心点是四种准则的采用点：(550,450)、(675,500)、(750,450)、(500,250)。

6.6 四种准则与网格

候选区只说明第二点可以落在哪里；落到哪一点，取决于对未知 D 的假设。四种准则都在同一张网格上选 (t, h) ： $t \in \{250, 275, \dots, 1000\}$ 、 $h \in \{150, 175, \dots, 700\}$ ，共 713 个格点； D 取 $[5, 1500]$ 上 80 个节点； K-条件在误差端点 $e_1, e_2 \in \{-\delta, 0, \delta\}$ 上取最坏。四种准则各给一个采用点：

1. **minimax (硬保证)**. 准则：最大化“保证前缀”——从 $D = 5$ m 起 K-条件连续成立、且用问题一的多边形核验（9 个误差端点）最坏 $R_{MEC} \leq 20$ m 的最远 D 。格点上最长前缀为 724 m (D 网格分辨率约 19 m)，在 $t \in \{525, 550\}$ 、 h 较大的一带并列；取代表点 (550,450)：多边形核验在 $D \in [5, 730]$ m（步长 2.5 m）上最坏 19.94 m，直到 735 m 才越界；移动 710 m。2. **概率最优**. 面积先验 $f_D \propto D$ （圆盘上均匀等价于半径密度正比于 D ）。同一网格上最大化两步成功的加权测度，得 (675,500)，两步成功概率 0.279，成功区间 $[232, 819]$ 。3. **保守**. 限制在杠杆带内，最小化成功样本的期望时间 $E[T]$ 。格点最优即 (750,450)： $E[T] = 318.7$ s，区间 $[402, 857]$ 。4. **时间 (实战)**. 不强制杠杆，同一 $E[T]$ 网格上最小：格点最优 (500,200) 为 264.4 s；取 (500,250) 为 267.9 s（相差 1.3%，取整到 25 m 网格，便于与代码一致），区间 $[81, 686]$ 。

$E[T]$ 的评价协议（第 3、4 条共用）： D 取 $[5, 1500]$ 上 40 个节点、权 $\propto D$ ；误差取 0；救援规则 $\rho = \max(50, 0.5L)$ ；按问题一的算法逐 D 模拟，两次不够就补第三次；仅当最终 $R_{MEC} \leq 20$ m 的样本计入平均。

图 10(a) 是 31×23 （713 点）网格上的两步成功概率，杠杆边界与四个采用点一并标出；(b) 是固定 $h = 250$ 与沿杠杆下界的 $E[T]-t$ 曲线。时间点 267.9 s、保守点 318.7 s，相对缩短 $(318.7 - 267.9)/318.7 \approx 16\%$ 。四流派在该模型里最终都能压进 20 m（成功概率 1），差距来自前期少走的路，而不是两步失败的惩罚。期望测量次数 2.7–2.8。

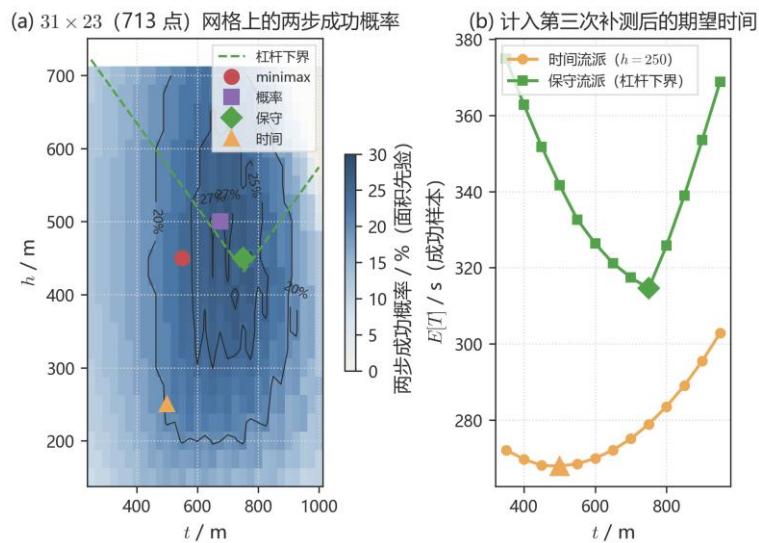


图 10 四准则的网格来源

图 10 (a) 31×23 网格（713 点）上的两步成功概率热图与四个采用点；(b) 固定 $h = 250$ 与杠杆下界的 $E[T]$ 随 t 变化，三角形、菱形为采用点。

表 2 第二检测点的四种设计（区间由 K-条件/多边形核验给出； $E[T]$ 仅对成功样本）

流派	准则	采用点 (t, h)	保证 / 成功区间	$E[T \text{成功}]$
minimax 硬保证	保证前缀最长（K-条件+多边形核验）	(550, 450)	[5, 724]	307 s
概率最优	面积先验下两步概率最大	(675, 500)	[232, 819]	325 s
保守流派	杠杆带内 $E[T]$ 最小	(750, 450)	[402, 857]	319 s
时间流派（实战）	无约束 $E[T]$ 最小	(500, 250)	[81, 686]	268 s

问题三采用时间点；需要硬保证则改 (550, 450)。问题四因定向波束改用小侧偏，不再沿用 (500, 250)。

6.7 两次不够时沿长轴再切

每个测点沿视线法向把区域压到约 $2\delta d_i$ 宽。若两次后 $R_{\text{MEC}} > 20$ m，新测点取当前长轴法向、朝来路一侧：

$$S_{\text{new}} = C_{\text{MEC}} + \rho \hat{n}, \rho = \max(50, 0.5L),$$

L 为当前直径。附图 5 是 $D = 1500$ m、第二点 (700, 462) 的全局位置：两次后直径 164.5 m。图 11 是局部：第三次后直径 33.1 m， $R_{\text{MEC}} = 16.5$ m，落入 20 m 清除圆。

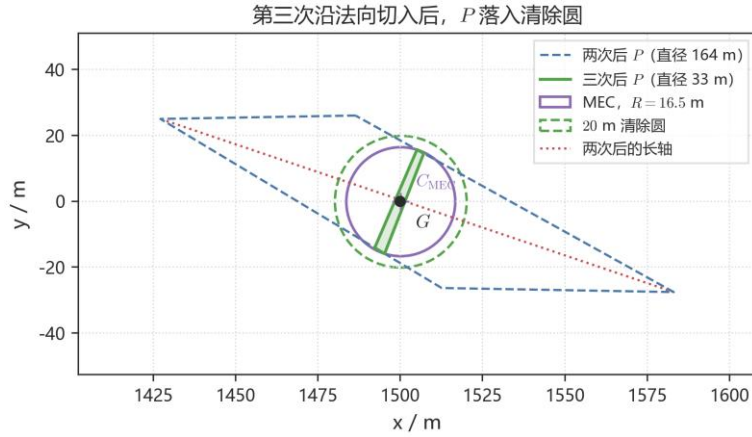


图 11 第三次切入后落入清除圆

图 11 蓝虚线为两次后的 P ，绿色为三次后的 P ，虚线圆为 20 m 清除圆。

单目标时间 $T = \sum \|S_{k+1} - S_k\|/5 + 5n_{\text{meas}} + 5$ 。仅当最终半径 ≤ 20 m 时计入平均。面积先验下成功样本的 $E[T]$ 见附图 6，时间流派谷底在 $t = 500$ m，用于问题三。

主要核对：设计角域内闭式最大相对误差 0.43%； $D^* = 1146$ m；采用点 (550, 450) 的核验最坏 MEC 19.94 m；(500, 250) 相对 (750, 450) 缩短约 16%。

七、问题三的模型建立与求解

7.1 模型建立

7.1.1 七点覆盖网与判空

全向源无信号只有两种可能：该频道已无未清源，或检测点已超出接收半径。一组监听点若能用半径 1000 m 的接收圆盘盖住整个 1800 m 圆域，某频道在这些点全无信号即可判空。未知总数因此变成有限点上的覆盖问题。

由等圆覆盖的 Kershner 下界 [6]， N 个半径 1000 m 的圆盘覆盖半径 1800 m 的目标圆至少需要 $N \geq 1.209 \times (1800/1000)^2 \approx 3.92$ ，故不少于 4 点。该渐近下界在小 N 下偏松：数值核验表明 4、5、6 点最优构型的覆盖半径分别为 1271.6、1110.6、1037.9 m，均无法保证 1000 m 覆盖。可行方案从七点起。

监听点取原点，再加上半径 r 圆周上均匀 n 个点。原点管住 $\|X\| \leq 1000$ ；外缘最差点出现在相邻环点的角平分线上，该点到最近环站的距离满足

$$d(r)^2 = 1800^2 + r^2 - 2 \cdot 1800 \cdot r \cdot \cos \frac{180^\circ}{n}.$$

要求 $d(r) \leq 1000$ 有解，得到 $\cos(\pi/n) \geq 0.8315$ ，即 $n \geq 5.33$ 。因此“原点+均匀环”这一族至少 6 个环点，共 7 点。 $n = 6$ 时令 $d(r) = 1000$ ，较小正根 $r^* \approx 1123$ m；取 $r = 1150$ m，径向余量约 27 m，最坏距离

$$d(1150) = \sqrt{1800^2 + 1150^2 - 2 \cdot 1800 \cdot 1150 \cdot \cos 30^\circ} \approx 988.5 \text{ m} < 1000 \text{ m}.$$

余量 $1000 - 988.5 = 11.5$ m。沿半径方向，边界点到正对环站只有 $1800 - 1150 = 650$ m。检测律是“距离不超过接收半径 R 就能听到”，题面 $R \in [1000, 1500]$ m 且接口不返回具体值。 R 越小越难听到，按最坏值 1000 m 设计的覆盖网对所有 $R \geq 1000$ m 都成立：若七点均无信号，则源到每个站都超过 1000 m，与“圆域内任一点到某站不超过 988.5 m”矛盾，故该频道无未清源。证书成立的前提是机器狗确实走遍这 7 个点。

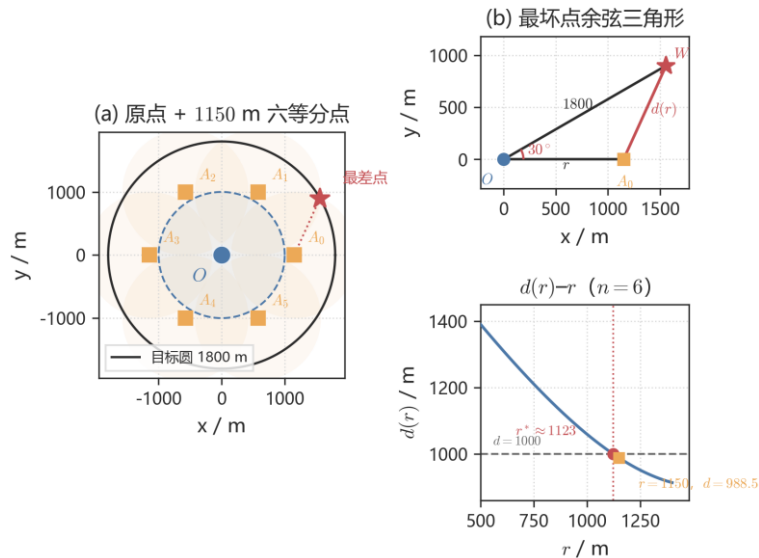


图 12 问题三覆盖网

图 12 (a) 原点加 1150 m 六等分点；(b) 角平分线处最坏点的余弦三角形与 $d(r)$ - r 曲线： $r^* \approx 1123$ m，取 1150 m，最坏距离 988.5 m。

7.1.2 频道账本与三阶段调度

每个频道独立维护状态：`unknown` → `heard` → `cleared`，或 `unknown` → `empty`。算法分三段。

1. **原点扫频**. 不移动，频道 1–20 各测一次，虚拟耗时 $20 \times 5 + 19 \times 1 = 119$ s。听到的源满足 $\|G\| \leq R \leq 1500$ ；近距源就地清除。2. **追击已听频道**. 从听测点走问题二的时间赌注 (500, 250) 再测；两锥交会后算最小包围圆，半径 > 20 m 则沿长轴法向再下一站， ≤ 20 m 则到圆心清除。第二点若因出界或坐标取整返回无信号，把“距该点超过 1000 m”纳入后续选点（换侧并加大纵向赌注），不写成第二点必然有信号。3. **环网补漏**. 从当前点最近的环站起绕一圈。每点先把剩余频道扫完再搬家；七点都无信号则标 `empty`。环上新听到的源，若专程去第二点再进圈的额外路程超过 300 m 则挂起，整圈走完再就近追。程序读取 `/enter` 返回的剩余现实时间，墙钟不足时停止补网。

调度上还做三件事：能交会且 $R_{\text{MEC}} \leq 20$ m 的源优先就近清除；移动途经某停点时，顺路复测仍只有一站、且相对第一站接近时间赌注的频道，已进 20 m 且顺路的源就地收割；清满 16 个（题面源数上界）即停。交会调用问题一半平面算法，最小包围圆与长轴调用问题二几何模块。

停机：已清除数达到 16，或二十个频道都是 `cleared` 或 `empty`。

7.1.3 时间目标

记第 i 次测量位置为 S_i 、测量次数为 N_{meas} 、换频次数为 N_{switch} 。总虚拟时间

$$T = \sum_i \frac{\|S_{i+1} - S_i\|}{5} + 5N_{\text{meas}} + N_{\text{switch}} + T_{\text{clear}}.$$

约束包括：区域约束；示向度误差不超过 1° ；清除约束 $R_k \leq 20$ m 才允许清除；覆盖判空约束要求频道 k 在七点骨架上均无信号方可判空。本问不另起几何模型，新工作是覆盖网与三阶段衔接。

7.2 模型求解

7.2.1 官方模拟器演练

程序入口为 `python q3/run\_q3.py`。单局案例 `WU4D-52HU-EUR9-Y4QR`：原点听测清除 8 个、环网补漏清除 6 个，覆盖判空 6 个，清除/失败/剩余为 14/0/0，虚拟时间 5189 s、平均 371 s/个。该局移动 4563 s (87.9%)，检测 470 s，切频道 86 s，清除 70 s。多数源两次测量即可清；较远的源第三次长轴切割后最小包围圆降到 10 m 量级。轨迹与进度见图 13、附图 7。

覆盖网策略在同一官方模拟器上开展 19 局随机案例实测，源数 13 ± 2 (10–16)，全部 19/19 全清，平均 352 ± 31 s/个 (290–402 s)，虚拟总时间 4480 ± 426 s，见表 3。按 19 局均值拆分，移动仍占八成以上。源越多，原点扫频与环网固定开销摊得越薄，单源平均才稳定在约 350 s。逐局案例编码与时间见附录 D。

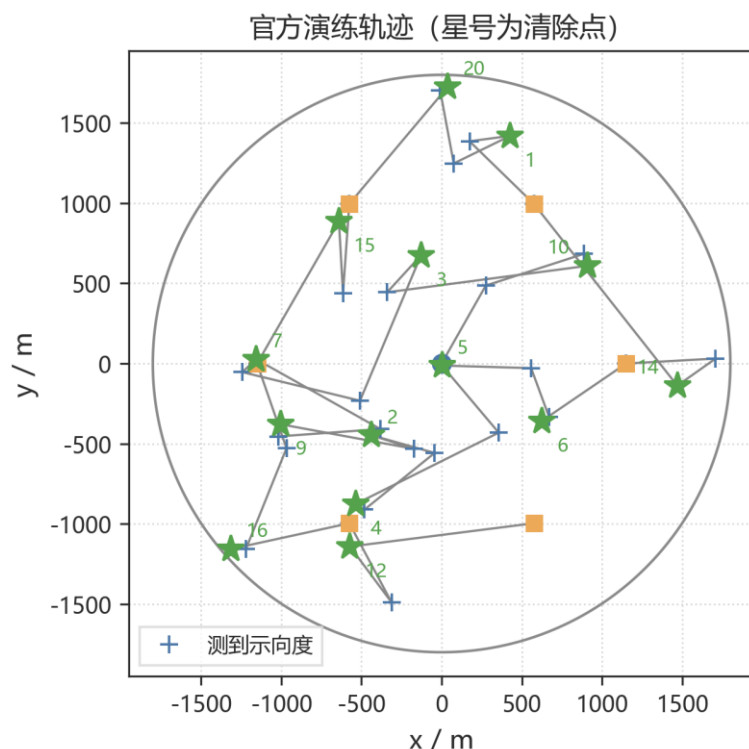


图 13 问题三官方演练轨迹

图 13 案例 `WU4D-52HU-EUR9-Y4QR`。星号为清除点，数字为频道。

压低单源时间的关键是减少空驶，而不是再减测量次数。

表 3 问题三官方模拟器 19 局演练汇总（覆盖网策略）

局数	全清	源数	平均时间 (s/个)	虚拟总时间 (s)
19	19/19	13 ± 2	352 ± 31	4480 ± 426

7.2.2 本地策略对照

在同一批 12 个随机世界上，把第二点赌注换成另外两个流派、并关掉顺路复测作对照，四种策略均 12/12 全清，见表 4 与附图 8。时间赌注加停点复用最低（314 s/个）；关掉复用后升至 326 s；minimax 与保守分别升至 336 s 与 332 s。因此问题三采用 (500,250) 并保留停点复用，与问题二单目标仿真的排序一致。表中 minimax 点 (550,498) 为该对照网格上的取值；问题二采用点为 (550,450)。

配对设计下，“时间+复用”对其余三组无一局反超，配对差均值约 12、22、18 s（相对约 4%–7%）。移动成本占主导，差值方向稳定。

表 4 问题三本地策略对照（12 局同一批世界）

策略	第二点 (t, h)	停点复用	全清	平均 (s/个)
时间+复用（采用）	(500, 250)	是	12/12	314
时间、无复用	(500, 250)	否	12/12	326
minimax	(550, 498)	是	12/12	336
保守	(750, 450)	是	12/12	332

7.2.3 策略无关的时间下界

任何“确保全部清除”的策略至少承担三项不可压缩的代价：(i) 原点扫频 119 s；(ii) 每个源至少 2 次检测（10 s）加 1 次清除（5 s）；(iii) 从原点出发访问全部清除点的最短路径，其长度不小于含原点最小生成树（MST）的长度，除以 5 m/s 即移动下界。用 19 局覆盖网实测日志中 `clear` 成功指令的清除点坐标离线计算，问题三下界约为 136 s/个，实测均值 352 s/个距下界约 +159%。剩余差距主要由测向定位所需的额外绕行与多次检测构成，而不是调度还能轻易压掉的空转。

八、问题四的模型建立与求解

8.1 模型建立

8.1.1 环抱判据

定向源只在发射方向两侧各 90° 辐射，可检区域是“以 G 为顶点、张角 180° 的闭半平面”与接收圆盘的交。“无信号”因此不再等于超距：源可能就在附近，只是波束朝另一边。问题三的圆盘覆盖直接失效。

要把任意朝向的定向源找出来，监听点不能只满足距离覆盖，还要在各个方向把候选位置围住。

命题（环抱判据）. 频道为空，当且仅当对目标圆域内任意 G 有

$$\max_{G \in \text{disk}(1800)} \max\text{gap}(N(G)) \leq 180^\circ,$$

其中 $N(G)$ 是与 G 距离不超过 1000 m 的探针集合， $\max\text{gap}$ 是它们绕 G 的最大方位间隙。

充分性：若最大间隙不超过 180°，则任意 180° 闭半平面内至少含一个探针，故无论朝向如何都能被听到；全向源是其特例。必要性：若某 G 的 $\max\text{gap}(G) > 180^\circ$ ，取发射方向背对那条空隙，则全部探针都落在盲侧，该朝向可能漏检。部分公开思路要求任意 1000 m 邻域内存在方位互差 120° 的三个观测点，那是本判据的特例（三互差 120° 的点最大间隙为 120°），反之不必。几何对照见附图 9。

8.1.2 圆外停点是必要的

源在 $G = (1800, 0)$ 、朝向 $(1, 0)$ 时，可检半平面为 $x \geq 1800$ ，与圆域的交几乎只剩 G 自身，圆内探针对此源的 $\max\text{gap} = 360^\circ$ ，见图 15。因此只用问题三的七点网，或把外圈停在 1750 m 的圆内实用网，都不能给出完整保证。题面允许检测坐标超出 1800 m 圆域，证书必须包含圆外停点。

8.1.3 26 点环抱骨架

取

$$\mathcal{S} = \text{ring}(400, 6) \cup \text{ring}(1300, 8) \cup \text{ring}(1900, 12),$$

共 26 点：内环 400 m 六点、中环 1300 m 八点、外环 1900 m 十二点，外环在圆域之外，用来照亮圆缘外向源，见图 14。20–25 点的穷尽搜索均出现最坏间隙大于 180°；外环半径取 1850 m 时圆缘外向源即失效，故 1900 m × 12 是必要的。走遍这 26 点的开路长度约 18758 m，合 3752 s，是判空移动的刚性下界。

在半径方向 19 层、每层至多 96 个方位的 1825 个网格点上核验： $\mathcal{S}$ 的最坏 $\max\text{gap}$ 恰为 180.0°，全部通过；圆缘点 $(1800, 0)$ 处为 99.2°。对照见表 5。

表 5 网格上的最大方位间隙 ($R = 1000\text{ m}$)

探针网	点数	最坏间隙	未通过格点	圆缘间隙
七点覆盖	7	360°	1069/1825	360°
圆内实用环绕	25	239.6°	174/1825	216.0°
26 点环抱 (采用)	26	180.0°	0/1825	99.2°

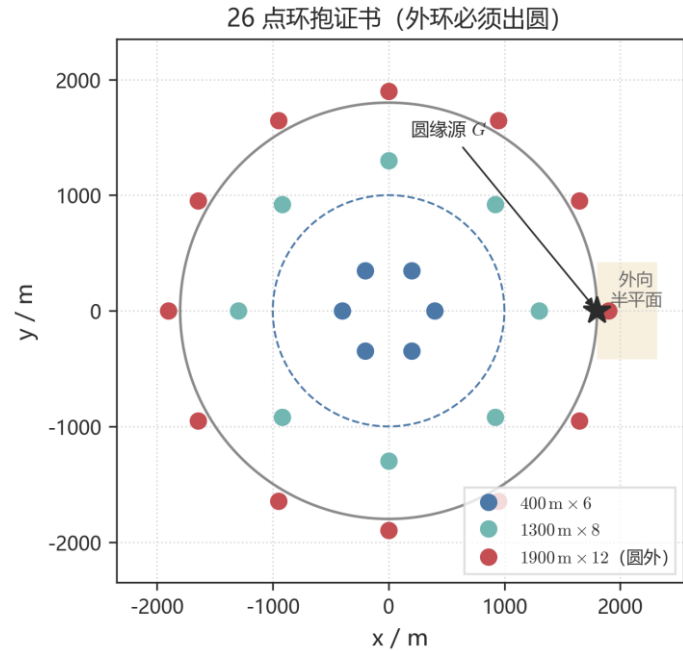


图 14 26 点环抱证书

图 14 内环 400 m、中环 1300 m、外环 1900 m (出圆)。阴影为圆缘源的外向半平面。

在更密的 60697 点采样上 (含边界 0.25° 细网), 一般位置上的最坏间隙为 174.84° , 距失效阈值 180° 尚有约 5.2° 的角度余量; 180.0° 只在一类测度零的退化构型上取到——当候选源 G 恰落在某对探针的连线上时, 两探针绕 G 严格对径, 间隙正好 180° 。题面规定定向源辐射张角为两侧各 90° (含), 合计恰为 180° 的闭扇区, 此时边界点仍可被照亮, 故 180° 仍属通过; 若实际有效扇区略窄于 175° , 一般位置将开始出现失效候选点。

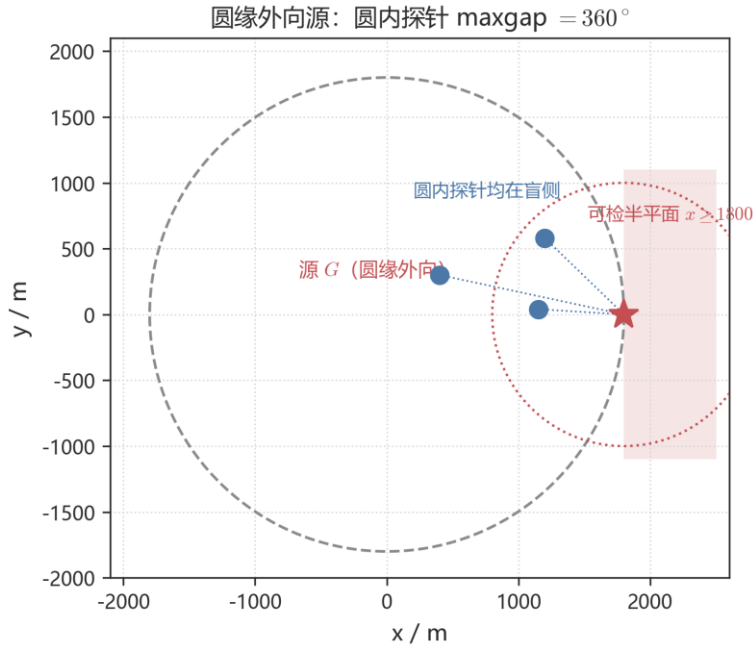


图 15 圆缘外向源使圆内探针全部落在盲侧

图 15 源在 $G = (1800, 0)$ 、朝向 $(1, 0)$ 时，可检半平面为 $x \geq 1800$ ，与圆域几乎只交于 G 自身。

8.1.4 合并巡回与定向源的第二检测点

策略分成三段，把证书扫描、定位与清除放在同一条路上，避免先走完探针再回头。

1. **原点扫频**. 频道 1–20 各测一次，虚拟 119 s。2. **合并巡回**. 节点集合 = 尚未访问的证书点 U 已检出未清除源的最小包围圆圆心。节点集合一变，就以当前位置为起点对剩余节点做最近邻加 2-opt。到证书点：把该点仍未知的频道扫完，并顺路补测最不确定的已听频道；到源节点：按预测点测量、尝试清除，失败则追踪。3. **收尾**. 对残余源做兜底巡回，追踪预算逐轮放大。理想情况下这一段为零。

第一站测到示向度 θ_1 后，第二点仍写成 $S_2 = S_1 + t\mathbf{e}_\theta + h\mathbf{e}_\theta^\perp$ ，但 (t, h) 改用小侧偏组合 $(260, \pm 80)$ 、 $(160, \pm 55)$ 、 $(360, \pm 45)$ 等，优先取离机器狗更近且留在已点亮一侧的那个。沿示向度推进时步长随交会区缩小。若下一点无信号，说明走过了源或掉进背向，则在“最近一次有效测站到失联点”的线段上搜索清除。交会后最小包围圆半径 ≤ 20 m 则到圆心清除。

定向追击另有三条规则：点亮侧优先（候选点偏向已有测点所在半平面）；黑侧惩罚（无信号方位不再反复撞）；长条端采样（最小包围圆半径较大时沿主轴两端取样，避免只围中心）。工程上不强制每次走满全部 26 点：优先追击已听源，证书巡回作兜底判空，同时监控 `/enter` 返回的剩余墙钟。

8.2 模型求解

8.2.1 官方模拟器演练

早期演练采用圆内实用环绕（七点覆盖后走 1750 m 外圈）。确定 26 点环抱证书后，环抱证书加合并巡回在同一官方模拟器上开展 17 局随机案例实测，实现 17/17 全清、平均 593 ± 95 s/个（411–740 s），总时间 7317 ± 644 s，见表 6(a)。正式测试三

局均值 600.26 s/个，与该演练一致。问题四平均时间明显高于问题三。附图 10 对照问题三 19 局演练与问题四正式测试。

同一套程序在 24 个确定性 mock 世界上 24/24 全清，平均 600.1 ± 83.2 s/个。时间账见表 6(c)：证书巡回移动约 3752 s 是判空地板，约占一半；测量约 21%，绕行接近源约 17%。把证书巡回、理想绕行与每源两次测量加起来，单源下界大约 472 s，实测 600 s 距该下界约 27%。

表 6 问题四官方模拟器实测与策略对照

(a) 官方模拟器实测汇总（最终版策略 v3）

测试来源	局数	全清	平均时间 (s/个)	总时间 (s)
最终版 v3 实测	17	17/17	593 ± 95 (411–740)	7317 ± 644
mock 24 种子	24	24/24	600.1 ± 83.2	—

(c) mock 24 种子平均时间账（合计约 7580 s， $T_{\text{avg}} \approx 592$ s/个）

分项	时间 (s)	占比
原点扫频	119	2%
证书巡回移动	~3752	49%
绕行接近源	~1250	17%
收尾	~460	6%
测量	~1615	21%
换频	~300	4%

(b) 本地策略对照（混合场景 / 圆缘外向应力场景）

策略	混合全清	混合平均 (s/个)	外向全清	外向平均 (s/个)
七点覆盖	1/8	409	0/6	455
实用环绕+早停	6/8	648	0/6	702
实用环绕走满	6/8	775	0/6	811
26 点环抱（采用）	7/8	681	5/6	720

由表 6(b) 与图 16，26 点环抱网是唯一能在圆缘外向应力下稳定全清的策略（5/6）；七点覆盖与实用环绕均无法检出圆缘外向源。环抱网两例未全清的源都被听到，失败发生在定向追击的末轮楔形清除而非证书漏检。

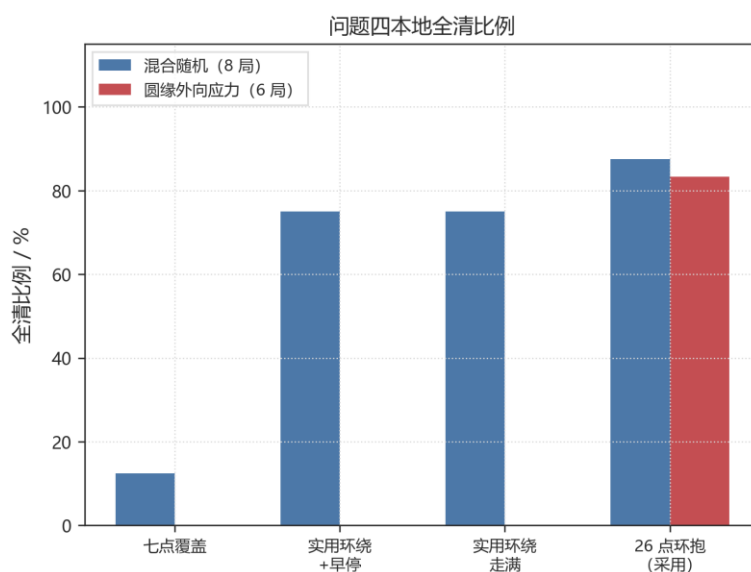


图 16 问题四本地全清比例

图 16 混合随机与圆缘外向应力下，四点策略的全清比例。只有 26 点环抱在外向场景仍有清除。

8.2.2 方位近共线退化与兜底

定向源漏检只有一类来源，即“方位近共线”退化几何：多次有效测站几乎共线，定位区域退化为细长条，最小包围圆半径始终降不到 20 m。演练日志复盘：某局频道 18 的 39 次检测中 4 次得到示向度，全部来自 3 个近乎共线的站位（读数彼此只差 2.3° ）；另一局频道 10 的 6 个有效站位集中在一段短弧上，示向度跨度仅 1.6° 。策略因此加入几何中心兜底：只要已积累方位、候选多边形几何中心可算，就强制到该中心尝试一次清除，把“区域有界但半径略超 20 m”的部分救回。机制加入后的验证演练中，此类漏检未再出现。

8.2.3 时间下界

沿用问题三的策略无关下界（固定开销 119 s + 每源 2 次检测 10 s + 1 次清除 5 s + 含原点 MST 移动下界），用 17 局最终版实测日志的清除点坐标离线计算，问题四下界约为 151 s/个，实测均值 593 s/个距下界约 +293%。与问题三（+159%）相比差距更大，这是定向源的本质代价：一次阴性观测几乎不携带位置信息，每源必须付出更多测向绕行；这一额外代价在松弛下界中未被计入。证书巡回本身还有 3752 s 的几何地板，源数偏少时摊薄不足，单源平均会被抬高。

8.2.4 正式测试

两次正式测试均使用与最终版完全相同的冻结参数。问题三 3 次正式测试均 100% 清除，平均定位清除时间 259.48–323.51 s/个、均值 285.34 s/个，程序运行时间 1.3–1.7 s。问题四 3 次正式测试均 100% 清除，平均 518.78–698.52 s/个、均值 600.26 s/个，程序运行时间 3.7–4.6 s。见表 7。

表 7 正式测试结果（官方模拟器）

(a) 问题三

案例编码	清除个数	平均时间 (s/个)	程序运行时间 (ms)
'EA6G-8KH7-MD9Y-RM5C'	11	323.51	1589

案例编码	清除个数	平均时间 (s/个)	程序运行时间 (ms)
`G9DQ-EJCW-SJYN-FWW6`	16	259.48	1743
`DNNF-446T-J6GT-8A29`	15	273.03	1294

(b) 问题四

案例编码	清除个数	平均时间 (s/个)	程序运行时间 (ms)
`2CS6-PYGG-YEXJ-CNMX`	12	583.47	3910
`HDGN-FSKE-BGQ4-HXMH`	13	518.78	3750
`GGJ8-K8GP-DBCX-QET9`	10	698.52	4619

九、模型评价

9.1 模型的优点

四问共用有界误差下的半平面交会：问题一在顶点集上取直径 d 、用中点检验覆盖，清除看 $R_{MEC} \leq 20$ m；问题二先给出蝶形候选区，再按假设选点；问题三用七点覆盖网把未知源数变成有限监听；问题四把覆盖改成环抱，26 点骨架含圆外停点。

判定都落在可核验的量上。等边三角形说明直径圆覆盖不能省；算例 G 说明直径圆盖不住仍可能 $R_{MEC} \leq 20$ m。问题二四个采用点由同一张网格给出；闭式直径在工作角域内相对误差不超过 0.43%。覆盖网最坏间隙 988.5 m，环抱证书粗网格最坏恰为 180.0° 。正式测试问题三、四均 100% 清除，平均 285.34 s/个与 600.26 s/个。

9.2 模型的缺点与改进

表 8 列出主要缺点、已采取的应对与可执行的改进。表中每一条都能在正文或代码里找到对应边界。

表 8 模型缺点、边界与改进

缺点	定量边界	已采取的应对	改进
时间赌注 (500,250) 不满足杠杆条件	$D > D^* \approx 1146$ m 时 两次不能压进 20 m	第三次长轴补测；面积先 验下约 79% 的源走第三 次	对 $D > D^*$ 强制第二次 后立即补测
覆盖网、环抱证书依赖走 遍停点	七点或 26 点未走满则不 能判空	未走遍的频道不标 'empty'；墙钟不足时停网	把“走遍证书点”写成显式 调度约束
定向追击的方位近共线 环抱证书余量薄	示向度跨度可小至 1.6° 粗网格最坏恰 180° ，一 般位置约 5.2°	几何中心兜底清除 60697 点加密采样仍通 过	沿失联线段折线加密 按扇区张角 2θ 重构造骨 架
$E[T]$ 是成功样本上的条 件期望	只计入 $R_{MEC} \leq 20$ m 的样本	正文写明口径	报告失败样本的时间分布
minimax 保证前缀只到 724 m	更远的源依赖第三次切割	救援规则 $\rho =$ $\max(50, 0.5L)$	联合优化两步区间与路程

关掉覆盖网补漏后，问题三清除率由 100% 跌至 56.3%；把问题四证书换成七点网或圆内环绕，清除率分别降至 94.1% 与 96.0%，见表 9。几何中心兜底在 24 个离

线世界上关闭与否都是 100% 全清，该批未触发它所针对的退化几何，其作用在官方演练中体现。

9.3 适用边界

集合定位适用于误差有界、分布未知的测向问题。目标区域远大于 1800 m 而接收半径仍为 1000 m 时，证书点数随面积增长，巡回可能超过时间预算。定向源张角明显窄于 180° 时，26 点骨架的零余量保证失效，须按扇区张角重造。机器狗不允许驶出目标区域时，圆缘外向源在几何上无法判空。

消融实验见表 9。

表 9 消融实验（本地数值仿真，同批随机世界）

问	配置	清除率	全清局	平均时间 (s/个)
三	完整方法	100%	30/30	317.9 ± 38.0
三	关覆盖网 补漏	56.3%	0/30	232.8
三	关长轴补 测	100%	30/30	327.7
四	完整方法	100%	24/24	601.2 ± 86.9
四	证书换成 七点网	94.1%	10/24	695.1
四	证书换成 圆内环绕	96.0%	13/24	1217.9
四	关几何中 心兜底	100%	24/24	600.1 ± 83.2

参考文献

- [1] WANG S, et al. Optimal geometry and motion coordination for multisensor target tracking with bearings-only measurements[J]. Sensors, 2023, 23(13): 4347.
- [2] BISHOP A N, FIDAN B, ANDERSON B D O, et al. Optimality analysis of sensor-target localization geometries[J]. Automatica, 2010, 46(6): 1169-1177.
- [3] OSHMAN Y, DAVIDSON P. Optimization of observer trajectories for bearings-only target localization[J]. IEEE Transactions on Aerospace and Electronic Systems, 1999, 35(3): 892-902.
- [4] BERTSIMAS D J, VAN RYZIN G. A stochastic and dynamic vehicle routing problem in the Euclidean plane[J]. Operations Research, 1991, 39(4): 601-615.
- [5] JUNG H. Über die kleinste Kugel, die eine räumliche Figur einschließt[J]. Journal für die reine und angewandte Mathematik, 1901, 123: 241-257.
- [6] KERSHNER R. The number of circles covering a set[J]. American Journal of Mathematics, 1939, 61(3): 665-671.
- [7] WANG Y, CAO G. On full-view coverage in camera sensor networks[C]//Proceedings of IEEE INFOCOM. 2011.

附录

正文与文献的对应：问题二对照 [1][2] 关于交会角 90° 最优、测点宜近的几何结论；候选区域形态对照 [1] 的几何不变性。问题三的追击顺序属于最小延迟路由的一类在线贪心，见 [4]；第三次测量沿长轴切入与 [3] 一致。七点覆盖网点数下界对照 [6]。问题四环抱判据与全视场覆盖同构，见 [7]。Jung 界与最小包围圆结构直接引用 [5]。Fisher 信息只作平行对照。

附录 A 问题二公式速查

用途	公式
交会角正弦	$\sin\gamma = h\cos\varepsilon - t\sin\varepsilon /d_2$
定位直径	$D_{\text{loc}} = 2\delta \sqrt{(d_1^2 + d_2^2 + 2d_1d_2 \cos\gamma)/\sin^2\gamma}$
两步成功（设计）	上式根号内 $\leq K \approx 1146 \text{ m}$
minimax 核验	采用点 (550, 450): $D \in [5, 730] \text{ m}$ 上多边形 $R_{\text{MEC}} \leq 19.94 \text{ m}$
杠杆条件	$ h \geq \frac{s_{\min}}{\sqrt{1 - s_{\min}^2}} \max(D_{\text{far}} - t , t - D_{\text{near}})$
必要距离界	$D \leq D^* = 20/\tan 1^\circ \approx 1146 \text{ m}$
全局变换	$S_2 = (x_1 + t\cos\theta_1 - h\sin\theta_1, y_1 + t\sin\theta_1 + h\cos\theta_1)$
环抱判空	$\max_G \max \text{gap}(N(G)) \leq 180^\circ, N(G) = \{p: \ p - G\ \leq 1000\}$
26 点骨架	$\text{ring}(400, 6) \cup \text{ring}(1300, 8) \cup \text{ring}(1900, 12)$

附录 B 复现入口

文件	内容
`q1/solver.py`	问题一半平面交会、直径、圆覆盖与最小包围圆
`q2/geometry.py`	问题二楔形、直径、最小包围圆、K-闭式与单目标仿真
`q2/derive_check.py`	闭式误差对照、 D^* 、四准则网格与 minimax 核验
`q3/run_q3.py`	问题三机器狗：原点扫频、追击、环网补漏
`q4/run_q4.py`	问题四机器狗：环抱证书与定向追击
`q4/ledger.py`	问题四探针集合与环抱判空
`q4/policy.py`	问题四 Policy3：扫频、合并巡回、收尾
`q4/score.py`	问题四 24 种子 mock 基准
`q4/skeleton_final.json`	26 点证书坐标
`robot.py`	问题四模拟器接口

附录 C 问题一算例构型

坐标用于核验算法，不是题面数据；示向度均指向示意源， $\delta = 1^\circ$ 。

算例	源 G	检测点 S_i	备注
A	(80, 220)	$S_1 = (-920, -480)$	单站，公共区域无界
B	(80, 220)	$S_1 =$ $(-920, -480)$ 、 $S_2 = (980, -420)$	图 1(b)、图 2(a)
C	(80, 220)	再加 $S_3 =$ $(-40, 1180)$	三站切入，六边形
D	—	$S_1 = (0, 0)$ 、 $S_2 =$ $(200, 0)$ ，示向度 90° 与 270°	相背，交会为空
E	—	等边三角形 $(0, 0)$ 、 $(2, 0)$ 、 $(1, \sqrt{3})$	几何核验
F	—	$S_1 = (0, 0)$ 、 $S_2 =$ $(200, 0)$ ，示向度同为 90°	同向，无界
G	(0, 0)	$S_1 = (800, 0)$ 、 $S_2 =$ $(-400, 400\sqrt{3})$ 、 $S_3 =$ $(-400, -400\sqrt{3})$	图 3

附录 D 问题三官方 19 局逐局结果

口径：平均时间 = 虚拟时间 ÷ 清除数；全清 = 清除数与真值源数一致且失败列表为空。数据见 `paper/官方演练\_Q3Q4\_最新.md`。

局	案例	源数	清除	判空	虚拟时间 (s)	平均 (s/个)
1	`KN4P-5R5W-BQXQ-TCN7`	12	12	8	3927	327
2	`PMBJ-SQ7V-HTJG-B6A2`	10	10	10	3798	380
3	`FTMA-VUGS-K8DX-Z3R7`	14	14	6	4604	329
4	`BC9E-SSG6-9JSQ-W92H`	13	13	7	4409	339
5	`F5TU-3NH5-SHDQ-AFGR`	15	15	5	5328	355
6	`VEEJ-W4CJ-F8SZ-H5FE`	16	16	4	4633	290
7	`VAFN-E33C-43CS-D983`	12	12	8	4418	368
8	`SK2Q-225V-FGR8-52QT`	12	12	8	4826	402
9	`QXAX-DMBS-CJS8-PCED`	15	15	5	4783	319

局	案例	源数	清除	判空	虚拟时间 (s)	平均 (s/个)
10	`A3Y2- PHNC-T322- DMGY`	16	16	4	5255	328
11	`AUBF- WSH8-EFPP- HPJS`	11	11	9	4369	397
12	`CTNN- 9W9K-CKB7- ATWA`	10	10	10	3658	366
13	`25HG- MHCG- H3PT-6478`	12	12	8	4610	384
14	`47DE- MRWU- YAAX-7TE6`	12	12	8	4578	382
15	`7KGY- T7RF-4GZK- SSVG`	13	13	7	4424	340
16	`QAPC- YE2V-PN6H- 563V`	11	11	9	4239	385
17	`VS3W- HZBE-RSXX- GQMP`	13	13	7	3974	306
18	`3XER- PVDM-2RB8- CXAK`	13	13	7	4565	351
19	`NYPY- CNZ7-4E9G- 2BRM`	14	14	6	4713	337

合计：19/19 全清，平均 352 ± 31 s/个。

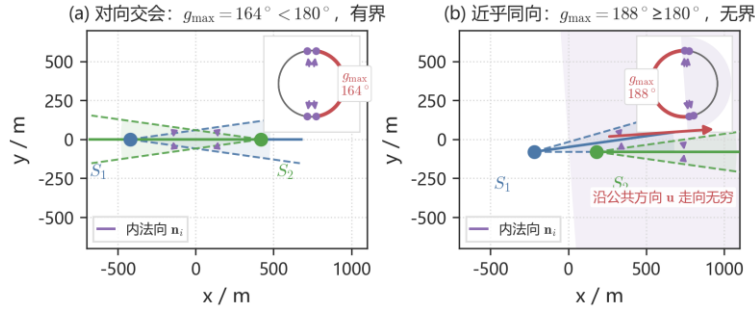
附录 E 问题四正式测试逐局

三次正式测试均 `outer\_verified`。清除失败次数（4/6/1）为同一源螺旋逼近的重复尝试，最终均清除，零漏检。

局	案例	清除个数	虚拟总时间 (s)	平均 (s/个)	程序 (ms)
1	`2CS6-PYGG- YEXJ-CNMX`	12	7002	583.47	3910
2	`HDGN-FSKE- BGQ4-HXMH`	13	6744	518.78	3750
3	`GGJ8-K8GP- DBCX-QET9`	10	6985	698.52	4619

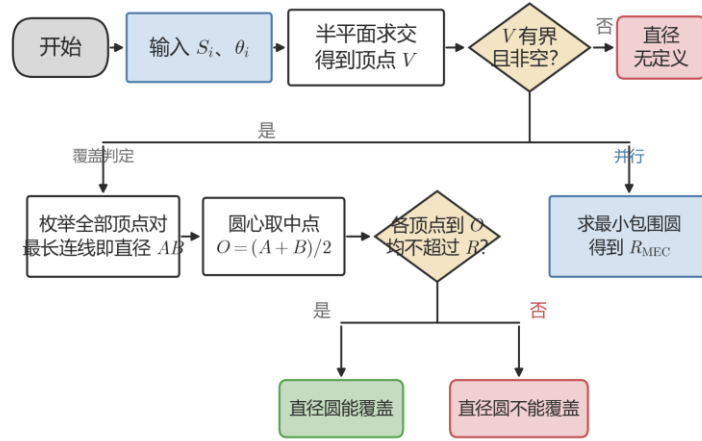
合计：3/3 全清，平均 600.26 s/个。

附录 F 正文未展开的附图



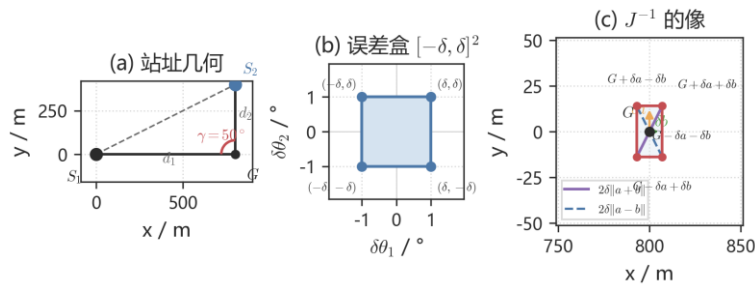
附图 1 法向角间隙与有界性

附图 1 主图只画两锥与检测点；右上角单位圆给出全部内法向 $\mathbf{n}_i$ 与相邻夹角，红色弧为 $g_{\max}$ 。无界时浅紫填充为法向所在的闭半平面，红箭头为公共方向 $\mathbf{u} = -(\cos\alpha^*, \sin\alpha^*)$ 。(a) 对向交会： $g_{\max} = 164^\circ < 180^\circ$ ，有界；(b) 近乎同向： $g_{\max} = 188^\circ \geq 180^\circ$ ，沿 $\mathbf{u}$ 走向无穷，直径无定义。对本题误差锥，无界等价于各 2° 示向度弧存在公共方向。



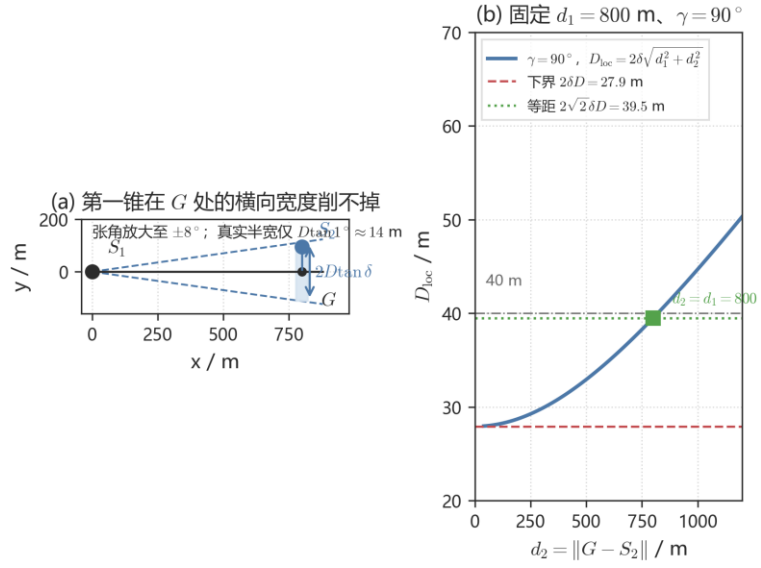
附图 2 直径与覆盖判定流程

附图 2 先求顶点，再取最长连线为直径，圆心取中点后逐点检验；有界时同时按算法 1 计算最小包围圆，供后续清除判据使用。



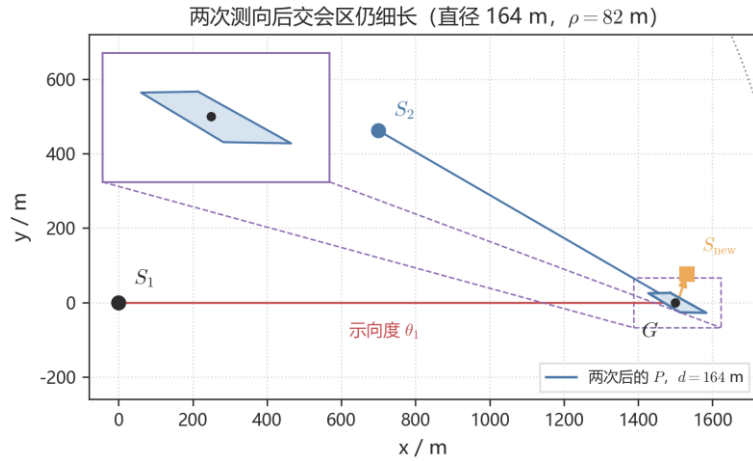
附图 3 误差盒与 J^{-1} 的像

附图 3 推导三件套：(a) 站址几何： S_1, S_2 与交会角 γ ；(b) 误差盒 $[-\delta, \delta]^2$ ，四角为 $(\pm\delta, \pm\delta)$ ；(c) J^{-1} 的像：四角 $G \pm \delta a \pm \delta b$ 、两支列向量 $\delta a, \delta b$ 与两条对角线。



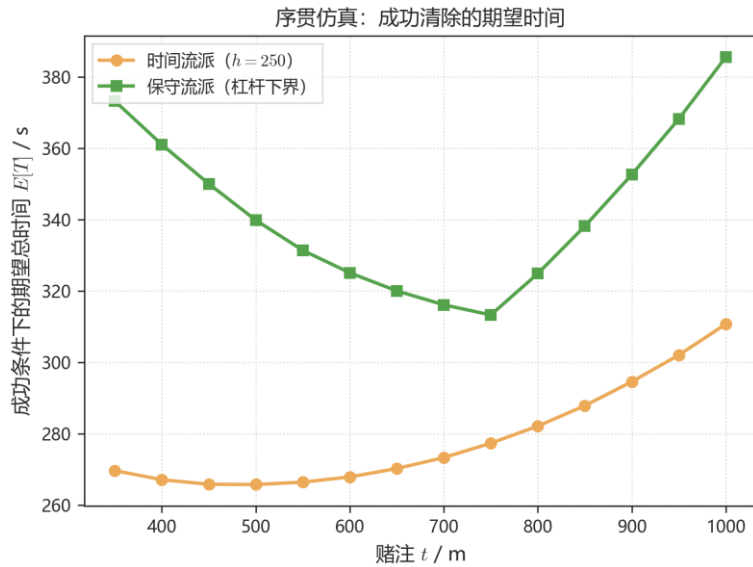
附图 4 绝对下界 $2\delta D$ 与等距点

附图 4 (a) 第一锥横向宽度（张角放大）；(b) D_{loc} 随 d_2 下降到 $2\delta D$ ，等距 800 m 对应 39.5 m。



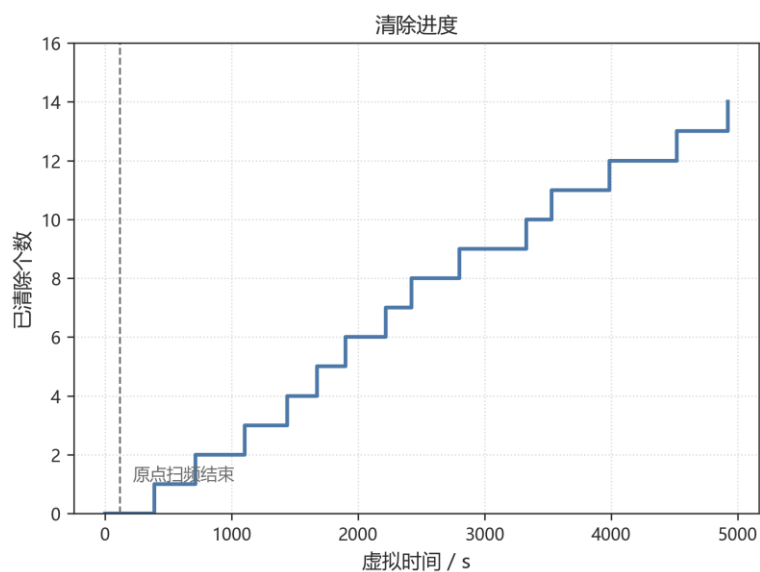
附图 5 两次后的细长交会区

附图 5 $D = 1500$ m。 S_{new} 在长轴法向， $\rho = 82$ m。



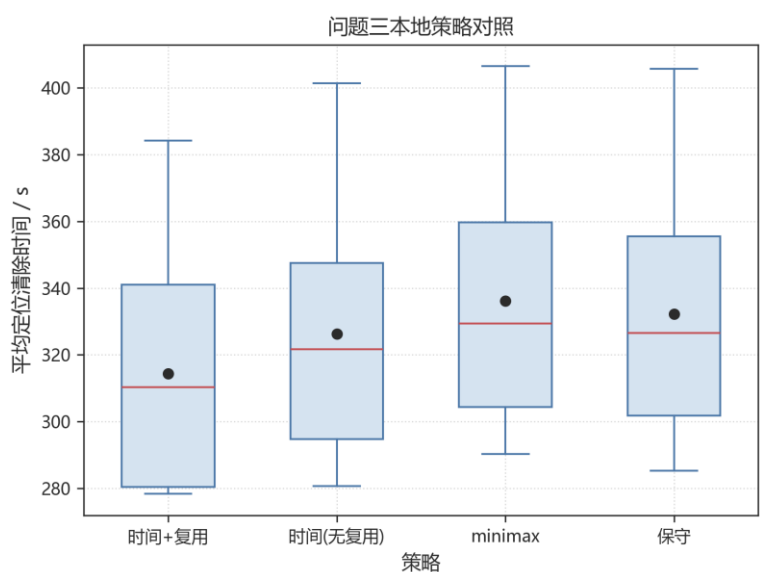
附图 6 成功清除的期望时间

附图 6 面积先验 $f_D \propto D$ 。时间流派谷底在 $t = 500$ m，用于问题三。



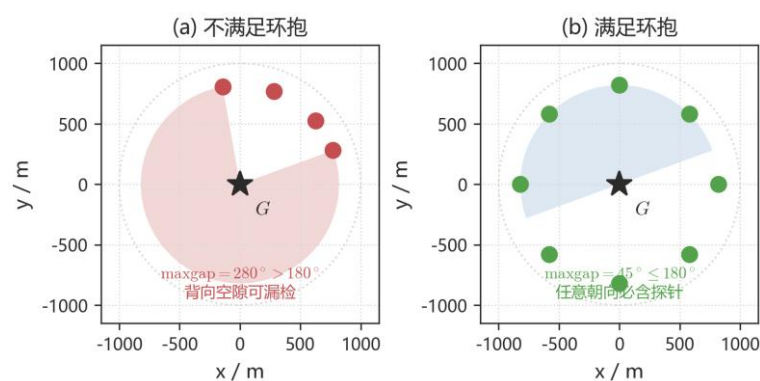
附图 7 问题三清除进度

附图 7 该局虚拟时间 5189 s。移动占 87.9%。竖线为原点扫频结束（119 s）。



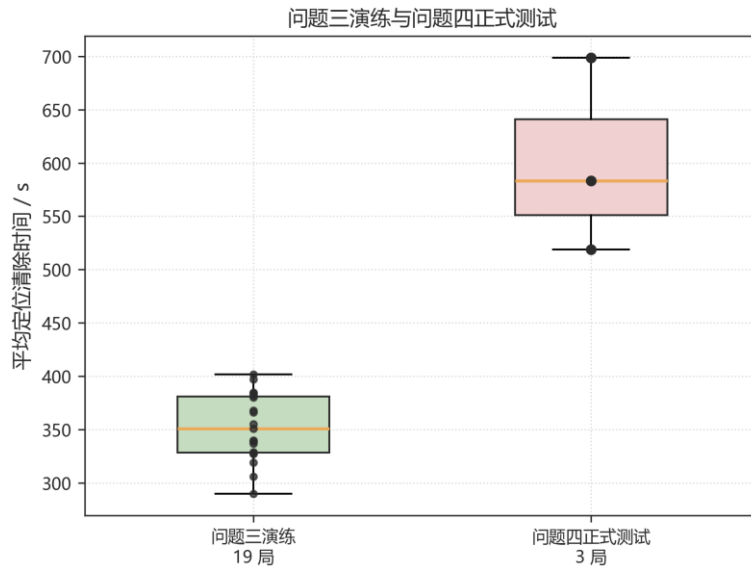
附图 8 问题三本地策略对照

附图 8 同一批 12 个世界上四种第二点策略的平均定位清除时间。



附图 9 环抱判据：最大方位间隙与 180°

附图 9 (a) $\text{maxgap} = 280^\circ$ ，存在朝向使全部探针落在盲侧；(b) $\text{maxgap} = 45^\circ$ ，任意朝向必被检出。



附图 10 问题三演练与问题四正式测试

附图 10 左：问题三官方 19 局演练（均值 352 s/个）；右：问题四正式测试三局（均值 600.26 s/个）。

附录 G 支撑材料文件列表

电子版支撑材料为 `支撑材料.zip`（与论文一并提交，大小不超过 20 MB）。论文附录给出重点源程序；完整可运行代码以压缩包为准。压缩包内不得含承诺书、编号页及队名、校名。

路径	说明
`q1/solver.py`	问题一半平面交会、直径、覆盖与最小包围圆
`q2/geometry.py`	问题二几何与 D_{loc} 闭式
`q3/run_q3.py`	问题三主程序：扫频、追击、环网补漏
`q4/ledger.py`	问题四探针账本与环抱判空
`q4/scheduler.py`	问题四证书巡回
`q4/policy.py`	问题四合并巡回与定向追击
`q4/run_q4.py`	问题四入口
`robot.py`	模拟器通信接口
`q2/derive_check.py`	闭式误差、四准则网格与 minimax 核验
`q4/skeleton_final.json`	26 点证书坐标
`q4/score.py`	问题四 mock 基准
`q1/figs/`、`q2/figs/`、`q3/figs/`、`q4/figs/`	论文插图源文件

附录 H 重点源程序

下列程序与提交的 `支撑材料.zip` 一致。问题三入口 `python q3/run\_q3.py`（默认关闭 `--adaptive-step`）；问题四入口 `python q4/run\_q4.py`。

程序 q1/solver.py

"""问题 1 求解：交会定位区域的顶点、直径、直径圆覆盖、最小包围圆。

算法与《第一问_论文参考稿.md》伪代码一致：

构造半平面 → 法向角间隙判定有界性 → 边界直线两两求交筛顶点
→ 顶点对最大距离即直径 → 检查直径圆是否覆盖 → 最小包围圆。

N 为个数。直径用顶点对枚举；最小包围圆用两点圆与三点外接圆枚举。

检测点很少时该方法足够。定位区域只取示向度误差锥交会，
不叠加目标圆域或有效接收半径圆盘。

"""

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from typing import List, Optional, Sequence, Tuple
```

```
import numpy as np
```

```
DELTA_DEG = 1.0 # 题面测向误差限
```

```
EPS = 1e-9 # 半平面归属
```

```
PARALLEL_EPS = 1e-12 # 直线求交: |det| 过小视为平行
```

```
DEDUP_TOL = 1e-7
```

```
COVER_TOL = 1e-8
```

```
def _unit(ang_deg: float) -> np.ndarray:
```

```
    t = np.radians(ang_deg)
```

```
    return np.array([np.cos(t), np.sin(t)], dtype=float)
```

```
def _wrap_deg(a: float) -> float:
```

```
    return float(a % 360.0)
```

```
@dataclass
```

```
class Halfplane:
```

```
    """(X - origin) · normal >= 0。normal 指向可行域内部。"""
```

```
    origin: np.ndarray
```

```
    normal: np.ndarray
```

```
    dir_angle: float # 边界直线的方向角(度)，仅用于求交
```

```
def cone_halfplanes(S: Sequence[float], theta_deg: float, delta_deg: float = DELTA_DEG) ->
```

```
List[Halfplane]:
```

```
    """检测点 S、示向度 theta 的误差锥：两条边界直线对应的半平面。"""
```

```
    S = np.asarray(S, dtype=float)
```

```
    lo = _wrap_deg(theta_deg - delta_deg)
```

```
    hi = _wrap_deg(theta_deg + delta_deg)
```

```
    u_lo, u_hi = _unit(lo), _unit(hi)
```

```
    # 可行域在 lo 的逆时针侧、hi 的顺时针侧
```

```
    n_lo = np.array([-u_lo[1], u_lo[0]])
```

```
    n_hi = np.array([u_hi[1], -u_hi[0]])
```

```
    return [
```

```
        Halfplane(S.copy(), n_lo, lo),
```

```
        Halfplane(S.copy(), n_hi, hi),
```

```
    ]
```

```
def line_intersection(h1: Halfplane, h2: Halfplane) -> Optional[np.ndarray]:
```

```
    """两条边界直线求交。直线 i: (X - o_i) · n_i = 0。"""
```

```
    A = np.vstack([h1.normal, h2.normal])
```

```
    det = float(np.linalg.det(A))
```

```
    if abs(det) < PARALLEL_EPS:
```

```
        return None
```

```
    b = np.array([np.dot(h1.normal, h1.origin), np.dot(h2.normal, h2.origin)])
```

```
    return np.linalg.solve(A, b)
```

```
def satisfies_all(p: np.ndarray, planes: Sequence[Halfplane], eps: float = EPS) -> bool:
```

```
    return all(float(np.dot(p - h.origin, h.normal)) >= -eps for h in planes)
```

```
def dedup_points(pts: Sequence[np.ndarray], tol: float = DEDUP_TOL) -> List[np.ndarray]:
```

```

    uniq: List[np.ndarray] = []
    for p in pts:
        if not any(np.linalg.norm(p - q) < tol for q in uniq):
            uniq.append(p)
    return uniq

def angular_sort(pts: Sequence[np.ndarray]) -> np.ndarray:
    arr = np.asarray(pts, dtype=float)
    c = arr.mean(axis=0)
    ang = np.arctan2(arr[:, 1] - c[1], arr[:, 0] - c[0])
    return arr[np.argsort(ang)]

def recession_unbounded(planes: Sequence[Halfplane]) -> bool:
    """若所有内法向落在某个闭半平面内（最大角间隙  $\geq 180^\circ$ ），则公共区域无界。

    对本题误差锥：无界等价于各  $2^\circ$  示向度弧有公共方向：沿该方向走到无穷
    必进入所有锥，故无界蕴含非空，不必再用探针判断可行性。
    """
    ang = np.sort([float(np.arctan2(h.normal[1], h.normal[0])) for h in planes])
    if len(ang) < 2:
        return True
    gaps = np.diff(ang)
    wrap = float(ang[0] + 2 * np.pi - ang[-1])
    max_gap = max(float(gaps.max()) if len(gaps) else 0.0, wrap)
    return max_gap >= np.pi - 1e-10

def halfplane_vertices(planes: Sequence[Halfplane]) -> List[np.ndarray]:
    cand: List[np.ndarray] = []
    m = len(planes)
    for i in range(m):
        for j in range(i + 1, m):
            p = line_intersection(planes[i], planes[j])
            if p is not None and satisfies_all(p, planes):
                cand.append(p)
    return dedup_points(cand)

def polygon_diameter(vertices: np.ndarray) -> Tuple[float, np.ndarray, np.ndarray]:
    """有界凸集直径：单点为 0，线段/共线顶点取两端，一般情形枚举顶点对。"""
    arr = np.asarray(vertices, dtype=float)
    if arr.ndim == 1:
        arr = arr.reshape(1, -1)
    m = len(arr)
    if m == 0:
        raise ValueError("空顶点集没有直径")
    if m == 1:
        p = arr[0].copy()
        return 0.0, p, p.copy()
    dmax, a, b = -1.0, arr[0], arr[0]
    for i in range(m):
        for j in range(i + 1, m):
            d = float(np.linalg.norm(arr[i] - arr[j]))
            if d > dmax:
                dmax, a, b = d, arr[i], arr[j]
    if dmax < 0.0:
        p = arr[0].copy()
        return 0.0, p, p.copy()
    return dmax, a, b

def circle_covers(
    vertices: np.ndarray,
    A: np.ndarray,
    B: np.ndarray,
    tol: float = COVER_TOL,
):
    """以 AB 为直径的圆是否覆盖全部顶点。含单点（R=0）与共线退化。"""
    verts = np.asarray(vertices, dtype=float)
    if verts.ndim == 1:
        verts = verts.reshape(1, -1)

```

```

O = (np.asarray(A, dtype=float) + np.asarray(B, dtype=float)) / 2.0
R = float(np.linalg.norm(np.asarray(A, dtype=float) - np.asarray(B, dtype=float)) / 2.0)
if len(verts) == 0:
    return True, O, R, np.zeros(0)
dist = np.linalg.norm(verts - O, axis=1)
return bool(np.all(dist <= R + tol)), O, R, dist

def _circumcircle(a: np.ndarray, b: np.ndarray, c: np.ndarray):
    """三点外接圆。共线则返回 None。"""
    d = 2.0 * (a[0] * (b[1] - c[1]) + b[0] * (c[1] - a[1]) + c[0] * (a[1] - b[1]))
    if abs(d) < 1e-12:
        return None
    a2, b2, c2 = float(np.dot(a, a)), float(np.dot(b, b)), float(np.dot(c, c))
    ux = (a2 * (b[1] - c[1]) + b2 * (c[1] - a[1]) + c2 * (a[1] - b[1])) / d
    uy = (a2 * (c[0] - b[0]) + b2 * (a[0] - c[0]) + c2 * (b[0] - a[0])) / d
    O = np.array([ux, uy], dtype=float)
    return O, float(np.linalg.norm(O - a))

def minimum_enclosing_circle(
    vertices: np.ndarray,
    tol: float = COVER_TOL,
) -> Tuple[np.ndarray, float]:
    """凸多边形最小包围圆：在覆盖全部顶点的两点圆、三点外接圆中取半径最小者。"""
    verts = np.asarray(vertices, dtype=float)
    if verts.ndim == 1:
        verts = verts.reshape(1, -1)
    m = len(verts)
    if m == 0:
        raise ValueError("空顶点集没有最小包围圆")
    if m == 1:
        return verts[0].copy(), 0.0

    best_O, best_R = None, np.inf
    for i in range(m):
        for j in range(i + 1, m):
            O = (verts[i] + verts[j]) / 2.0
            R = float(np.linalg.norm(verts[i] - O))
            if np.all(np.linalg.norm(verts - O, axis=1) <= R + tol) and R < best_R:
                best_O, best_R = O, R
    for i in range(m):
        for j in range(i + 1, m):
            for k in range(j + 1, m):
                circ = _circumcircle(verts[i], verts[j], verts[k])
                if circ is None:
                    continue
                O, R = circ
                if np.all(np.linalg.norm(verts - O, axis=1) <= R + tol) and R < best_R:
                    best_O, best_R = O, R
    if best_O is None:
        raise ValueError("未能求出最小包围圆")
    return best_O, float(best_R)

@dataclass
class LocateResult:
    status: str # OK / UNBOUNDED / EMPTY
    message: str
    vertices: Optional[np.ndarray]
    diameter: Optional[float]
    A: Optional[np.ndarray]
    B: Optional[np.ndarray]
    center: Optional[np.ndarray]
    radius: Optional[float]
    covers: Optional[bool]
    n_planes: int
    mec_center: Optional[np.ndarray] = None
    mec_radius: Optional[float] = None

    def pretty(self) -> str:
        lines = [f"status = {self.status}", self.message]
        if self.vertices is not None:

```

```

        lines.append(f"顶点数 m = {len(self.vertices)}")
        for i, v in enumerate(self.vertices, 1):
            lines.append(f" V{i} = ({v[0]:.6f}, {v[1]:.6f})")
        if self.diameter is not None:
            lines.append(f"直径 D = {self.diameter:.6f} m")
            lines.append(f"端点 A = ({self.A[0]:.6f}, {self.A[1]:.6f})")
            lines.append(f"端点 B = ({self.B[0]:.6f}, {self.B[1]:.6f})")
            lines.append(f"圆心 O = ({self.center[0]:.6f}, {self.center[1]:.6f}), R = {self.radius:.6f}")
        lines.append(f"直径圆覆盖 P? {'是' if self.covers else '否'}")
        if self.mec_radius is not None:
            c = self.mec_center
            lines.append(f"最小包围圆 MEC: 圆心 = ({c[0]:.6f}, {c[1]:.6f}), R = {self.mec_radius:.6f}")
        return "\n".join(lines)

def locate(
    sensors: Sequence[Sequence[float]],
    thetas_deg: Sequence[float],
    delta_deg: float = DELTA_DEG,
) -> LocateResult:
    """交会定位主程序。sensors[i]、thetas_deg[i] 一一对应。

    定位区域 P 只取示向度误差锥的公共交集，不叠加目标圆域或接收半径圆盘。
    判定顺序：先看法向角间隙判定无界，有界后再枚举交点；交点空则为空集。
    """
    N = len(sensors)
    if N != len(thetas_deg):
        raise ValueError("检测点与示向度数量不一致")

    if N < 2:
        return LocateResult(
            "UNBOUNDED",
            "N<2: 单个误差锥无界，直径无定义，需要增加检测点。",
            None, None, None, None, None, None, None, 2 * max(N, 0),
        )

    planes: List[Halfplane] = []
    for S, th in zip(sensors, thetas_deg):
        planes.extend(cone_halfplanes(S, th, delta_deg))
    m = len(planes)

    if recession_unbounded(planes):
        verts = halfplane_vertices(planes)
        return LocateResult(
            "UNBOUNDED",
            "半平面公共区域无界（法向未包围原点），直径无定义，需要增加或调整检测点。",
            np.array(verts) if verts else None,
            None, None, None, None, None, None, m,
        )

    verts = halfplane_vertices(planes)
    if len(verts) == 0:
        return LocateResult(
            "EMPTY",
            "交会区域为空：半平面无公共点，定位失败，应更换或增加检测点。",
            None, None, None, None, None, None, None, m,
        )

    P = angular_sort(verts)
    D, A, B = polygon_diameter(P)
    covers, O, R, _ = circle_covers(P, A, B)
    mec_c, mec_r = minimum_enclosing_circle(P)
    n = len(P)
    if n == 1:
        msg = "交会区域退化为单点。半平面数 2N = {}".format(m)
    elif n == 2:
        msg = "交会区域退化为线段。半平面数 2N = {}".format(m)
    else:
        msg = "有界凸 {} 边形。半平面数 2N = {}".format(n, m)
    return LocateResult("OK", msg, P, D, A, B, O, R, covers, m, mec_c, mec_r)

```

```

def bearing_deg(src, dst) -> float:
    v = np.asarray(dst, float) - np.asarray(src, float)
    return _wrap_deg(np.degrees(np.arctan2(v[1], v[0])))

def _demo():
    # 与示意图同一组站址: 示向度指向示意图源 G
    G = np.array([80.0, 220.0])
    S1 = np.array([-920.0, -480.0])
    S2 = np.array([980.0, -420.0])
    S3 = np.array([-40.0, 1180.0])
    th1, th2, th3 = bearing_deg(S1, G), bearing_deg(S2, G), bearing_deg(S3, G)

    print("=" * 60)
    print("算例 A N=1 (应 UNBOUNDED)")
    print(locate([S1], [th1]).pretty())

    print("\n" + "=" * 60)
    print("算例 B N=2,  $\delta=1^\circ$  (题面真实误差)")
    r2 = locate([S1, S2], [th1, th2], 1.0)
    print(r2.pretty())

    print("\n" + "=" * 60)
    print("算例 C N=3,  $\delta=1^\circ$ ")
    print(locate([S1, S2, S3], [th1, th2, th3], 1.0).pretty())

    print("\n" + "=" * 60)
    print("算例 D 两站相背, 交会为空 (应 EMPTY)")
    print(locate([[0.0, 0.0], [200.0, 0.0]], [90.0, 270.0], 1.0).pretty())

    print("\n" + "=" * 60)
    print("算例 E 等边三角形: 直径圆不能覆盖 (圆覆盖反例)")
    eq = np.array([[0.0, 0.0], [2.0, 0.0], [1.0, np.sqrt(3.0)]])
    D, A, B = polygon_diameter(eq)
    covers, O, R, dist = circle_covers(eq, A, B)
    mec_c, mec_r = minimum_enclosing_circle(eq)
    print(f"边长=2, D={D:.6f}, R={R:.6f}, 顶点到圆心 = {dist}, 覆盖? {covers}")
    print(f"MEC R={mec_r:.6f} (等边三角形外接圆)")

    print("\n" + "=" * 60)
    print("算例 F 两站同向 (无有限直径, 应 UNBOUNDED 而非 EMPTY)")
    print(locate([[0.0, 0.0], [200.0, 0.0]], [90.0, 90.0], 1.0).pretty())

    print("\n" + "=" * 60)
    print("算例 G 三站环绕: 交会六边形, 直径圆不能覆盖")
    G0 = np.array([0.0, 0.0])
    Ss = [
        np.array([800.0, 0.0]),
        np.array([-400.0, 400.0 * np.sqrt(3.0)]),
        np.array([-400.0, -400.0 * np.sqrt(3.0)]),
    ]
    ths = [bearing_deg(S, G0) for S in Ss]
    print(locate(Ss, ths, 1.0).pretty())

    print("\n" + "=" * 60)
    print("退化: 单点 / 共线三点")
    p1 = np.array([[1.0, 2.0]])
    d1, a1, b1 = polygon_diameter(p1)
    c1, _, r1, _ = circle_covers(p1, a1, b1)
    print(f"单点 D={d1}, 覆盖? {c1}, R={r1}")
    col = np.array([[0.0, 0.0], [1.0, 0.0], [2.0, 0.0]])
    dc, ac, bc = polygon_diameter(col)
    cc, _, rc, distc = circle_covers(col, ac, bc)
    print(f"共线 D={dc:.6f}, 覆盖? {cc}, 顶点到圆心 = {distc}")

if __name__ == "__main__":
    _demo()

```

程序 q2/geometry.py

```

# -*- coding: utf-8 -*-
"""问题 2 共用几何工具：角度带楔形 / 半平面裁剪 / 凸多边形直径与长轴 / 最小覆盖圆(MEC)"""
import numpy as np
from itertools import combinations

DELTA = np.deg2rad(1.0)      # 测向误差半宽
V = 5.0                      # 机器狗速度 m/s
MEAS_T = 5.0                 # 单次测量耗时 s
CLEAR_T = 5.0                # 光学定位+激光清除耗时 s

def cross2(a, b):
    return a[0] * b[1] - a[1] * b[0]

def wedge_hp(S, th):
    """以 S 为顶点、示向度 th 为中心、±δ 张角的楔形 -> 两个半平面 (S, 法向 e, 符号)"""
    return [(S, np.array([np.cos(th - DELTA), np.sin(th - DELTA)]), +1),
            (S, np.array([np.cos(th + DELTA), np.sin(th + DELTA)]), -1)]

def clip(poly, S, e, sign):
    """Sutherland-Hodgman: 保留 sign*cross(e, P-S) >= 0"""
    side = lambda p: sign * cross2(e, p - S)
    res = []
    n = len(poly)
    for i in range(n):
        c, nx = poly[i], poly[(i + 1) % n]
        dc, dn = side(c), side(nx)
        if dc >= 0:
            res.append(c)
        if (dc > 0 and dn < 0) or (dc < 0 and dn > 0):
            res.append(c + dc / (dc - dn) * (nx - c))
    return np.array(res) if res else np.zeros((0, 2))

def region(HPs, box=6000.0):
    """全部半平面与大框的交集 -> 凸多边形顶点"""
    poly = np.array([[-box, -box], [box, -box], [box, box], [-box, box]], float)
    for S, e, s in HPs:
        if len(poly) < 3:
            return np.zeros((0, 2))
        poly = clip(poly, S, e, s)
    return poly

def diam_and_axis(v):
    """凸多边形直径及长轴方向单位向量"""
    best = (0.0, None, None)
    for a, b in combinations(range(len(v)), 2):
        d = np.linalg.norm(v[a] - v[b])
        if d > best[0]:
            best = (d, v[a], v[b])
    ax = best[2] - best[1]
    return best[0], ax / np.linalg.norm(ax)

def mec(v):
    """最小覆盖圆 (半径, 圆心), 暴力: 直径对 或 三点外接圆"""
    n = len(v)
    cand = []
    for a, b in combinations(range(n), 2):
        c = (v[a] + v[b]) / 2
        r = np.linalg.norm(v[a] - c)
        if all(np.linalg.norm(x - c) <= r + 1e-9 for x in v):
            cand.append((r, c))
    for tri in combinations(range(n), 3):
        p, q, r_ = v[tri[0]], v[tri[1]], v[tri[2]]
        d = 2 * (p[0] * (q[1] - r_[1]) + q[0] * (r_[1] - p[1]) + r_[0] * (p[1] - q[1]))
        if abs(d) < 1e-12:
            continue
        ux = ((p @ p) * (q[1] - r_[1]) + (q @ q) * (r_[1] - p[1]) + (r_ @ r_) * (p[1] - q[1])) / d
        uy = ((p @ p) * (r_[0] - q[0]) + (q @ q) * (p[0] - r_[0]) + (r_ @ r_) * (q[0] - p[0])) / d

```

```

        c = np.array([ux, uy])
        r = np.linalg.norm(p - c)
        if all(np.linalg.norm(x - c) <= r + 1e-9 for x in v):
            cand.append((r, c))
        return min(cand, key=lambda t: t[0]) if cand else (np.inf, None)

def k_expr(t, h, D, eps=0.0):
    """K-闭式:  $(\max(|a \pm b|))^2 = (d_1^2 + d_2^2 + 2d_1d_2|\cos y|)/\sin^2 y$ , 含数值保护"""
    G = np.array([D * np.cos(eps), D * np.sin(eps)])
    S1 = np.array([0., 0.])
    S2 = np.array([t, h])
    d1 = np.linalg.norm(G - S1)
    d2 = np.linalg.norm(G - S2)
    v1, v2 = S1 - G, S2 - G
    sg = abs(cross2(v1, v2)) / (d1 * d2)
    if sg < 1e-4:
        return np.inf
    cg = (v1 @ v2) / (d1 * d2)
    return (d1 * d1 + d2 * d2 + 2 * d1 * d2 * abs(cg)) / (sg * sg)

def _try_clear(v):
    """仅当 MEC 半径  $\leq 20$  m 才允许进圈清除。"""
    if v is None or len(v) < 3:
        return False, np.inf, None
    r, C = mec(v)
    if C is None or not np.isfinite(r) or r > 20.0:
        return False, r, C
    return True, r, C

def episode(t, h, D, rng=None, rho_rule=lambda L: 300.0, max_meas=7):
    """单目标一次完整定位清除流程（角度交会模型，不裁接收圆盘与目标圆域）。

    测量坐标系: S1 处测得示向度=0; 真实  $G=D*(\cos e_1, \sin e_1)$ ,  $e_i \sim U[-\delta, \delta]$  (rng=None 时  $e=0$ )。
    仅当定位区域 MEC 半径  $\leq 20$  m 时计入清除时间。
    返回 (总时间 s 或 inf, 测量次数, 是否成功清除)。
    """
    e1 = rng.uniform(-DELTA, DELTA) if rng is not None else 0.0
    G = np.array([D * np.cos(e1), D * np.sin(e1)])
    S1 = np.array([0., 0.])
    HPs = wedge_hp(S1, 0.0) # 第一次测量(给定), 楔形以测得示向度 0 为中心
    cur = S1
    T = 0.0
    n = 1

    def meas_at(S):
        """在 S 测量: 返回楔形(中心=真实方位+误差)"""
        th = np.arctan2(G[1] - S[1], G[0] - S[0])
        ei = rng.uniform(-DELTA, DELTA) if rng is not None else 0.0
        return wedge_hp(S, th + ei)

    S2 = np.array([t, h])
    T += np.linalg.norm(S2 - cur) / V + MEAS_T
    cur = S2
    n += 1
    HPs += meas_at(S2)

    while n < max_meas:
        v = region(HPs)
        ok, _, C = _try_clear(v)
        if ok:
            T += np.linalg.norm(C - cur) / V + CLEAR_T
            return T, n, True
        if v is None or len(v) < 3:
            return np.inf, n, False
        d2, ax = diam_and_axis(v)
        r, C = mec(v)
        if C is None or not np.isfinite(r):
            return np.inf, n, False
        rho = rho_rule(d2)
        nhat = np.array([-ax[1], ax[0]])

```

```

        if nhath @ (cur - C) < 0:
            nhath = -nhath
        Sn = C + rho * nhath
        T += np.linalg.norm(Sn - cur) / V + MEAS_T
        cur = Sn
        n += 1
        HPS += meas_at(Sn)
    v = region(HPS)
    ok, _, C = _try_clear(v)
    if not ok:
        return np.inf, n, False
    T += np.linalg.norm(C - cur) / V + CLEAR_T
    return T, n, True

def expected_time(t, h, Ds, w=None, rng=None, rho_rule=lambda L: 300.0, max_meas=7):
    """面积先验下 E[T | 成功] 与成功概率。失败样本不进入平均时间。"""
    Ds = np.asarray(Ds, float)
    if w is None:
        w = Ds.copy()
    w = np.asarray(w, float)
    w = w / w.sum()
    Ts = np.empty(len(Ds), dtype=float)
    ok = np.zeros(len(Ds), dtype=bool)
    ns = np.zeros(len(Ds), dtype=float)
    for i, D in enumerate(Ds):
        T, n, suc = episode(t, h, float(D), rng=rng, rho_rule=rho_rule, max_meas=max_meas)
        Ts[i], ns[i], ok[i] = T, n, suc
    p = float(w[ok].sum()) if np.any(ok) else 0.0
    if p <= 0.0:
        return np.inf, 0.0, np.nan
    ww = w[ok]
    ww = ww / ww.sum()
    et = float(Ts[ok] @ ww)
    en = float(ns[ok] @ ww)
    return et, p, en

```

程序 q3/run_q3.py

```

# -*- coding: utf-8 -*-
"""问题 3: 全向干扰源搜索、交会定位与清除。

策略:
1. 原点扫 1-20 频道 (不移动, 约 119 s 虚拟时间);
2. 已听频道: 能交会且 MEC≤20 m 的先就近清除; 其余按估计位置追击。
   第二点取 (t,h)=(500,±250) 中离狗更近的一侧; 若当前位置已有足够侧向间隔则就地再测;
   MEC>20 m 再按长轴切。每个停点顺便测仍只有一站、且相对第一站接近 (500,250) 的已听频道;
   交会已进 20 m 且顺路的源就地收割, 避免专程再跑;
3. 半径 1150 m 上 6 个监听点按最短巡线补网 (6! 枚举, 含从当前位置到第一环点);
   环上新听到的源若去第二点 (500,±250) 再进圈的额外路程 > 300 m 则挂起, 整圈走完再就近追。
   原点扫完后同样按该额外路程判挂起 (对照预规划环的第一点); 墙钟不够则跳出剩余环点并立即 drain。
   `--no-hang` 关闭环上挂起; `--no-origin-hang` 关闭原点挂起; 二者独立。
   追击第二点纵向距离默认 500 m; 若该频道在别处无信号, 则沿射线剔除 |G-M|≤1005 m 的 p,
   把赌注夹到允许区间 (`--no-miss-constraint` 关闭)。
   `--replan` 打开滚动重规划: 任务池 = 未访环点 + 非挂起已听 (入口 closer_offset) + 可清除 (顺路 q);
   每次只执行开路巡线的第一项再重解。默认仍是原点追完再走环 (`--no-replan`)。
   `--no-onroute-clear` 关闭顺路清点 (默认在 C_MEC 朝下一节点平移)。
   `--adaptive-step` 在已有 MEC 中心时把第二点纵向赌注改成 0.85|C-S0| (默认关)。

```

用法:

```

python q3/run_q3.py
python q3/run_q3.py --watch
python q3/run_q3.py --local [--episodes N]
python q3/run_q3.py --local --replan
python q3/run_q3.py --local --no-origin-hang --no-miss-constraint
python q3/run_q3.py --no-hang --no-origin-hang
python tools/practice_loop.py --problem 3 --n 10

```

环境变量 ROBOT_ID 可覆盖默认参赛队号。

"""

from __future__ import annotations

```

import argparse
import json
import math
import os
import sys
import time
from itertools import permutations
from pathlib import Path

import numpy as np

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "q1"))
sys.path.insert(0, str(ROOT / "q2"))
sys.path.insert(0, str(Path(__file__).resolve().parent))

from api import Simulator, ApiError # noqa: E402
from geometry import diam_and_axis, mec # noqa: E402
from solver import locate # noqa: E402
from tactics import closer_offset, estimate_pos, lateral_offset, ray_coords # noqa: E402

ROBOT_ID = os.environ.get("ROBOT_ID", "YOUR_TEAM_ID")
ARENA_R = 1800.0
RING_R = 1150.0
N_RING = 6
BET_T = 500.0
BET_H = 250.0
CLEAR_R = 20.0
MAX_HUNT_MEAS = 7
HANG_M = 300.0
CLEAR_EST = 480.0
REPLAN_DEFAULT = False
ONROUTE_CLEAR_DEFAULT = True

LOG_DIR = Path(__file__).resolve().parent / "logs"

def unit(deg: float) -> np.ndarray:
    t = math.radians(deg)
    return np.array([math.cos(t), math.sin(t)], dtype=float)

def clamp_xy(p: np.ndarray) -> np.ndarray:
    p = np.asarray(p, dtype=float)
    n = float(np.linalg.norm(p))
    if n > ARENA_R - 10.0 and n > 1e-9:
        p = p * ((ARENA_R - 10.0) / n)
    return p

def ring_points(radius: float = RING_R, n: int = N_RING) -> list:
    return [
        radius * np.array([math.cos(2 * math.pi * k / n), math.sin(2 * math.pi * k / n)])
        for k in range(n)
    ]

def covering_worst_gap(radius: float = RING_R, n: int = N_RING, arena: float = ARENA_R) -> float:
    """圆盘内到最近监听点（原点+环）的最大距离，应 < 1000。"""
    pts = [np.zeros(2)] + ring_points(radius, n)
    worst = 0.0
    for r in np.linspace(0.0, arena, 73):
        n_ang = 240 if r > 1 else 1
        for k in range(n_ang):
            a = 2 * math.pi * k / n_ang
            x = r * np.array([math.cos(a), math.sin(a)])
            d = min(float(np.linalg.norm(x - p)) for p in pts)
            if d > worst:
                worst = d
    return worst

def ring_tour_bruteforce(start, pts):

```

```

"""6! 枚举环点顺序, 含从当前位置到第一环点的路程, 取最短。"""
start = np.asarray(start, float)
pts = [np.asarray(p, float) for p in pts]
n = len(pts)
best_perm = None
best_L = float("inf")
for perm in permutations(range(n)):
    L = 0.0
    cur = start
    for i in perm:
        L += float(np.linalg.norm(pts[i] - cur))
        cur = pts[i]
    if L < best_L:
        best_L = L
        best_perm = perm
return [(int(i), pts[i]) for i in best_perm], float(best_L)

def two_opt_tour(pts, start):
    """开路 2-opt: 从 start 走完 pts (不回到起点)。"""
    left = [np.asarray(p, float) for p in pts]
    cur = np.asarray(start, float)
    order = []
    while left:
        k = min(range(len(left)), key=lambda i: float(np.linalg.norm(left[i] - cur)))
        order.append(left.pop(k))
        cur = order[-1]

    def length(seq):
        s, c = 0.0, np.asarray(start, float)
        for p in seq:
            s += float(np.linalg.norm(p - c))
            c = p
        return s

    improved = True
    while improved:
        improved = False
        for i in range(len(order) - 1):
            for j in range(i + 1, len(order)):
                new = order[:i] + order[i : j + 1][::-1] + order[j + 1 :]
                if length(new) < length(order) - 1e-6:
                    order, improved = new, True
                    break
            if improved:
                break
    return order, length(order)

class TaskNode:
    __slots__ = ("kind", "key", "xy", "C", "r")

    def __init__(self, kind, key, xy, C=None, r=None):
        self.kind = kind # "ring" | "chase" | "clear"
        self.key = key
        self.xy = np.asarray(xy, float)
        self.C = None if C is None else np.asarray(C, float)
        self.r = r

    def label(self) -> str:
        if self.kind == "ring":
            return f"ring:{self.key}"
        return f"{self.kind}:ch{int(self.key):02d}"

def _open_tour_length(start, seq) -> float:
    s, c = 0.0, np.asarray(start, float)
    for n in seq:
        s += float(np.linalg.norm(n.xy - c))
        c = n.xy
    return s

def solve_open_tour(start, nodes, max_rounds: int = 5):

```

```

"""开路巡线：最近邻起步，再 2-opt 与 relocate，起点固定、不回到起点。"""
nodes = list(nodes)
if not nodes:
    return []
start = np.asarray(start, float)
left = list(nodes)
order = []
cur = start
while left:
    k = min(range(len(left)), key=lambda i: float(np.linalg.norm(left[i].xy - cur)))
    order.append(left.pop(k))
    cur = order[-1].xy
if len(order) <= 2:
    return order

def L(seq):
    return _open_tour_length(start, seq)

for _ in range(max_rounds):
    improved = False
    while True:
        found = False
        for i in range(len(order) - 1):
            for j in range(i + 1, len(order)):
                new = order[:i] + order[i : j + 1][::-1] + order[j + 1 :]
                if L(new) < L(order) - 1e-6:
                    order = new
                    found = True
                    improved = True
                    break
            if found:
                break
        if not found:
            break
    while True:
        found = False
        n = len(order)
        for i in range(n):
            node = order[i]
            tmp = order[:i] + order[i + 1 :]
            for ins in range(len(tmp) + 1):
                if ins == i:
                    continue
                new = tmp[:ins] + [node] + tmp[ins:]
                if L(new) < L(order) - 1e-6:
                    order = new
                    found = True
                    improved = True
                    break
            if found:
                break
        if not found:
            break
    if not improved:
        break
    return order

def path_from_log(log_rows):
    pts = []
    for r in log_rows:
        pl = r.get("payload") or {}
        pos = pl.get("position")
        if not pos:
            continue
        p = np.array([float(pos["x"]), float(pos["y"])], dtype=float)
        if pts and float(np.linalg.norm(p - pts[-1])) < 1e-6:
            continue
        pts.append(p)
    return pts

def unique_stops(pts, tol: float = 1.0):
    uniq = []

```

```

for p in pts:
    p = np.asarray(p, float)
    if not uniq or min(float(np.linalg.norm(p - q)) for q in uniq) > tol:
        uniq.append(p)
return uniq

def actual_path_length(pts):
    if len(pts) < 2:
        return 0.0
    return float(sum(np.linalg.norm(pts[i] - pts[i - 1]) for i in range(1, len(pts))))

def second_point(S: np.ndarray, theta_deg: float, t: float = BET_T, h: float = BET_H) -> np.ndarray:
    e = unit(theta_deg)
    n = np.array([-e[1], e[0]])
    return clamp_xy(np.asarray(S, float) + t * e + h * n)

def region_center(vertices: np.ndarray):
    v = np.asarray(vertices, dtype=float)
    if v.ndim == 1:
        v = v.reshape(1, -1)
    if len(v) == 0:
        return float("inf"), np.zeros(2)
    if len(v) == 1:
        return 0.0, v[0].copy()
    if len(v) == 2:
        C = (v[0] + v[1]) / 2.0
        r = 0.5 * float(np.linalg.norm(v[0] - v[1]))
        return r, C
    r, C = mec(v)
    if C is None or not np.isfinite(r):
        C = v.mean(axis=0)
        r = float("inf")
    return float(r), np.asarray(C, float)

def long_axis_point(vertices: np.ndarray, robot: np.ndarray) -> np.ndarray:
    v = np.asarray(vertices, dtype=float)
    r, C = region_center(v)
    if len(v) < 2:
        return clamp_xy(C + np.array([80.0, 0.0]))
    L, ax = diam_and_axis(v)
    if ax is None or L <= 1e-9:
        ax = np.array([1.0, 0.0])
        L = 40.0
    rho = max(50.0, 0.5 * float(L))
    nhat = np.array([-ax[1], ax[0]], dtype=float)
    if float(nhat @ (np.asarray(robot, float) - C)) < 0:
        nhat = -nhat
    return clamp_xy(C + rho * nhat)

class ChannelBook:
    def __init__(self):
        self.cleared = set()
        self.empty = set()
        self.failed = set()
        self.hits = {ch: [] for ch in range(1, 21)} # (xy, svd_deg)
        self.miss_pts = {ch: [] for ch in range(1, 21)}
        self.miss_ids = {ch: set() for ch in range(1, 21)}
        self.clear_time = {}

    def remaining(self) -> list:
        done = self.cleared | self.empty | self.failed
        return [ch for ch in range(1, 21) if ch not in done]

    def heard(self) -> list:
        return [ch for ch in self.remaining() if self.hits[ch]]

class Dog:
    def __init__(

```

```

self,
sim: Simulator,
remain_s: float = 1200.0,
bet_t: float = BET_T,
bet_h: float = BET_H,
reuse_stops: bool = True,
hang: bool = True,
origin_hang: bool = True,
miss_constraint: bool = True,
replan: bool = REPLAN_DEFAULT,
onroute_clear: bool = True,
adaptive_step: bool = False,
):
    self.sim = sim
    self.xy = np.zeros(2)
    self.heading = np.array([1.0, 0.0], dtype=float)
    self.next_xy = None
    self.last_ch = 1
    self.book = ChannelBook()
    self.n_clear_ok = 0
    self.remain_s = remain_s
    self.wall0 = time.time()
    self.bet_t = float(bet_t)
    self.bet_h = float(bet_h)
    self.reuse_stops = bool(reuse_stops)
    self.hang = bool(hang)
    self.origin_hang = bool(origin_hang)
    self.miss_constraint = bool(miss_constraint)
    self.replan = bool(replan)
    self.onroute_clear = bool(onroute_clear)
    self.adaptive_step = bool(adaptive_step)
    self.hung = set()
    self.n_replan_decisions = 0
    self.n_replan_diverge = 0

def wall_low(self, need: float = 50.0) -> bool:
    return (time.time() - self.wall0) > (self.remain_s - need)

def _note_move(self, p) -> np.ndarray:
    p = clamp_xy(np.asarray(p, float))
    d = p - self.xy
    n = float(np.linalg.norm(d))
    if n > 1e-6:
        self.heading = d / n
    return p

def measure_at(self, p, ch: int, cover_id: str | None = None) -> dict:
    p = self._note_move(p)
    body = self.sim.measure(float(p[0]), float(p[1]), ch)
    self.xy = p
    self.last_ch = int(ch)
    kind = body.get("measure_result")
    print(
        f" meas ch{ch:02d} ({p[0]:.1f},{p[1]:.1f}) -> {kind}"
        + (f" svd={body.get('svd_deg')}" if kind == "direction" else "")
        + f" t={self.sim.virtual_time_s:.1f}s"
    )
    if kind == "direction":
        self.book.hits[ch].append((p.copy(), float(body["svd_deg"])))
    elif kind == "no_signal":
        self.book.miss_pts[ch].append(p.copy())
        if cover_id:
            self.book.miss_ids[ch].add(cover_id)
    return body

def clear_at(self, p, ch: int) -> bool:
    p = self._note_move(p)
    body = self.sim.clear(float(p[0]), float(p[1]), ch)
    self.xy = p
    self.last_ch = int(ch)
    ok = body.get("clear_result") == "success"
    print(
        f" CLEAR ch{ch:02d} ({p[0]:.1f},{p[1]:.1f}) -> {body.get('clear_result')}"
        + f" t={self.sim.virtual_time_s:.1f}s"
    )

```

```

    )
    if ok:
        self.book.cleared.add(ch)
        self.book.failed.discard(ch)
        self.book.clear_time[ch] = self.sim.virtual_time_s
        self.n_clear_ok += 1
    return ok

def try_localize(self, ch: int):
    hits = self.book.hits[ch]
    if len(hits) < 2:
        return None
    sensors = [h[0] for h in hits]
    thetas = [h[1] for h in hits]
    return locate(sensors, thetas, 1.0)

def ready_mec(self, ch: int):
    loc = self.try_localize(ch)
    if loc is None or loc.status != "OK" or loc.vertices is None:
        return None
    r, C = region_center(loc.vertices)
    if r <= CLEAR_R + 1e-6:
        return float(r), np.asarray(C, float)
    return None

def ready_center(self, ch: int):
    info = self.ready_mec(ch)
    return None if info is None else info[1]

def choose_clear_point(self, C, r, next_xy=None):
    """顺路清点: 在  $|q-C|+r \leq 19.5$  内朝下一节点平移, 近距就地清不走这里。"""
    C = np.asarray(C, float)
    r = float(r)
    if (not self.onroute_clear) or r > CLEAR_R + 1e-6:
        return C
    delta = max(0.0, 19.5 - r)
    cur = np.asarray(self.xy, float)
    nxt = None if next_xy is None else np.asarray(next_xy, float)
    u = None
    if nxt is not None:
        v = nxt - C
        n = float(np.linalg.norm(v))
        if n > 1e-9:
            u = v / n
    if u is None:
        h = np.asarray(self._heading, float)
        hn = float(np.linalg.norm(h))
        if hn > 1e-9:
            u = h / hn
        else:
            v = C - cur
            n = float(np.linalg.norm(v))
            u = v / n if n > 1e-9 else np.array([1.0, 0.0])
    cand = [C, C + 0.5 * delta * u, C + delta * u]

def feasible(q):
    return float(np.linalg.norm(np.asarray(q, float) - C)) + r <= 19.5 + 1e-9

def cost(q):
    q = np.asarray(q, float)
    if nxt is not None:
        return float(np.linalg.norm(q - cur)) + float(np.linalg.norm(q - nxt))
    return float(np.linalg.norm(q - cur))

feas = [q for q in cand if feasible(q)]
if not feas:
    return C
return min(feas, key=cost)

def piggyback(self, p, exclude_ch: int | None = None, max_n: int = 2) -> None:
    """停点顺便测: 只测仍为一站、且相对第一站接近时间赌注 (500,250) 的已听频道。"""
    if self.wall_low(55.0):
        return
    p = np.asarray(p, float)

```

```

scored = []
for ch in self.book.heard():
    if ch == exclude_ch:
        continue
    hits = self.book.hits[ch]
    if len(hits) >= 2:
        continue
    S, th = hits[-1]
    if float(np.linalg.norm(p - S)) < 60.0:
        continue
    t_along, hlat = ray_coords(S, th, p)
    if t_along < 200.0 or t_along > 1000.0:
        continue
    if hlat < 80.0 or hlat > 600.0:
        continue
    geom = abs(t_along - self.bet_t) + abs(hlat - self.bet_h)
    scored.append((geom, abs(ch - self.last_ch), ch))
scored.sort()
for _, _, ch in scored[:max_n]:
    if self.wall_low(45.0):
        break
    body = self.measure_at(p, ch)
    if body.get("measure_result") == "near":
        self.clear_at(p, ch)

def harvest_ready(self, max_dist: float = 80.0, exclude_ch: int | None = None) -> None:
    """已经交会进 20 m 的, 若离当前位置不远就顺路清 (不中断当前追击频道)。"""
    ready = []
    for ch in list(self.book.heard()):
        if ch == exclude_ch:
            continue
        info = self.ready_mec(ch)
        if info is None:
            continue
        r, C = info
        q = self.choose_clear_point(C, r, self._next_xy)
        ready.append((float(np.linalg.norm(q - self.xy)), ch, q))
    ready.sort()
    for d, ch, q in ready:
        if ch in self.book.cleared:
            continue
        if d > max_dist:
            continue
        self.clear_at(q, ch)

def after_stop(self, exclude_ch: int | None = None) -> None:
    if not self.reuse_stops:
        return
    self.piggyback(self.xy, exclude_ch=exclude_ch)
    self.harvest_ready(max_dist=80.0, exclude_ch=exclude_ch)

def miss_bet_window(self, ch: int, S0, th0):
    """无信号点给出 |G-M|>1000 m. 沿末听射线 1 m 扫描 p∈[5,1500],
    与 [200,1400] 相交得到 [p_lo, p_hi]; 空集则 None (调用方回退 BET_T)。"""
    if not self.miss_constraint:
        return None
    misses = self.book.miss_pts.get(ch) or []
    if not misses:
        return None
    S0 = np.asarray(S0, float)
    u = unit(th0)
    rho = np.arange(5.0, 1500.0 + 0.5, 1.0)
    pts = S0[None, :] + rho[:, None] * u[None, :]
    r2 = 1005.0 ** 2
    allowed = np.ones(rho.shape, dtype=bool)
    for M in misses:
        M = np.asarray(M, float)
        d2 = np.sum((pts - M) ** 2, axis=1)
        allowed &= d2 > r2
    mask = allowed & (rho >= 200.0) & (rho <= 1400.0)
    if not np.any(mask):
        return None
    surv = rho[mask]
    return float(surv.min()), float(surv.max())

```

```

def hunt(self, ch: int) -> bool:
    print(f"-- hunt channel {ch}")
    if ch in self.book.cleared:
        return True
    logged_miss = False
    for _step in range(MAX_HUNT_MEAS):
        if ch in self.book.cleared:
            return True
        loc = self.try_localize(ch)
        if loc is not None and loc.status == "OK" and loc.vertices is not None:
            r, C = region_center(loc.vertices)
            print(f"    localize OK m={len(loc.vertices)} MEC={r:.2f}m D={loc.diameter:.2f}m")
            if r <= CLEAR_R + 1e-6:
                q = self.choose_clear_point(C, r, self._next_xy)
                if self.clear_at(q, ch):
                    self.after_stop(exclude_ch=ch)
                    return True
                if ch in self.book.cleared:
                    return True
                self.book.failed.add(ch)
                return False
            nxt = long_axis_point(loc.vertices, self.xy)
            body = self.measure_at(nxt, ch)
            self.after_stop(exclude_ch=ch)
            if ch in self.book.cleared:
                return True
            if body.get("measure_result") == "near":
                if self.clear_at(self.xy, ch):
                    self.after_stop(exclude_ch=ch)
                    return True
            continue

        hits = self.book.hits[ch]
        if not hits:
            return False
        if loc is not None and loc.status == "EMPTY" and len(hits) >= 2:
            self.book.hits[ch] = hits[-1:]
            hits = self.book.hits[ch]

        S0, th0 = hits[-1]
        if _step == 0 and float(np.linalg.norm(self.xy - S0)) > 300.0:
            if lateral_offset(S0, th0, self.xy) > 70.0:
                body = self.measure_at(self.xy, ch)
                self.after_stop(exclude_ch=ch)
                if ch in self.book.cleared:
                    return True
                kind = body.get("measure_result")
                if kind == "near":
                    if self.clear_at(self.xy, ch):
                        self.after_stop(exclude_ch=ch)
                        return True
                if kind == "direction":
                    continue
            t_base = self.bet_t
            if self.adaptive_step:
                loc_ad = self.try_localize(ch)
                if loc_ad is not None and loc_ad.status == "OK" and loc_ad.vertices is not None:
                    _, C_ad = region_center(loc_ad.vertices)
                    t_base = float(
                        np.clip(0.85 * float(np.linalg.norm(np.asarray(C_ad) - S0)), 20.0, 1100.0)
                    )
            t_raw = t_base + 120.0 * (_step // 2)
            win = self.miss_bet_window(ch, S0, th0)
            t = t_raw
            if win is not None:
                rho_lo, rho_hi = win
                t = float(np.clip(t_raw, rho_lo, rho_hi))
                if not logged_miss and abs(t - self.bet_t) > 1e-9:
                    print(
                        f"    miss-constraint ch={ch} t_eff={t:.0f} lo={rho_lo:.0f} hi={rho_hi:.0f} "
                        f"n_miss={len(self.book.miss_pts[ch])}"
                    )
                logged_miss = True

```

```

nxt = closer_offset(S0, th0, self.xy, t, self.bet_h)
if float(np.linalg.norm(nxt - self.xy)) < 20.0:
    t2 = t + 200.0
    if win is not None:
        t2 = float(np.clip(t2, rho_lo, rho_hi))
    nxt = closer_offset(S0, th0, self.xy, t2, self.bet_h)
body = self.measure_at(nxt, ch)
self.after_stop(exclude_ch=ch)
if ch in self.book.cleared:
    return True
kind = body.get("measure_result")
if kind == "near":
    if self.clear_at(self.xy, ch):
        self.after_stop(exclude_ch=ch)
        return True
if kind == "no_signal":
    nxt2 = second_point(S0, th0, t=t + 220.0, h=-self.bet_h)
    body2 = self.measure_at(nxt2, ch)
    self.after_stop(exclude_ch=ch)
    if ch in self.book.cleared:
        return True
    if body2.get("measure_result") == "near":
        if self.clear_at(self.xy, ch):
            self.after_stop(exclude_ch=ch)
            return True
    continue

if ch in self.book.cleared:
    return True
loc = self.try_localize(ch)
if loc is not None and loc.status == "OK" and loc.vertices is not None:
    r, C = region_center(loc.vertices)
    if r <= CLEAR_R + 1e-6:
        q = self.choose_clear_point(C, r, self._next_xy)
        if self.clear_at(q, ch):
            self.after_stop(exclude_ch=ch)
            return True
print(f"  hunt failed ch{ch}")
if ch not in self.book.cleared:
    self.book.failed.add(ch)
return False

def listen_point(self, p, channels: list, cover_id: str | None = None) -> list:
    """只在 p 扫描，不追击。near 就地清除。cover_id 用于覆盖网扫空。"""
    heard_now = []
    p = np.asarray(p, float)
    ordered = sorted(list(channels), key=lambda c: abs(int(c) - self.last_ch))
    for ch in ordered:
        if ch not in self.book.remaining():
            continue
        if self.book.hits[ch]:
            continue
        body = self.measure_at(p, ch, cover_id=cover_id)
        kind = body.get("measure_result")
        if kind == "near":
            self.clear_at(p, ch)
        elif kind == "direction":
            heard_now.append(ch)
    if self.reuse_stops:
        self.piggyback(p, max_n=3)
        self.harvest_ready(max_dist=160.0)
    return heard_now

def hunt_key(self, ch: int):
    loc = self.try_localize(ch)
    if loc is not None and loc.status == "OK" and loc.vertices is not None:
        r, C = region_center(loc.vertices)
        tier = 0 if r <= CLEAR_R + 1e-6 else 1
        return (tier, float(np.linalg.norm(C - self.xy)))
    return (2, float(np.linalg.norm(estimate_pos(self.book.hits[ch]) - self.xy)))

def chase_extra(self, ch: int, a_next) -> float:
    """P→S2(500,±250)→C(480 m along bearing)→A_next 相对直行 P→A_next 的额外路程。"""
    if a_next is None or not self.book.hits[ch]:

```

```

        return 0.0
    S, th = self.book.hits[ch][-1]
    P = np.asarray(self.xy, float)
    S2 = closer_offset(S, th, P, self.bet_t, self.bet_h)
    C = estimate_pos(self.book.hits[ch], dist=CLEAR_EST)
    A = np.asarray(a_next, float)
    extra = (
        float(np.linalg.norm(S2 - P))
        + float(np.linalg.norm(C - S2))
        + float(np.linalg.norm(A - C))
        - float(np.linalg.norm(A - P))
    )
    return extra

def _chase_t_eff(self, ch: int, S0, th0) -> float:
    t = self.bet_t
    if self.adaptive_step:
        loc = self.try_localize(ch)
        if loc is not None and loc.status == "OK" and loc.vertices is not None:
            _, C = region_center(loc.vertices)
            t = float(np.clip(0.85 * float(np.linalg.norm(np.asarray(C) - S0)), 20.0, 1100.0))
    win = self.miss_bet_window(ch, S0, th0)
    if win is not None:
        t = float(np.clip(t, win[0], win[1]))
    return t

def _channel_entry_xy(self, ch: int, next_xy=None):
    info = self.ready_mec(ch)
    if info is not None:
        r, C = info
        return self.choose_clear_point(C, r, next_xy)
    S0, th0 = self.book.hits[ch][0]
    t = self._chase_t_eff(ch, S0, th0)
    return closer_offset(S0, th0, self.xy, t, self.bet_h)

def _next_ring_xy(self, rings, visited):
    rem = [rings[k] for k in range(len(rings)) if k not in visited]
    if not rem:
        return None
    tour, _ = ring_tour_bruteforce(self.xy, rem)
    return tour[0][1] if tour else None

def _task_pool(self, rings, visited):
    nodes = []
    for k, p in enumerate(rings):
        if k not in visited:
            nodes.append(TaskNode("ring", k, p))
    for ch in self.book.heard():
        if ch in self.hung:
            continue
        info = self.ready_mec(ch)
        if info is not None:
            r, C = info
            q = self.choose_clear_point(C, r, None)
            nodes.append(TaskNode("clear", ch, q, C=C, r=r))
        else:
            S0, th0 = self.book.hits[ch][0]
            t = self._chase_t_eff(ch, S0, th0)
            xy = closer_offset(S0, th0, self.xy, t, self.bet_h)
            nodes.append(TaskNode("chase", ch, xy))
    return nodes

def _replan_dispatch(self, t0, remain):
    print("== replan dispatch ==")
    rings = ring_points()
    visited = set()
    while True:
        if len(self.book.cleared) >= 16:
            print("16 sources cleared, skip remaining ring")
            break
        if not self.book.remaining():
            break
        if time.time() - t0 > remain - 45:
            print("wall-clock budget low, abort remaining ring, drain pending")

```

```

        break
    pool = self._task_pool(rings, visited)
    if not pool:
        break
    tour = solve_open_tour(self.xy, pool)
    first = tour[0]
    nearest = min(pool, key=lambda n: float(np.linalg.norm(n.xy - self.xy)))
    self.n_replan_decisions += 1
    if first.kind != nearest.kind or first.key != nearest.key:
        self.n_replan_diverge += 1
        print(f"replan first={first.label()} nearest={nearest.label()}")
    next_xy = tour[1].xy if len(tour) > 1 else None
    self._next_xy = next_xy
    if first.kind == "ring":
        k = int(first.key)
        p = first.xy
        left = self.book.remaining()
        print(f"-- ring visit R{k} left={left} pool={len(pool)}")
        heard = self.listen_point(p, left, cover_id=f"R{k}")
        visited.add(k)
        self.mark_empty_by_cover()
        a_next = self._next_ring_xy(rings, visited)
        if self.hang:
            for ch in heard:
                extra = self.chase_extra(ch, a_next)
                if a_next is not None and extra > HANG_M:
                    self.hung.add(ch)
                    print(f"    hang ch{ch} extra={extra:.0f}m > {HANG_M:.0f}m")
                else:
                    self.hung.discard(ch)
    elif first.kind == "chase":
        ch = int(first.key)
        if ch not in self.book.heard() or ch in self.book.cleared:
            continue
        self.hunt(ch)
        self.hung.discard(ch)
        self.harvest_ready(max_dist=200.0)
    elif first.kind == "clear":
        ch = int(first.key)
        if ch not in self.book.heard() or ch in self.book.cleared:
            continue
        info = self.ready_mec(ch)
        if info is None:
            self.hunt(ch)
            self.hung.discard(ch)
            continue
        r, C = info
        q = self.choose_clear_point(C, r, next_xy)
        if self.clear_at(q, ch):
            self.after_stop(exclude_ch=ch)
        elif ch not in self.book.cleared:
            self.book.failed.add(ch)
            self.hung.discard(ch)

def hunt_heard(self, skip=None, include_hung: bool = False) -> None:
    skip = set() if skip is None else set(skip)
    if not include_hung:
        skip |= set(self.hung)
    tried = set()
    while True:
        hs = [c for c in self.book.heard() if c not in tried and c not in skip]
        if not hs:
            break
        ch = min(hs, key=self.hunt_key)
        others = [c for c in hs if c != ch]
        if others:
            ch2 = min(others, key=self.hunt_key)
            self._next_xy = self._channel_entry_xy(ch2)
        tried.add(ch)
        self.hunt(ch)
        self.hung.discard(ch)
        self.harvest_ready(max_dist=200.0)

def mark_empty_by_cover(self):

```

```

"""七点覆盖网按监听点编号记账：全部无信号则判空。"""
need = {"0"} | {"R{k}" for k in range(N_RING)}
for ch in list(self.book.remaining()):
    if self.book.hits[ch]:
        continue
    if need <= self.book.miss_ids[ch]:
        self.book.empty.add(ch)
        print(f"-- channel {ch} empty (covered, all no_signal)")

def mark_empty_by_cap(self):
    """题面源数至多 16：清满后剩余频道必为空。"""
    if len(self.book.cleared) < 16:
        return
    for ch in list(self.book.remaining()):
        self.book.empty.add(ch)
        print(f"-- channel {ch} empty (at most 16 sources)")

def run(self, do_enter: bool = True):
    t0 = time.time()
    if do_enter:
        enter = self.sim.enter()
        remain = float(enter.get("remaining_real_duration_s", 1200))
        self.remain_s = remain
        self.wall0 = t0
        print(f"entered remaining_real={remain}s wall={time.strftime('%H:%M:%S')}")
    else:
        remain = float(self.remain_s)
        self.wall0 = t0
        print(f"resume remaining_real~{remain}s wall={time.strftime('%H:%M:%S')}")
    gap = covering_worst_gap()
    print(f"cover net 1+6 R=1150 worst-gap={gap:.1f}m (need <1000)")
    print(
        f"flags hang(ring)={self.hang} origin_hang={self.origin_hang} "
        f"miss_constraint={self.miss_constraint} replan={self.replan} "
        f"onroute_clear={self.onroute_clear} adaptive_step={self.adaptive_step}"
    )

# 1) 原点扫频后再追，保证 20 个频道的覆盖票都在原点
print("== phase 1 origin scan ==")
self.listen_point(np.zeros(2), list(range(1, 21)), cover_id="0")
print(f"after origin listen: cleared={sorted(self.book.cleared)} heard={self.book.heard()}")
if self.origin_hang:
    tour0, tour0_L = ring_tour_bruteforce(self.xy, ring_points())
    a0 = tour0[0][1] if tour0 else None
    print(
        f"origin hang: planned ring L={tour0_L:.1f}m "
        f"first=({a0[0]:.0f},{a0[1]:.0f}) thresh={HANG_M:.0f}m"
    )
    for ch in list(self.book.heard()):
        extra = self.chase_extra(ch, a0)
        if a0 is not None and extra > HANG_M:
            self.hung.add(ch)
            print(f"origin-hang ch{ch} extra={extra:.0f}m > {HANG_M:.0f}m")

if self.replan:
    self._replan_dispatch(t0, remain)
else:
    tour0, _ = ring_tour_bruteforce(self.xy, ring_points())
    self._next_xy = tour0[0][1] if tour0 else None
    self.hunt_heard()
    print(
        f"after origin hunts: cleared={sorted(self.book.cleared)} "
        f"left={self.book.remaining()} hung={sorted(self.hung)}"
    )

# 2) 补网：最短环序；新听源绕路 >300 m 则挂起，整圈后再就近追
print("== phase 2 ring cover ==")
rings = ring_points()
tour, tour_L = ring_tour_bruteforce(self.xy, rings)
print(f"ring tour brute-force L={tour_L:.1f}m hang={self.hang} thresh={HANG_M:.0f}m")
for step_i, (k, p) in enumerate(tour):
    if len(self.book.cleared) >= 16:
        print("16 sources cleared, skip remaining ring")
        break

```

```

        left = self.book.remaining()
        if not left:
            break
        if time.time() - t0 > remain - 45:
            print("wall-clock budget low, abort remaining ring, drain pending")
            break
        a_next = tour[step_i + 1][1] if step_i + 1 < len(tour) else None
        self._next_xy = a_next
        print(f"-- ring {step_i + 1}/{N_RING} left={left}")
        heard = self.listen_point(p, left, cover_id=f"R{k}")
        self.mark_empty_by_cover()
        if self.hang:
            for ch in heard:
                extra = self.chase_extra(ch, a_next)
                if a_next is not None and extra > HANG_M:
                    self.hung.add(ch)
                    print(f"    hang ch{ch} extra={extra:.0f}m > {HANG_M:.0f}m")
                else:
                    self.hung.discard(ch)
                    self.hunt(ch)
                    self.hung.discard(ch)
            self.harvest_ready(max_dist=200.0)
        else:
            self.hunt_heard()

# 环结束或墙钟 abort 后立即 drain: 原点挂起 + 环上挂起均在此就近追
self._next_xy = None
if self.hung or (self.hang and self.book.heard()):
    hung_left = sorted(c for c in self.book.heard() if c in self.hung)
    if hung_left:
        print(f"== drain hung nearest-first {hung_left} ==")
        others = set(self.book.heard()) - set(self.hung)
        self.hunt_heard(skip=others, include_hung=True)
    leftover = sorted(c for c in self.book.heard())
    if leftover:
        print(f"== drain pending nearest-first {leftover} ==")
        self.hunt_heard(include_hung=True)

self.mark_empty_by_cover()
self.mark_empty_by_cap()
print(
    f"done cleared={len(self.book.cleared)} empty={len(self.book.empty)} "
    f"failed={sorted(self.book.failed)} left={self.book.remaining()} "
    f"virtual={self.sim.virtual_time_s:.1f}s wall={time.time()-t0:.1f}s"
)
try:
    ex = self.sim.exit()
    print("exit", {k: ex.get(k) for k in ("accepted", "exit_reason", "virtual_time_s")})
except Exception as e:
    print("exit failed:", e)
return self.summary(time.time() - t0)

def summary(self, wall_s: float) -> dict:
    return {
        "cleared": sorted(self.book.cleared),
        "n_cleared": len(self.book.cleared),
        "empty": sorted(self.book.empty),
        "failed": sorted(self.book.failed),
        "left": self.book.remaining(),
        "clear_virtual_times": self.book.clear_time,
        "virtual_time_s": self.sim.virtual_time_s,
        "avg_s": (float(self.sim.virtual_time_s) / len(self.book.cleared))
        if self.book.cleared
        else None,
        "wall_s": wall_s,
        "n_replan_decisions": self.n_replan_decisions,
        "n_replan_diverge": self.n_replan_diverge,
    }

def save_log(sim, summary: dict, tag: str = "q3"):
    LOG_DIR.mkdir(exist_ok=True)
    stamp = time.strftime("%Y%m%d_%H%M%S")
    out = LOG_DIR / f"{tag}_{stamp}.json"

```

```

payload = {"summary": summary, "log": sim.log_rows[-800:]}
out.write_text(json.dumps(payload, ensure_ascii=False, indent=2, default=str), encoding="utf-8")
print("log written", out)
return out

def run_official(
    resume: bool = False,
    remain_s: float = 1180.0,
    hang: bool = True,
    origin_hang: bool = True,
    miss_constraint: bool = True,
    replan: bool = REPLAN_DEFAULT,
    onroute_clear: bool = True,
    adaptive_step: bool = False,
):
    gap = covering_worst_gap()
    if gap >= 1000.0:
        print(f"WARNING covering gap {gap:.1f} >= 1000")
    sim = Simulator(ROBOT_ID)
    dog = Dog(
        sim,
        remain_s=remain_s,
        hang=hang,
        origin_hang=origin_hang,
        miss_constraint=miss_constraint,
        replan=replan,
        onroute_clear=onroute_clear,
        adaptive_step=adaptive_step,
    )
    try:
        summary = dog.run(do_enter=not resume)
    except ApiError as e:
        print("API error:", e)
        try:
            sim.exit()
        except Exception:
            pass
        summary = {"error": str(e), "cleared": sorted(dog.book.cleared), "n_cleared":
len(dog.book.cleared)}
    save_log(sim, summary)
    return summary

def watch_loop(
    hang: bool = True,
    origin_hang: bool = True,
    miss_constraint: bool = True,
    replan: bool = REPLAN_DEFAULT,
    onroute_clear: bool = True,
    adaptive_step: bool = False,
):
    print("watch: waiting for simulator tests, Ctrl+C to stop")
    n_fail = 0
    while True:
        sim = Simulator(ROBOT_ID)
        try:
            body = sim._post(
                "/enter",
                {"arena_id": "default", "robot_id": sim.robot_id, "request_id": sim._new_id("enter")},
                retries=1,
            )
            if body.get("accepted") is not True:
                n_fail = 0
                time.sleep(2.0)
                continue
        except Exception as e:
            n_fail += 1
            if n_fail <= 2 or n_fail % 8 == 0:
                print(f"enter wait ({n_fail}): {e}")
                time.sleep(3.0)
                continue
            n_fail = 0
        remain = float(body.get("remaining_real_duration_s", 1200))

```

```

print(f"\n==== new test remaining={remain}s {time.strftime('%H:%M:%S')} ====")
dog = Dog(
    sim,
    remain_s=remain,
    hang=hang,
    origin_hang=origin_hang,
    miss_constraint=miss_constraint,
    replan=replan,
    onroute_clear=onroute_clear,
    adaptive_step=adaptive_step,
)
try:
    summary = dog.run(do_enter=False)
except ApiError as e:
    print("API error:", e)
    try:
        sim.exit()
    except Exception:
        pass
    summary = {"error": str(e), "cleared": sorted(dog.book.cleared), "n_cleared":
len(dog.book.cleared)}
    save_log(sim, summary)
    print("waiting for next test...")
    time.sleep(1.5)

def print_local_summary(rows) -> dict:
    avgs = [float(r["avg_s"]) for r in rows if r.get("avg_s") is not None]
    Ts = [float(r["virtual_time_s"]) for r in rows]
    full = sum(1 for r in rows if r.get("full_clear"))
    ratios = [float(r["path_vs_2opt"]) for r in rows if r.get("path_vs_2opt") is not None]
    n_dec = sum(int(r.get("n_replan_decisions") or 0) for r in rows)
    n_div = sum(int(r.get("n_replan_diverge") or 0) for r in rows)
    mean_avg = float(np.mean(avgs)) if avgs else float("nan")
    std_avg = float(np.std(avgs, ddof=1)) if len(avgs) > 1 else 0.0
    mean_T = float(np.mean(Ts)) if Ts else float("nan")
    mean_ratio = float(np.mean(ratios)) if ratios else None
    rec = {
        "n": len(rows),
        "avg_s": mean_avg,
        "std_s": std_avg,
        "mean_T": mean_T,
        "full_clears": full,
        "path_vs_2opt": mean_ratio,
        "n_replan_decisions": n_dec,
        "n_replan_diverge": n_div,
        "replan_diverge_rate": (n_div / n_dec) if n_dec else None,
    }
    ratio_txt = f"{mean_ratio:.3f}" if mean_ratio is not None else "n/a"
    div_txt = f"{n_div}/{n_dec}" if n_dec else "n/a"
    print("=" * 64)
    print(
        f"LOCAL SUMMARY n={len(rows)} avg={mean_avg:.1f}±{std_avg:.1f} s/each "
        f"meanT={mean_T:.0f}s full={full}/{len(rows)} path/2opt={ratio_txt} "
        f"replan#NN={div_txt}"
    )
    print("=" * 64)
    return rec

def run_local(
    n_ep: int = 4,
    hang: bool = True,
    origin_hang: bool = True,
    miss_constraint: bool = True,
    replan: bool = REPLAN_DEFAULT,
    onroute_clear: bool = True,
    adaptive_step: bool = False,
):
    from local_world import LocalSimulator

    rows = []
    for i in range(n_ep):
        rng = np.random.default_rng(20260911 + i)

```

```

sim = LocalSimulator(rng)
print(
    f"\n==== local ep {i+1} n={sim.n_true} hang={hang} "
    f"origin_hang={origin_hang} miss={miss_constraint} "
    f"replan={replan} onroute={onroute_clear} adaptive={adaptive_step} ==="
)
dog = Dog(
    sim,
    hang=hang,
    origin_hang=origin_hang,
    miss_constraint=miss_constraint,
    replan=replan,
    onroute_clear=onroute_clear,
    adaptive_step=adaptive_step,
)
summary = dog.run()
alive = [ch for ch, s in sim.sources.items() if s["alive"]]
n = max(int(summary["n_cleared"]), 1)
avg = float(summary["virtual_time_s"]) / n
summary["alive_left"] = alive
summary["n_true"] = sim.n_true
summary["avg_s"] = avg
summary["full_clear"] = (not alive) and (int(summary["n_cleared"]) == int(sim.n_true))
traj = path_from_log(sim.log_rows)
actual = actual_path_length(traj)
stops = unique_stops(traj)
if len(stops) >= 3 and actual > 1.0:
    _ord, opt_L = two_opt_tour(stops, np.zeros(2))
    ratio = actual / opt_L if opt_L > 1e-9 else None
    summary["path_actual_m"] = actual
    summary["path_2opt_m"] = opt_L
    summary["path_vs_2opt"] = ratio
    print(
        f"path vs 2-opt: actual={actual:.0f}m 2opt={opt_L:.0f}m "
        f"ratio={ratio:.3f} n_stops={len(stops)}"
    )
else:
    print("path vs 2-opt: skipped (no trajectory / too few stops)")
rows.append(summary)
print(
    f"alive_left {alive} cleared {summary['n_cleared']}/{sim.n_true} "
    f"T={summary['virtual_time_s']:.0f}s avg={avg:.0f}s "
    f"full={int(summary['full_clear'])}"
)
print_local_summary(rows)
return rows

def dog_kwargs(args) -> dict:
    return {
        "hang": not args.no_hang,
        "origin_hang": not args.no_origin_hang,
        "miss_constraint": not args.no_miss_constraint,
        "replan": args.replan,
        "onroute_clear": not args.no_onroute_clear,
        "adaptive_step": args.adaptive_step,
    }

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--local", action="store_true")
    ap.add_argument("--watch", action="store_true")
    ap.add_argument("--resume", action="store_true")
    ap.add_argument("--episodes", type=int, default=4)
    ap.add_argument(
        "--no-hang",
        action="store_true",
        help="disable ring-phase hang (chase immediately on ring); origin hang still ON unless --no-origin-hang",
    )
    ap.add_argument(
        "--no-origin-hang",
        action="store_true",
    )

```

```

        help="disable origin-phase hang (chase all heard after origin scan); ring hang still ON
unless --no-hang",
    )
    ap.add_argument(
        "--no-miss-constraint",
        action="store_true",
        help="disable miss-based |G-M|>1000 longitudinal bet clipping (always t=BET_T)",
    )
    replan_g = ap.add_mutually_exclusive_group()
    replan_g.add_argument(
        "--replan",
        dest="replan",
        action="store_true",
        help="enable rolling replan dispatcher (open tour on pool; execute first node only; default
off)",
    )
    replan_g.add_argument(
        "--no-replan",
        dest="replan",
        action="store_false",
        help="disable rolling replan; sequential origin-hunt then ring (current default)",
    )
    ap.set_defaults(replan=REPLAN_DEFAULT)
    ap.add_argument(
        "--no-onroute-clear",
        action="store_true",
        help="clear at C_MEC instead of shifting toward the next planned node",
    )
    ap.add_argument(
        "--adaptive-step",
        action="store_true",
        help="if MEC center C exists, second-point t=clip(0.85*|C-S0|,20,1100) instead of BET_T",
    )
    args = ap.parse_args()
    kw = dog_kwargs(args)
    if args.local:
        run_local(n_ep=args.episodes, **kw)
        return
    if args.watch:
        watch_loop(**kw)
        return
    run_official(resume=args.resume, **kw)

if __name__ == "__main__":
    main()

```

程序 q4/ledger.py

```

# -*- coding: utf-8 -*-
"""Q4 v2 双账状态层: 探针集合 (probe set) + 覆盖证书判空.

核心升级 (相对 q4/ledger.py):
1. 每个频道记录**实际测量过的位置** (probes), 而不是离散骨架布尔数组.
2. 判空判据 = "探针集合对全圆域 G 满足环抱条件":
    max_{G in disk(1800)} maxgap(probes n B(G,1000)) <= 180°
    这是定向源 (±90°) + 全向源的**统一、充要**覆盖证书:
    - 若 G 落在 conv({p in probes : |p-G|<=1000}) 内部, 则无论朝向 φ 如何,
      必有探针落在 G 的 180° 覆盖半平面内 -> 必被检出.
    - 反之存在朝向使全圆域内探针都背对 -> 可能漏检.
3. 证书只依赖"探针几何", 因此追踪途中的停点也自动计入证书 (信息复用).
"""
import numpy as np
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(os.path.dirname(__file__)), "q2"))
from geometry import DELTA, wedge_hp, clip, region, diam_and_axis, mec, cross2

# ===== 常量 =====
ARENA_R = 1800.0
RECV_R_MIN = 1000.0
RECV_R_MAX = 1500.0

```

```

CLEAR_R = 20.0
NEAR_R = 5.0
SPEED = 5.0
SWITCH_COST = 1.0
MEASURE_COST = 5.0
CLEAR_COST = 5.0
N_CHANNELS = 20

CERT_MARGIN = -1e-6 # 证书阈值 = 180 - CERT_MARGIN = 180 + 1e-6
# 定向源覆盖角为**闭**半平面 ( $\pm 90^\circ$  含), 故 maxgap == 180° 仍成立
# (此时任意闭 180° 扇区必含一个探针方向); 仅留浮点容差.

# ===== 覆盖证书点集 =====
def _make_skeleton():
    d = os.path.dirname(os.path.abspath(__file__))
    # 只加载**已严格校验**的点集; skeleton_best.npy 是扫描候选 (可能未通过超密校验), 不加载.
    for nm in ("skeleton_final.npy", "skeleton_opt2.npy", "skeleton_opt.npy"):
        p = os.path.join(d, nm)
        if os.path.exists(p):
            return np.load(p)
    def ring(r, k, ph=0.0):
        a = ph + np.arange(k) * 2 * np.pi / k
        return np.column_stack([r * np.cos(a), r * np.sin(a)])
    return np.vstack([ring(400, 6), ring(1300, 8), ring(1900, 12)])

SKELETON = _make_skeleton()
SKELETON_N = len(SKELETON)

# ===== 证书采样点 =====
def _make_G_samples():
    """判空证书的检验点 (必须足够密, 否则可能误判为空 -> 漏源)."""
    pts = []
    rr = 50.0
    while rr < ARENA_R:
        n = max(60, int(2 * np.pi * rr / 50))
        a = np.arange(n) * 2 * np.pi / n
        pts.append(np.column_stack([rr * np.cos(a), rr * np.sin(a)]))
        rr += 50.0
    for rr in (1750.0, 1780.0, 1800.0):
        n = 720
        a = np.arange(n) * 2 * np.pi / n
        pts.append(np.column_stack([rr * np.cos(a), rr * np.sin(a)]))
    return np.vstack(pts)

G_SAMPLES = _make_G_samples()

def max_gap_all(probes, Gs=None):
    """返回全部 G 的最大角度间隙的最大值 (度). 向量化."""
    Gs = G_SAMPLES if Gs is None else Gs
    probes = np.asarray(probes, float)
    if len(probes) < 3:
        return 999.0
    diff = Gs[:, None, :] - probes[None, :, :]
    d2 = (diff ** 2).sum(-1)
    mask = d2 <= RECV_R_MIN ** 2 + 1e-9
    if not mask.any():
        return 999.0
    ang = np.arctan2(diff[:, :, 1], diff[:, :, 0])
    ang = np.where(mask, ang, np.nan)
    ang.sort(axis=1)
    k = np.sum(~np.isnan(ang), axis=1)
    m = len(Gs)
    gaps = np.diff(ang, axis=1)
    gaps = np.where(np.isnan(gaps), -1.0, gaps)
    gmax = gaps.max(axis=1) if gaps.shape[1] else np.full(m, -1.0)
    last = ang[np.arange(m), np.clip(k - 1, 0, ang.shape[1] - 1)]
    wrap = ang[:, 0] + 2 * np.pi - last
    wrap = np.where(k >= 1, wrap, 2 * np.pi)
    gap = np.maximum(gmax, wrap)
    gap = np.where(k == 0, np.inf, gap)
    return float(np.degrees(gap.max()))

```

```

# ===== 多边形工具 =====
def disk_poly(S, R, n=16):
    S = np.asarray(S, float)
    a = np.linspace(0, 2 * np.pi, n, endpoint=False)
    return S + R * np.column_stack([np.cos(a), np.sin(a)])

def poly_to_halfplanes(poly):
    hps = []
    n = len(poly)
    for i in range(n):
        a = poly[i]; b = poly[(i + 1) % n]
        edge = b - a
        nm = np.linalg.norm(edge)
        if nm > 1e-12:
            edge = edge / nm
        hps.append((a, edge, +1))
    return hps

def intersect_disk(poly, S, R, n=64):
    if len(poly) < 3:
        return np.zeros((0, 2))
    for hp_S, hp_e, hp_s in poly_to_halfplanes(disk_poly(S, R, n)):
        if len(poly) < 3:
            return np.zeros((0, 2))
        poly = clip(poly, hp_S, hp_e, hp_s)
    return poly

def poly_area(poly):
    if len(poly) < 3:
        return 0.0
    return 0.5 * abs(cross2(poly[0], poly[-1]) +
                     sum(cross2(poly[i], poly[(i + 1) % len(poly)])
                         for i in range(len(poly) - 1)))

def poly_centroid(poly):
    if len(poly) == 0:
        return np.array([0., 0.])
    return poly.mean(axis=0)

# ===== 频道状态 =====
class ChannelState:
    def __init__(self, ch):
        self.ch = ch
        self.state = "unknown"
        self.bearings = []          # [(pos, bearing_deg), ...]
        self.W = None               # 候选凸多边形
        self.probes = []            # 测量位置 (仅 no_signal / direction 都会记录)
        self._mec = None
        self._diam = None
        self._axis = None
        self._certified = None      # 缓存: (n_probes, bool)

    @property
    def is_active(self):
        return self.state not in ("cleared", "empty")

    @property
    def mec(self):
        if self._mec is None and self.W is not None and len(self.W) >= 3:
            r, c = mec(self.W)
            self._mec = (r, c)
        return self._mec

    @property
    def diameter(self):
        if self._diam is None and self.W is not None and len(self.W) >= 3:

```

```

        d, ax = diam_and_axis(self.W)
        self._diam = d; self._axis = ax
    return self._diam if self._diam is not None else 0.0

@property
def axis(self):
    if self._axis is None and self.W is not None and len(self.W) >= 3:
        d, ax = diam_and_axis(self.W)
        self._diam = d; self._axis = ax
    return self._axis

@property
def is_clearable(self):
    m = self.mec
    return m is not None and m[0] <= CLEAR_R

@property
def centroid(self):
    if self.W is not None and len(self.W) >= 1:
        return poly_centroid(self.W)
    return None

def invalidate(self):
    self._mec = None; self._diam = None; self._axis = None

# ===== 总管 =====
class Ledger:
    def __init__(self):
        self.channels = {ch: ChannelState(ch) for ch in range(1, N_CHANNELS + 1)}
        self.cur_pos = np.array([0., 0.])
        self.cur_channel = 1
        self.vt = 0.0
        self.cleared = set()
        self._cert_cache = {}

    # ---- 探针 ----
    def add_probe(self, ch, S):
        cs = self.channels[ch]
        cs.probes.append(np.asarray(S, float).copy())

    def check_empty(self, ch):
        """环抱证书: 探针集合对全圆域环抱 -> 频道必空."""
        cs = self.channels[ch]
        n = len(cs.probes)
        if n < 3:
            return False
        c = self._cert_cache.get(ch)
        if c is not None and c[0] == n:
            return c[1]
        if n > 40:
            # 只保留最外圈探针以节省计算 (内点对环抱贡献小)
            P = np.asarray(cs.probes, float)
            r = np.linalg.norm(P, axis=1)
            idx = np.argsort(-r)[:40]
            P = P[idx]
        else:
            P = np.asarray(cs.probes, float)
            ok = max_gap_all(P) <= 180.0 - CERT_MARGIN
            self._cert_cache[ch] = (n, ok)
        return ok

    # ---- 更新 ----
    def update_no_signal(self, ch, S):
        self.add_probe(ch, S)

    def update_direction(self, ch, S, bearing_deg):
        cs = self.channels[ch]
        S = np.asarray(S, float)
        th = np.radians(bearing_deg)
        cs.bearings.append((S.copy(), bearing_deg))
        self.add_probe(ch, S)
        if cs.W is None:

```

```

        cs.W = disk_poly([0., 0.], ARENA_R, 32)
    for hp_S, hp_e, hp_s in wedge_hp(S, th):
        if len(cs.W) >= 3:
            cs.W = clip(cs.W, hp_S, hp_e, hp_s)
    if len(cs.W) >= 3:
        cs.W = intersect_disk(cs.W, S, RECV_R_MAX, 64)
    if cs.state == "unknown":
        cs.state = "detected"
    cs.invalidate()

def update_near(self, ch, S):
    cs = self.channels[ch]
    cs.state = "clearable"
    cs._mec = (0.0, np.asarray(S, float))
    cs._diam = 0.0

def update_cleared(self, ch):
    cs = self.channels[ch]
    cs.state = "cleared"
    self.cleared.add(ch)

def update_clear_fail(self, ch):
    cs = self.channels[ch]
    cs.state = "detected"
    cs._mec = None; cs._diam = None

def check_clearable(self, ch):
    cs = self.channels[ch]
    if cs.is_clearable and cs.state == "detected":
        cs.state = "clearable"
    return cs.state == "clearable"

# ---- 查询 ----
def is_channel_empty(self, ch):
    cs = self.channels[ch]
    if cs.state == "empty":
        return True
    return self.check_empty(ch)

def mark_empty(self):
    for ch in range(1, N_CHANNELS + 1):
        cs = self.channels[ch]
        if cs.state == "unknown" and self.check_empty(ch):
            cs.state = "empty"

def active_channels(self):
    return [ch for ch in range(1, N_CHANNELS + 1) if self.channels[ch].is_active]

def unknown_channels(self):
    return [ch for ch in range(1, N_CHANNELS + 1) if self.channels[ch].state == "unknown"]

def detected_channels(self):
    return [ch for ch in range(1, N_CHANNELS + 1)
            if self.channels[ch].state in ("detected", "clearable")]

def clearable_channels(self):
    return [ch for ch in range(1, N_CHANNELS + 1) if self.channels[ch].state == "clearable"]

def resolved(self):
    return len(self.unknown_channels()) == 0 and len(self.detected_channels()) == 0

def is_terminated(self):
    for ch in range(1, N_CHANNELS + 1):
        cs = self.channels[ch]
        if cs.state in ("cleared", "empty"):
            continue
        if cs.state == "unknown" and self.check_empty(ch):
            cs.state = "empty"
            continue
    return False
return True

```

程序 q4/scheduler.py

```
# -*- coding: utf-8 -*-
"""Q3 调度器 (§5.4 + §5.5).

SmartStart:  $L \leq (\theta-1) \cdot t_{\text{已耗}} \Rightarrow$  收割, 否则探索 [Birx-Disser]
2-opt 收割巡回: 最近邻初解 + 2-opt 改进, 环形序约束 [Kamiński Thm 10]
CELLF: 停点内频道选择, 惰性优先队列 [Krause §1.4 / Leskovec KDD'07]
"""

import numpy as np
import heapq
from ledger import Ledger, SPEED, SWITCH_COST, MEASURE_COST, RECV_R_MIN, RECV_R_MAX

# ===== SmartStart =====
def harvest_tour_length(ledger):
    """可清除队列各 MEC 中心的 2-opt 巡回长度 (虚拟秒)."""
    clearable = ledger.clearable_channels()
    if not clearable:
        return 0.0
    pts = []
    for ch in clearable:
        m = ledger.channels[ch].mec
        if m is not None and m[1] is not None:
            pts.append(m[1])
        else:
            c = ledger.channels[ch].centroid
            if c is not None:
                pts.append(c)
    if not pts:
        return 0.0
    tour, dist = two_opt_tour(pts, ledger.cur_pos)
    return dist / SPEED

def smartstart_check(ledger, theta=2.05):
    """ $L \leq (\theta-1) \cdot t_{\text{已耗}} \Rightarrow$  True (出发收割)."""
    L = harvest_tour_length(ledger)
    t_elapsed = ledger.vt
    return L <= (theta - 1.0) * max(t_elapsed, 1.0)

# ===== 2-opt TSP =====
def two_opt_tour(points, start_pos):
    """2-opt 巡回: 从 start_pos 出发, 访问全部 points, 不需返程.

    返回 (order: [idx], total_dist: float).
    """
    n = len(points)
    if n == 0:
        return [], 0.0
    pts = [np.asarray(p, float) for p in points]
    start = np.asarray(start_pos, float)

    # 初解: 极角序 (环形序约束, Kamiński Thm 10)
    if n >= 2:
        angles = [np.arctan2(p[1] - start[1], p[0] - start[0]) for p in pts]
        order = np.argsort(angles).tolist()
    else:
        order = [0]

    # 最近邻改进初解
    order = _nearest_neighbor(pts, start)

    # 计算总距离 (不返程)
    def tour_dist(ord_):
        d = np.linalg.norm(pts[ord_[0]] - start)
        for i in range(len(ord_) - 1):
            d += np.linalg.norm(pts[ord_[i + 1]] - pts[ord_[i]])
        return d

    best_order = list(order)
    best_dist = tour_dist(best_order)
```

```

# 2-opt
improved = True
while improved:
    improved = False
    for i in range(n - 1):
        for j in range(i + 1, n):
            new_order = best_order[:i] + best_order[i:j + 1][::-1] + best_order[j + 1:]
            new_dist = tour_dist(new_order)
            if new_dist < best_dist - 1e-6:
                best_dist = new_dist
                best_order = new_order
                improved = True
    # 只做一轮 (n 小, 足够)
    break

return best_order, best_dist

def _nearest_neighbor(pts, start):
    """最近邻初解."""
    n = len(pts)
    visited = [False] * n
    order = []
    cur = start
    for _ in range(n):
        best_i = -1
        best_d = np.inf
        for i in range(n):
            if not visited[i]:
                d = np.linalg.norm(pts[i] - cur)
                if d < best_d:
                    best_d = d
                    best_i = i
        order.append(best_i)
        visited[best_i] = True
        cur = pts[best_i]
    return order

# ===== 收割批次 =====
def harvest_batch_plan(ledger):
    """生成收割批次: (频道, MEC 中心) 列表, 2-opt+环形序排序."""
    clearable = ledger.clearable_channels()
    if not clearable:
        return []

    pts = []
    chs = []
    for ch in clearable:
        cs = ledger.channels[ch]
        m = cs.mec
        center = m[1] if (m and m[1] is not None) else cs.centroid
        if center is not None:
            pts.append(center)
            chs.append(ch)

    if not pts:
        return []

    order, dist = two_opt_tour(pts, ledger.cur_pos)
    return [(chs[order[i]], pts[order[i]]) for i in range(len(order))]

# ===== CELF 停点内频道调度 =====
def celf_select(ledger, p, budget_channels=8):
    """CELF lazy greedy: 在停点 p 选择要测量的频道组合.

    收益 = 探索排除面积 (unknown) + E/D 精化增量 (detected)
    成本 = 5s (measure) + 1s (switch if ch != cur)
    """
    candidates = []
    cur_ch = ledger.cur_channel

```

```

# unknown 频道: 探索收益
for ch in ledger.unknown_channels():
    area = ledger.exploration_area(ch, p)
    cost = MEASURE_COST + (SWITCH_COST if ch != cur_ch else 0)
    if area > 0:
        candidates.append((area / cost, ch, area, cost))

# detected 频道: 精化收益 (简化: 距候选域近才测)
from score import bearing_scatter_matrix, det2
for ch in ledger.detected_channels():
    cs = ledger.channels[ch]
    if cs.W is None or len(cs.W) < 3 or len(cs.bearings) == 0:
        continue
    centroid = cs.centroid
    if centroid is None:
        continue
    d = np.linalg.norm(np.asarray(p, float) - centroid)
    if d > RECV_R_MAX:
        continue
    # E/D 增量
    S_cur = bearing_scatter_matrix(cs.bearings, centroid)
    det_cur = max(det2(S_cur), 1e-15)
    diff = centroid - np.asarray(p, float)
    d2 = max(float(diff @ diff), 1.0)
    theta = np.arctan2(diff[1], diff[0])
    u_perp = np.array([-np.sin(theta), np.cos(theta)])
    dS = np.outer(u_perp, u_perp) / d2
    det_new = max(det2(S_cur + dS), 1e-15)
    benefit = cs.diameter * (np.log(det_new) - np.log(det_cur))
    cost = MEASURE_COST + (SWITCH_COST if ch != cur_ch else 0)
    if benefit > 0:
        candidates.append((benefit / cost, ch, benefit, cost))

# 按效益/成本比降序
candidates.sort(key=lambda x: -x[0])

# 选择直到预算耗尽
selected = []
total_cost = 0.0
for ratio, ch, benefit, cost in candidates:
    if total_cost + cost > budget_channels * (MEASURE_COST + SWITCH_COST):
        break
    if benefit <= 0:
        break
    selected.append(ch)
    total_cost += cost

return selected

def en_route_clear(ledger, dest, threshold=50.0):
    """顺路清除: 移动到 dest 途中, 距可清除源 MEC ≤ threshold 返回该频道."""
    p = np.asarray(dest, float)
    for ch in ledger.clearable_channels():
        cs = ledger.channels[ch]
        m = cs.mec
        if m and m[1] is not None:
            d = np.linalg.norm(m[1] - p)
            if d <= threshold:
                return ch, m[1]
    return None

```

程序 q4/policy.py

```

# -*- coding: utf-8 -*-
"""Q4 v3 主策略: **合并巡回** (证书 + 定位 + 清除) + 长轴切割 + 末段 TSP 清尾.

对症 v2 (policy2) 的残余浪费:
- v2 先单独 hunt 一轮 (seed1: 6499m/1375s), 再走证书巡回; 两者路径重复.
- v3 取消独立 hunt: 证书巡回途中用"顺路自由交会"(_refine) 收集方位,

```

- 凡 $MEC \leq 20m$ 且绕行代价小的频道就地清除，其余统一交给末段 TSP 清尾。
- 末段 TSP：从当前点出发对剩余待清源做 2-opt 巡回，而不是逐个螺旋重试。

巡回节点集合 = 证书骨架点 $\cup$ 已检测源的 MEC 中心 (动态重规划)。

```

"""
import numpy as np
import math
import sys, os

sys.path.insert(0, os.path.join(os.path.dirname(os.path.dirname(__file__)), "q2"))
from geometry import diam_and_axis, mec as compute_mec

from ledger2 import (Ledger, SKELETON, SKELETON_N, ARENA_R, RECV_R_MIN, RECV_R_MAX,
                     CLEAR_R, NEAR_R, SPEED, SWITCH_COST, MEASURE_COST, N_CHANNELS,
                     poly_centroid, max_gap_all, G_SAMPLES)

REAL_TIME_LIMIT = 16.5 * 60

BET_T = 500.0
BET_H = 250.0
MAX_PURSUIT_STEPS = 6
CLEAR_TOL = CLEAR_R
CUT_RHO_MIN = 60.0

ENROUTE_MAX = 420.0          # 巡回途中顺路清除的最大单程绕行
REFINE_D_MAX = 1250.0        # 顺路交会的作用距离
REFINE_BUDGET = 2
REFINE_MIN_ANG = 45.0        # 新方位与已有方位的最小夹角 (度)
SRC_NODE_TOL = 60.0          # 源作为巡回节点的位置不确定度上限
SRC_COOLDOWN = 6             # 一个源节点服务后冷却的节点数

LIT_BONUS = 260.0            # 候选点落在"已点亮"半平面的距离奖励 (m)
LIT_COS = math.cos(math.radians(50.0))
BLIND_PEN = 600.0            # 候选点落在已知"黑侧"的惩罚 (m)
BLIND_COS = math.cos(math.radians(55.0))
MAX_BLIND = 6

def _unit(deg):
    t = math.radians(deg)
    return np.array([math.cos(t), math.sin(t)], dtype=float)

def _clamp(p, R=2.0e6):
    p = np.asarray(p, float)
    n = float(np.linalg.norm(p))
    if n > R:
        p = p * (R / n)
    return p

def _perp(e):
    return np.array([-e[1], e[0]])

def _tsp_open(pts, start, passes=5):
    """开放路径 TSP (起点固定 start), 返回 (长度, 访问次序索引)."""
    pts = np.asarray(pts, float)
    n = len(pts)
    if n == 0:
        return 0.0, []
    A = np.vstack([np.asarray(start, float), pts])
    D = np.linalg.norm(A[:, None, :] - A[None, :, :], axis=-1)
    N = n + 1
    un = list(range(1, N))
    order = [0]; cur = 0
    while un:
        nxt = min(un, key=lambda j: D[cur, j])
        order.append(nxt); un.remove(nxt); cur = nxt
    best = order[:n]

    def L(o):
        o = np.asarray(o)

```

```

        return float(D[o[:-1], o[1:]].sum())

    bl = L(best)
    for _ in range(passes):
        imp = False
        for i in range(1, N - 1):
            for j in range(i + 1, N):
                no = best[:i] + best[i:j + 1][::-1] + best[j + 1:]
                l = L(no)
                if l < bl - 1e-9:
                    best, bl, imp = no, l, True
        if not imp:
            break
    return bl, [o - 1 for o in best[1:]]

class Policy3:
    def __init__(self, sim, verbose=False, refine_budget=REFINE_BUDGET):
        self.sim = sim
        self.ledger = Ledger()
        self.verbose = verbose
        self.refine_budget = refine_budget
        self._visited = []
        self._tried_pts = {}
        self._src_cool = {}
        self._blind = {}
        self._fin_log = []
        self._phase_stats = {}
        # 诊断
        self._last_pos = np.zeros(2)
        self.mv = 0.0
        self.n_meas = 0
        self.n_clear = 0
        self.n_fail = 0
        self.ch_meas = {}
        self._stats_acc = {}
        self._cur_phase = None
        self._ph = (0.0, 0.0, 0, 0)
        self._tour_len = 0.0
        self._order_len = 0.0

    # ----- 诊断 -----
    def _bump(self, pos):
        p = np.asarray(pos, float)
        self.mv += float(np.linalg.norm(p - self._last_pos))
        self._last_pos = p

    def _phase(self, name):
        if self._cur_phase is not None:
            self._stats_acc[self._cur_phase] = (
                self.sim.vt - self._ph[0], self.mv - self._ph[1],
                self.n_meas - self._ph[2], self.n_clear - self._ph[3])
        self._cur_phase = name
        self._ph = (self.sim.vt, self.mv, self.n_meas, self.n_clear)

    # ----- 基础 -----
    def _sync(self):
        self.ledger.cur_pos = np.array(self.sim.cur_pos, float)
        self.ledger.cur_channel = self.sim.cur_channel
        self.ledger.vt = self.sim.vt
        self.ledger.cleared = self.sim.cleared

    def _measure(self, pos, ch):
        pos = _clamp(pos)
        self.n_meas += 1
        self.ch_meas[ch] = self.ch_meas.get(ch, 0) + 1
        resp = self.sim.measure(tuple(pos), ch)
        self._bump(pos)
        mr = resp.get("measure_result")
        if mr == "direction":
            self.ledger.update_direction(ch, pos, resp["svd_deg"])
        elif mr == "near":
            self.ledger.update_near(ch, pos)

```

```

        if self.sim.clear(tuple(np.asarray(self.sim.cur_pos, float)), ch):
            self.ledger.update_cleared(ch)
        else:
            self.ledger.update_no_signal(ch, pos)
            self._visited.append(np.asarray(pos, float))
            self._sync()
            return mr

def _clear_at(self, ch, pos):
    pos = _clamp(pos)
    self.n_clear += 1
    ok = self.sim.clear(tuple(pos), ch)
    self._bump(pos)
    if ok:
        self.ledger.update_cleared(ch)
    else:
        self.n_fail += 1
        self.ledger.update_clear_fail(ch)
    self._sync()
    return ok

def _mec_center(self, ch):
    cs = self.ledger.channels[ch]
    m = cs.mec
    if m is None or m[1] is None:
        return None
    return np.asarray(m[1], float)

def _try_clear(self, ch, force=False):
    """尝试清除 ch.

    默认: 仅当 MEC 半径 <= CLEAR_TOL 时尝试.
    force=True: 若 MEC 不收敛但已有方位 (centroid 已知), 按候选多边形几何
    中心兜底尝试一次. 失败无害 (5 s 虚拟时间); 成功可救回接近共线/单方位源.
    """
    cs = self.ledger.channels[ch]
    m = cs.mec
    if m is not None and m[1] is not None and m[0] <= CLEAR_TOL:
        return self._clear_at(ch, m[1])
    if force and cs.bearings:
        cen = cs.centroid
        if cen is not None:
            return self._clear_at(ch, np.asarray(cen, float))
    return False

# ----- 追踪 (定向源: 从"已点亮"侧逼近) -----
def _lit_dir(self, ch):
    """由已有 bearing 的测量点方向给出"已点亮"平均方向 (从 MEC 中心看出)."""
    cs = self.ledger.channels[ch]
    m = cs.mec
    if m is None or m[1] is None:
        return None
    C = np.asarray(m[1], float)
    u = np.zeros(2)
    for S, _ in cs.bearings:
        v = np.asarray(S, float) - C
        n = float(np.linalg.norm(v))
        if n > 1.0:
            u += v / n
    n = float(np.linalg.norm(u))
    if n < 1e-6:
        return None
    return u / n

def _cand_points(self, ch):
    """候选测量点池.

    (a) Q2 单方位推进点:  $S_0 + T \cdot e \pm k \cdot H \cdot n$  ( $k=1,2,3$ ) — 单 bearing 时最有效,
    且两个镜像点中通常恰有一个落在定向源的**点亮**半平面内.
    (b) **长条端采样** (关键修复): 两条近乎平行的示向度会把 MEC 拉成长条
    (半径可达 ~700m), 而源必落在长条内部. 旧版只在 C 周围  $\rho \leq 380m$  布点,
    永远够不到长条两端 -> 反复 no_signal, 单源烧掉数千秒.
    """

```

现沿 MEC 主轴 $\pm t \cdot R$ 与"最后一条示向度"方向 $t \cdot R$ 采样, 直达两端:
两个镜像端中恰有一端落在定向源的点亮半平面内.

(c) MEC 中心 C 周围的 12 向 + 已点亮方向, 半径自适应 ($\sim 0.9 \cdot R$, 上界 420m).

```

"""
cs = self.ledger.channels[ch]
out = []
m = cs.mec
R = float(m[0]) if (m is not None and m[1] is not None) else 0.0
if cs.bearings:
    S0, th0 = cs.bearings[-1]
    e = _unit(th0); n = _perp(e)
    S0 = np.asarray(S0, float)
    for mult in (1.0, 2.0, 3.0):
        for sgn in (1.0, -1.0):
            out.append(S0 + BET_T * e + sgn * mult * BET_H * n)
if m is not None and m[1] is not None:
    C = np.asarray(m[1], float)
    L = cs.diameter
    if L is None or L <= 1e-9:
        L = max(2.0 * R, 120.0)
    # (b) 长条端采样: 沿主轴两镜像端 + 沿最后示向度方向的远端
    ax = cs.axis
    if R > 40.0 and ax is not None:
        u = np.asarray(ax, float)
        nu = float(np.linalg.norm(u))
        if nu > 1e-9:
            u = u / nu
            for t in (0.5, 0.85, 1.2, 1.55):
                for sgn in (1.0, -1.0):
                    out.append(C + sgn * t * R * u)
    if R > 40.0 and cs.bearings:
        e = _unit(cs.bearings[-1][1])
        for t in (0.6, 1.0, 1.4):
            out.append(C + t * R * e)
    # (c) 各向候选环
    rho = max(90.0, min(0.9 * R + 30.0, 420.0))
    lu = self._lit_dir(ch)
    if lu is not None:
        out.append(C + rho * lu)
    for k in range(12):
        a = 2 * math.pi * k / 12.0
        out.append(C + rho * np.array([math.cos(a), math.sin(a)]))
return out

def _tried(self, ch):
    return self._tried_pts.setdefault(ch, [])

def _mec_center_raw(self, ch):
    cs = self.ledger.channels[ch]
    m = cs.mec
    return None if (m is None or m[1] is None) else np.asarray(m[1], float)

def _next_cand(self, ch):
    """选下一个未试过的候选点: 距离为主, 点亮方向轻微加权, 黑侧轻微惩罚.

    长条 MEC (R>150, 近共线双方位) 时, 额外给**沿主轴远端**的候选加分,
    使追踪优先直达长条两端 (源就在其中一端), 而不是在中间反复试探.
    """
    cur = np.asarray(self.sim.cur_pos, float)
    C = self._mec_center_raw(ch)
    cands = self._cand_points(ch)
    tried = self._tried(ch)
    u = self._lit_dir(ch)
    blind = self._blind.get(ch, [])
    cs = self.ledger.channels[ch]
    R = float(cs.mec[0]) if (cs.mec is not None and cs.mec[1] is not None) else 0.0
    axv = None
    if R > 150.0 and cs.axis is not None:
        a = np.asarray(cs.axis, float)
        na = float(np.linalg.norm(a))
        if na > 1e-9:
            axv = a / na
    best = None; bs = 1e18

```

```

for c in cand:
    if not np.all(np.isfinite(c)):
        continue
    d0 = float(np.linalg.norm(c - cur))
    if d0 < 15.0:
        continue
    if any(np.linalg.norm(c - t) < 26.0 for t in tried):
        continue
    if self._is_visited(c, tol=18.0):
        continue
    sc = d0
    if C is not None:
        v = c - C
        nv = float(np.linalg.norm(v))
        if nv > 1e-6:
            v = v / nv
            if u is not None and float(v @ u) > LIT_COS:
                sc -= LIT_BONUS
            for b in blind:
                if float(v @ b) > BLIND_COS:
                    sc += BLIND_PEN
            break
    if axv is not None:
        sc -= 0.45 * abs(float((c - C) @ axv)) # 长条端优先
    if sc < bs:
        bs = sc; best = c
return best

def pursue(self, ch, max_steps=MAX_PURSUIT_STEPS):
    """有界追踪：每步选一个未试过的候选点；no_signal 则记黑侧方向并换点."""
    meas = 0
    while meas < max_steps:
        if ch in self.sim.cleared:
            return True
        cs = self.ledger.channels[ch]
        if not cs.is_active:
            return True
        if self._try_clear(ch):
            return True
        if not cs.bearings:
            return False
        C0 = self._mec_center_raw(ch)
        cand = self._next_cand(ch)
        if cand is None:
            break
        cand = _clamp(cand)
        self._tried(ch).append(np.asarray(cand, float).copy())
        mr = self._measure(cand, ch)
        if mr == "no_signal" and C0 is not None:
            v = np.asarray(cand, float) - C0
            nv = float(np.linalg.norm(v))
            if nv > 1e-6:
                bl = self._blind.setdefault(ch, [])
                bl.append(v / nv)
                if len(bl) > MAX_BLIND:
                    bl.pop(0)
        meas += 1
    return self._try_clear(ch)

def _is_visited(self, p, tol=25.0):
    p = np.asarray(p, float)
    for v in self._visited[-24:]:
        if np.linalg.norm(p - v) < tol:
            return True
    return False

# ----- 顺路交会 -----
def _refine_at(self, pos, budget=None):
    """在停点对已检出未清除频道补测；**按不确定度（MEC 半径）从大到小**优先。

    v2 用"方位夹角多样性"筛选，但方位夹角依赖非常粗糙的楔形重心估计，
    会把真正困难（单方位、楔形 MEC~750m）的频道长期排除在外。改为不确定性优先。
    """

```

```

budget = self.refine_budget if budget is None else budget
pos = np.asarray(pos, float)
cand = []
for ch in self.ledger.detected_channels():
    if ch in self.sim.cleared:
        continue
    cs = self.ledger.channels[ch]
    cen = cs.centroid
    if cen is None:
        continue
    d = float(np.linalg.norm(np.asarray(cen, float) - pos))
    m = cs.mec
    if m is not None and m[0] <= CLEAR_TOL:
        if d <= ENROUTE_MAX:
            self._try_clear(ch)
            continue
    if d > REFINE_D_MAX:
        continue
    unc = float(m[0]) if m is not None else 5000.0
    cand.append((-unc, d, ch))
cand.sort()
for _, d, ch in cand[:budget]:
    if ch in self.sim.cleared:
        continue
    self._measure(pos, ch)
    self._try_clear(ch)

# ----- 阶段 -----
def initial_scan(self):
    for ch in range(1, N_CHANNELS + 1):
        self._measure((0.0, 0.0), ch)
    self._visited.append(np.zeros(2))

def _nodes(self):
    out = [("s", i) for i in sorted(self._pend)]
    for ch in self.ledger.detected_channels():
        if ch in self.sim.cleared:
            continue
        if self._src_cool.get(ch, 0) > 0:
            continue
        cs = self.ledger.channels[ch]
        m = cs.mec
        if m is not None and m[1] is not None and m[0] <= SRC_NODE_TOL:
            out.append(("c", ch))
    return out

def _tick_cool(self):
    for k in list(self._src_cool):
        self._src_cool[k] -= 1
        if self._src_cool[k] <= 0:
            self._src_cool.pop(k, None)

def _sig(self, nodes):
    s = []
    for t, v in nodes:
        if t == "s":
            s.append(("s", v))
        else:
            c = self._mec_center(v)
            s.append(("c", v, round(float(c[0]), 0), round(float(c[1]), 0)))
    return tuple(s)

def merged_walk(self, max_passes=3):
    """证书巡回 + 途中清源; 节点动态重规划."""
    for _pass in range(max_passes):
        self._pend = set(range(SKELETON_N))
        self._order = []
        self._osig = None
        guard = 0
        while True:
            guard += 1
            if guard > 400:
                break
            if self.sim.elapsed_real() > REAL_TIME_LIMIT:

```

```

        return
    if self.ledger.resolved():
        return
    nodes = self._nodes()
    if not nodes:
        break
    sig = self._sig(nodes)
    if sig != self._osig:
        pts = [self._node_pos(t, v) for t, v in nodes]
        tl, order = _tsp_open(pts, self.sim.cur_pos)
        self._order = [nodes[k] for k in order]
        self._osig = sig
        self._order_len = max(self._order_len, tl)
    if not self._order:
        break
    nd = self._order.pop(0)
    self._tick_cool()
    if nd[0] == "s":
        self._serve_skeleton(nd[1])
    else:
        self._serve_source(nd[1])
    self._osig = None
    # 若仍有未知频道，再来一遍（证书未达成）
    if not self.ledger.unknown_channels():
        return

def _node_pos(self, t, v):
    if t == "s":
        return SKELETON[v]
    c = self._mec_center(v)
    return np.zeros(2) if c is None else c

def _serve_skeleton(self, idx):
    target = np.asarray(SKELETON[idx], float)
    self._pend.discard(idx)
    unknown = self.ledger.unknown_channels()
    for ch in sorted(unknown, key=lambda c: abs(c - self.ledger.cur_channel)):
        if self.sim.elapsed_real() > REAL_TIME_LIMIT:
            return
        cs = self.ledger.channels[ch]
        if not cs.is_active:
            continue
        if self.ledger.check_empty(ch):
            cs.state = "empty"
            continue
        self._measure(target, ch)
    self.ledger.mark_empty()
    self._refine_at(target)

def _serve_source(self, ch):
    if ch in self.sim.cleared:
        return
    if not self.ledger.channels[ch].is_active:
        return
    c = self._mec_center(ch)
    if c is None:
        return
    # 走到 MEC 中心并就地测量（既清除又补探针）
    self._measure(c, ch)
    if not self._try_clear(ch):
        # 兜底：MEC 未收敛时按几何中心强清一次（救回接近共线/单方位源）
        self._try_clear(ch, force=True)
        # 未清除 -> 顺路再追一小步，然后冷却，让 TSP 先走别处
        self.pursue(ch, max_steps=3)
    if ch not in self.sim.cleared:
        self._src_cool[ch] = SRC_COOLDOWN

def finalize(self, rounds=3):
    """末段：对剩余待清源做 TSP 巡回清除；追踪预算逐步放大."""
    for r in range(rounds):
        if self.sim.elapsed_real() > REAL_TIME_LIMIT:
            return

```

```

pend = []
for ch in self.ledger.detected_channels():
    if ch in self.sim.cleared:
        continue
    c = self._mec_center(ch)
    if c is not None:
        pend.append((ch, c))
if not pend:
    return
self.fin_log.append(("round", r, "n_pend", len(pend),
                    "mecs", [round(float(self.ledger.channels[ch].mec[0]), 1)
                             for ch, _ in pend],
                    "pos", [round(float(v), 0) for v in self.sim.cur_pos]))
pts = [c for _, c in pend]
tl, order = _tsp_open(pts, self.sim.cur_pos)
self.fin_log.append(("tsp", r, round(tl, 0)))
progressed = False
for k in order:
    if self.sim.elapsed_real() > REAL_TIME_LIMIT:
        return
    ch, c = pend[k]
    if ch in self.sim.cleared:
        continue
    cs = self.ledger.channels[ch]
    if not cs.is_active:
        continue
    b0 = len(cs.bearings)
    mv0 = self.mv; n0 = self.n_meas
    self._measure(c, ch)
    if self._try_clear(ch):
        self.fin_log.append(("clear", ch, "mv", round(self.mv - mv0),
                            "meas", self.n_meas - n0))
        progressed = True
        continue
    # finalize 是最后机会, MEC 未收敛也按 centroid 兜底强清一次
    if self._try_clear(ch, force=True):
        self.fin_log.append(("clear_fb", ch, "mv", round(self.mv - mv0),
                            "meas", self.n_meas - n0))
        progressed = True
        continue
    self.pursue(ch, max_steps=5 + 3 * r)
    self.fin_log.append(("pursue", ch, "mv", round(self.mv - mv0),
                        "meas", self.n_meas - n0,
                        "mec", round(float(cs.mec[0]), 1) if cs.mec else -1,
                        "bear", len(cs.bearings)))
    if ch in self.sim.cleared or len(self.ledger.channels[ch].bearings) > b0:
        progressed = True
    elif self._mec_center(ch) is not None:
        progressed = True
if not progressed:
    return

# ----- 主循环 -----
def run(self):
    sim = self.sim
    try:
        self._phase("initial")
        self.initial_scan()
        self._phase("merged")
        self.merged_walk()
        self._phase("final")
        self.finalize()
        self._phase("done")
        self.phase_stats = dict(self._stats_acc)
    except Exception as e:
        print(f"[WARN] Policy3 异常: {e}")
        import traceback; traceback.print_exc()
    self._sync()
    n = len(sim.cleared)
    vt = sim.vt
    if self.verbose:
        print(f"[Policy3] cleared={n} vt={vt:.0f} order={self._order_len:.0f} "
              f"phases={self.phase_stats}")
    return n, vt

```

```
# 兼容旧命名：正式交付统一使用 Policy
Policy = Policy3
```

程序 q4/run_q4.py

```
# -*- coding: utf-8 -*-
"""启动 Q4 环抱证书策略 (v3 三段式合并巡回) 连接真实测试服务器。

可从任意工作目录运行: python q4/run_q4.py 或 cd q4 && python run_q4.py
python q4/run_q4.py --watch
"""
import argparse
import datetime
import os
import sys

_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, _root)
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, os.path.join(_root, "q2"))
import robot as R
from policy import Policy3

TEAM = "YOUR_TEAM_ID"
BASE = "http://127.0.0.1:2026"
LOGDIR = os.path.join(_root, "logs")

def make_policy(sim):
    return Policy3(sim)

def run_once(wait_s=600):
    os.makedirs(LOGDIR, exist_ok=True)
    stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    log_path = os.path.join(LOGDIR, f"robot_p4_{stamp}.jsonl")
    log = open(log_path, "w", encoding="utf-8")
    print(f"robot_id={TEAM}, 问题 4, 接口 {BASE}")
    print(f"日志: {log.name}")
    print("等待测试窗口开启...")
    sim = R.Sim(BASE, TEAM, log)
    sim.enter(wait_s=wait_s)
    print("[ENTER] 成功进入, 开始 Q4 v3 策略")
    make_policy(sim).run()
    try:
        sim.exit()
        print("[EXIT] 已退出")
    except Exception as e:
        print(f"[EXIT] 失败: {e}")
    log.close()

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--watch", action="store_true", help="持续监听测试窗口, 一局结束再等下一局")
    ap.add_argument("--wait", type=int, default=600, help="单局等待窗口秒数(仅非 --watch)")
    args = ap.parse_args()
    if args.watch:
        R.watch_loop(BASE, TEAM, 4, make_policy, "Q4 v3")
        return
    run_once(wait_s=args.wait)

if __name__ == "__main__":
    main()
```

程序 robot.py

```

# -*- coding: utf-8 -*-
"""
2026 高教社杯 B 题 机器狗程序（问题 3 / 问题 4 简版策略）

用法：
python robot.py --team <参赛队号> --problem 3 [--port 2026] [--wait 600]
python robot.py --team <参赛队号> --problem 4

策略概述：
1. 等待测试窗口开启(轮询 /enter, 复用同一 request_id, 幂等安全)
2. 扫描阶段：依次走到 13/17 个扫描点，在每个点对未清除频道逐个测向
   扫描点 = 中心 + 6@r1000 + 6@r1600(偏移 30°)，已数值验证：
   区域内任一点到最近扫描点距离 <= 801m < 1000m(接收半径下限)
3. 定位阶段：对每个听到信号的频道，补充 2~3 次不同位置的测向，
   所得方位线做最小二乘交会 -> 估计干扰源位置
4. 清除阶段：走到估计点 /clear；若失败按 25m 螺旋扩大搜索清除
5. 问题 4：定向干扰源存在盲区，用"换几何形状的第二轮扫描"兜底，
   连续 2 轮无新发现则结束
6. 输出统计：清除数 / 虚拟总时间 / 平均定位清除时间，并写日志
"""
import argparse
import datetime
import json
import math
import os
import sys
import time

import requests

# ----- 常量 -----
SPEED = 5.0 # m/s
SWITCH_COST = 1.0 # s
MEASURE_COST = 5.0 # s
CLEAR_COST = 3.0 # s (未发现) / 成功为 5
NEAR_R = 5.0 # 近距离阈值
CLEAR_R = 20.0 # 清除半径
ARENA_R = 1800.0
RECV_R_MIN = 1000.0 # 有效接收半径下限(规划用)
ERR_DEG = 1.0 # 测向误差 ±1°
CHANNELS = list(range(1, 21))
REAL_TIME_LIMIT = 18 * 60 # 程序运行自保上限(秒)，留裕量给 /exit

def pol(r, a_deg):
    return (r * math.cos(math.radians(a_deg)), r * math.sin(math.radians(a_deg)))

def scan_points(rot=0.0):
    """区域内 13 个扫描点；rot 为整体旋转角(度)"""
    pts = [(0.0, 0.0)]
    for i in range(6):
        pts.append(pol(1000, i * 60 + rot))
    for i in range(6):
        pts.append(pol(1600, i * 60 + 30 + rot))
    return pts

def outer_points(rot=15.0):
    """区域外圈 12 个扫描点(r=2000, 允许在区域外)。
    作用：兜住"贴边且朝向朝外"的定向干扰源 — 它们的覆盖半平面
    几乎全部在目标区域外，区域内的扫描点全部处于盲区。
    数值验证：内圈 + 外圈对随机定向源 20 万样本 0 漏检。"""
    return [pol(2000, i * 30 + rot) for i in range(12)]

def greedy_path(start, pts):
    """简单最近邻排序，减少移动耗时"""
    remain = list(pts)
    cur = start
    order = []

```

```

while remain:
    nxt = min(remain, key=lambda p: math.hypot(p[0] - cur[0], p[1] - cur[1]))
    order.append(nxt)
    remain.remove(nxt)
    cur = nxt
return order

class TestEnded(Exception):
    """模拟器返回 accepted=false(测试已结束/被拒), 需要优雅收尾"""
    pass

class Sim:
    """模拟器 HTTP 客户端: 串行动作, 幂等重试"""

    def __init__(self, base, team, log):
        self.base = base
        self.team = team
        self.log = log
        self.n = 0
        self.cur_channel = 1      # /enter 后测向机在频道 1
        self.cur_pos = (0.0, 0.0)
        self.vt = 0.0            # 最近一次 accepted 的虚拟时刻
        self.t0 = None           # /enter 成功的现实时刻
        self.cleared = set()
        self.measure_cnt = 0
        self.switch_cnt = 0
        self.clear_fail = 0

    def _rid(self, tag):
        self.n += 1
        return f"{tag}-{self.n}"

    def _post(self, path, body, rid, timeout=15):
        """幂等 POST: 网络异常/5xx/429 复用同一 request_id 重试"""
        last = None
        for attempt in range(10):
            try:
                r = requests.post(self.base + path, json=body, timeout=timeout,
                                   headers={"Content-Type": "application/json; charset=utf-8"})
                if r.status_code in (429, 500):
                    # 瞬时服务端错误/限流: 幂等重试同一动作
                    last = f"HTTP {r.status_code}"
                    time.sleep(1.5)
                    continue
            except:
                try:
                    return r.status_code, r.json()
                except ValueError:
                    last = f"HTTP {r.status_code}, non-JSON body"
            except requests.RequestException as e:
                last = repr(e)
            time.sleep(0.5)
        raise RuntimeError(f"请求 {path} {rid} 多次失败: {last}")

    def act(self, path, body, tag, timeout=15):
        rid = self._rid(tag)
        body = dict(body, arena_id="default", robot_id=self.team, request_id=rid)
        code, resp = self._post(path, body, rid, timeout)
        self.log.write(json.dumps({"t": datetime.datetime.now().isoformat(timespec="seconds"),
                                   "rid": rid, "path": path, "req": body,
                                   "http": code, "resp": resp}, ensure_ascii=False) + "\n")
        self.log.flush()
        if code == 200 and resp.get("accepted"):
            if path in ("/measure", "/clear"):
                self.vt = resp["virtual_time_s"]
            return resp
        # accepted=false: 多为测试已结束/窗口超时; 优雅收尾而不是崩溃
        raise TestEnded(f"动作被拒绝 {path} {rid}: HTTP {code} {resp}")

    def try_enter(self, request_id):
        """试一次 /enter; 成功返回 resp, 否则 (None, 原因)"""
        body = {"arena_id": "default", "robot_id": self.team, "request_id": request_id}

```

```

try:
    r = requests.post(self.base + "/enter", json=body, timeout=5,
                      headers={"Content-Type": "application/json; charset=utf-8"})
    code, resp = r.status_code, None
    try:
        resp = r.json()
    except ValueError:
        pass
except requests.RequestException as e:
    return None, repr(e)
if code == 200 and resp and resp.get("accepted"):
    self.t0 = time.time()
    self.vt = resp["virtual_time_s"]
    self.log.write(json.dumps({"t": datetime.datetime.now().isoformat(),
                              "rid": request_id, "path": "/enter",
                              "http": code, "resp": resp}) + "\n")
    self.log.flush()
    print(f"[ENTER] ok, 剩余现实时间 {resp.get('remaining_real_duration_s')}s")
    return resp, None
return None, f"HTTP {code} {resp}"

def enter(self, wait_s=600, request_id=None):
    """等待测试窗口开启并进入；同一次等待复用同一 request_id。
    wait_s<=0 表示一直等到成功。"""
    rid = request_id or "enter-1"
    t_end = (time.time() + wait_s) if wait_s and wait_s > 0 else None
    last = None
    while True:
        resp, last = self.try_enter(rid)
        if resp is not None:
            return resp
        if t_end is not None and time.time() > t_end:
            raise RuntimeError(f"等待测试窗口超时({wait_s}s): {last}")
        time.sleep(1.0)

def measure(self, pos, ch):
    x, y = round(float(pos[0]), 3), round(float(pos[1]), 3)
    resp = self.act("/measure", {"position": {"x": x, "y": y}, "channel": ch}, "m")
    if ch != self.cur_channel:
        self.switch_cnt += 1
    self.cur_channel = ch
    self.cur_pos = (x, y)
    self.measure_cnt += 1
    return resp

def clear(self, pos, ch):
    x, y = round(float(pos[0]), 3), round(float(pos[1]), 3)
    resp = self.act("/clear", {"position": {"x": x, "y": y}, "channel": ch}, "c")
    self.cur_pos = (x, y)
    ok = resp.get("clear_result") == "success"
    if ok:
        self.cleared.add(ch)
    else:
        self.clear_fail += 1
    return ok

def exit(self):
    return self.act("/exit", {}, "x")

def elapsed_real(self):
    return time.time() - self.t0 if self.t0 else 0.0

def watch_loop(base, team, problem, make_policy, label):
    """持续监听官方窗口：进局 → 跑策略 → 退出 → 再等下一局。Ctrl+C 停止。"""
    log_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), "logs")
    os.makedirs(log_dir, exist_ok=True)
    print(f"watch: robot_id={team}, 问题{problem}, 接口 {base}")
    print(f"策略: {label}")
    print("持续等待测试窗口, Ctrl+C 停止")
    n_fail = 0
    game = 0
    while True:

```

```

game += 1
stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
log_path = os.path.join(log_dir, f"robot_p{problem}_{stamp}.jsonl")
log = open(log_path, "w", encoding="utf-8")
sim = Sim(base, team, log)
rid = f"enter-{stamp}-{game}"
while True:
    resp, err = sim.try_enter(rid)
    if resp is not None:
        n_fail = 0
        remain = resp.get("remaining_real_duration_s")
        print(f"\n=== 第{game}局 remaining={remain}s "
              f"{time.strftime('%H:%M:%S')} ===")
        print(f"日志: {log_path}")
        break
    n_fail += 1
    if n_fail <= 2 or n_fail % 8 == 0:
        print(f"enter wait ({n_fail}): {err}")
        time.sleep(3.5)
try:
    n, vt = make_policy(sim).run()
    avg = (vt / n) if n else float("inf")
    print(f"[DONE] cleared={n} vt={vt:.1f} avg={avg:.1f}s")
except TestEnded as e:
    print(f"[END] {e}")
except Exception as e:
    print(f"[ERR] {type(e).__name__}: {e}")
try:
    sim.exit()
    print("[EXIT] 已退出")
except Exception as e:
    print(f"[EXIT] {e}")
log.close()
print("等待下一局...")
time.sleep(2.0)

# ----- 定位数学 -----
def bearing_line(p, deg):
    """返回 (A, u): 过点A、方向u 的方位线"""
    a = math.radians(deg)
    return p, (math.cos(a), math.sin(a))

def lsq_estimate(lines):
    """最小二乘交会: 每条线给 n·x = n·A, n 为法向"""
    Sxx = Sxy = Syy = sx = sy = 0.0
    for (A, u) in lines:
        n = (-u[1], u[0])
        b = n[0] * A[0] + n[1] * A[1]
        Sxx += n[0] * n[0]; Sxy += n[0] * n[1]; Syy += n[1] * n[1]
        sx += n[0] * b; sy += n[1] * b
    det = Sxx * Syy - Sxy * Sxy
    if abs(det) < 1e-9:
        return None
    return ((sx * Syy - sy * Sxy) / det, (Sxx * sy - Sxy * sx) / det)

def dist_to_line(p, line):
    (A, u) = line
    dx, dy = p[0] - A[0], p[1] - A[1]
    return abs(dx * (-u[1]) + dy * u[0])

# ----- 策略 -----
class Robot:
    def __init__(self, sim, problem):
        self.sim = sim
        self.problem = problem
        self.bearings = {} # ch -> [(pos, deg)]

# -- 单目标: 补测 -> 交会 -> 清除 --
def hunt(self, ch):

```

```

sim = self.sim
lines = [bearing_line(p, d) for (p, d) in self.bearings[ch]]
est = None
# 补测至多 4 次: 优先垂直偏移(交角大), 无信号则沿方位线推进
for k in range(4):
    if len(lines) >= 2:
        est = lsq_estimate(lines)
        res = max(dist_to_line(est, L) for L in lines)
        if len(lines) >= 3 and res < 15.0:
            break
        # 已有估计: 到估计点前 300m、侧偏 150m 处补测(交角好且近)
        dx, dy = est[0] - sim.cur_pos[0], est[1] - sim.cur_pos[1]
        dd = math.hypot(dx, dy) or 1.0
        u = (dx / dd, dy / dd)
        pp = (-u[1], u[0])
        q = (est[0] - 300 * u[0] + 150 * pp[0], est[1] - 300 * u[1] + 150 * pp[1])
    else:
        # 只有一条方位线: 先两侧垂直偏移 600m, 无信号则沿线推进
        (A0, u0) = lines[0]
        perp = (-u0[1], u0[0])
        if k == 0:
            q = (A0[0] + 600 * perp[0], A0[1] + 600 * perp[1])
        elif k == 1:
            q = (A0[0] - 600 * perp[0], A0[1] - 600 * perp[1])
        else:
            d_along = (800.0, 1300.0)[k - 2]
            q = (A0[0] + d_along * u0[0], A0[1] + d_along * u0[1])
    r = sim.measure(q, ch)
    mr = r.get("measure_result")
    if mr == "near":
        # 距离<=5m, 直接原地清除(清除半径 20m)
        if sim.clear(sim.cur_pos, ch):
            print(f" [ch{ch}] 近距离直接清除 @ {sim.cur_pos}")
            return True
    elif mr == "direction":
        self.bearings[ch].append((sim.cur_pos, r["svd_deg"]))
        lines.append(bearing_line(sim.cur_pos, r["svd_deg"]))
    # no_signal: 换下一个候选点重试
est = lsq_estimate(lines) if len(lines) >= 2 else None
if est is None:
    # 退化: 只有一条方位线, 沿线走 800m 试探
    (A0, u0) = lines[0]
    est = (A0[0] + 800 * u0[0], A0[1] + 800 * u0[1])
# 清除: 估计点 -> 螺旋兜底
if sim.clear(est, ch):
    print(f" [ch{ch}] 交会估计 {est[0]:.0f},{est[1]:.0f} 清除成功 (vt={sim.vt:.0f}s)")
    return True
# 螺旋: 半径 25,50,...,175, 每圈 6 点
for rad in range(25, 200, 25):
    for a in range(0, 360, 60):
        q = (est[0] + rad * math.cos(math.radians(a)),
            est[1] + rad * math.sin(math.radians(a)))
        if sim.clear(q, ch):
            print(f" [ch{ch}] 螺旋搜索 r={rad} 清除成功 (vt={sim.vt:.0f}s)")
            return True
print(f" [ch{ch}] 清除失败!")
return False

# -- 一轮扫描: 输入扫描点列表, 返回本轮新听到(未清除)的频道 --
def sweep(self, pts):
    sim = self.sim
    found = []
    pts = greedy_path(sim.cur_pos, pts)
    np_ = len(pts)
    for i, p in enumerate(pts):
        for ch in CHANNELS:
            if ch in sim.cleared:
                continue
            # 现实时间自保
            if sim.elapsed_real() > REAL_TIME_LIMIT:
                print(f"[WARN] 现实时间接近上限, 提前结束扫描")
                return found, True

```

```

        r = sim.measure(p, ch)
        mr = r.get("measure_result")
        if mr == "direction":
            self.bearings.setdefault(ch, []).append((sim.cur_pos, r["svd_deg"]))
            if ch not in found:
                found.append(ch)
            print(f"  [扫描点{i+1}/{np_}] ch{ch} 方向 {r['svd_deg']:.2f}° @
{p[0]:.0f},{p[1]:.0f} (vt={sim.vt:.0f}s)")
        elif mr == "near":
            if sim.clear(sim.cur_pos, ch):
                print(f"  [扫描点{i+1}/{np_}] ch{ch} 近距离直接清除 (vt={sim.vt:.0f}s)")
    return found, False

def run(self):
    sim = self.sim
    # 问题 3: 全向源无盲区, 1 轮内圈扫描即全覆盖(几何保证)
    # 问题 4: 内圈 -> 区域外圈(兜贴边朝外定向源) -> 旋转 30°内圈(保险)
    if self.problem == 3:
        round_sets = [scan_points(0.0)]
    else:
        round_sets = [scan_points(0.0), outer_points(15.0), scan_points(30.0)]
    rounds_no_new = 0
    try:
        for ri, pts in enumerate(round_sets):
            print(f"\n===== 扫描轮 {ri+1}/{len(round_sets)} ({len(pts)}个点) =====")
            found, abort = self.sweep(pts)
            # 逐个清除本轮发现的目标
            pend = [c for c in found if c not in sim.cleared]
            while pend:
                if sim.elapsed_real() > REAL_TIME_LIMIT:
                    break
                ch = pend[0]
                ok = self.hunt(ch)
                pend = [c for c in pend[1:] if c not in sim.cleared] if not ok \
                    else [c for c in pend if c not in sim.cleared]
            if not found:
                rounds_no_new += 1
            else:
                rounds_no_new = 0
            if abort or rounds_no_new >= 2 or sim.elapsed_real() > REAL_TIME_LIMIT:
                break
    except TestEnded as e:
        print(f"[WARN] 测试被模拟器终止, 优雅收尾: {e}")
    # 收尾
    try:
        sim.exit()
        print("\n[EXIT] 已退出")
    except Exception as e:
        print(f"[EXIT] 失败(测试可能已结束): {e}")
    # 统计
    n = len(sim.cleared)
    vt = sim.vt
    print("\n===== 统计 =====")
    print(f"清除干扰源数 : {n}")
    print(f"虚拟世界总时间: {vt:.1f} s ({vt/3600:.2f} h)")
    print(f"平均定位清除时间: {(vt/n if n else float('inf')):.1f} s/个")
    print(f"检测次数 {sim.measure_cnt}, 换频 {sim.switch_cnt} 次, 清除失败 {sim.clear_fail} 次")
    print(f"程序现实运行时间: {sim.elapsed_real():.1f} s")

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--team", required=True, help="参赛队号(robot_id)")
    ap.add_argument("--problem", type=int, default=3, choices=[3, 4])
    ap.add_argument("--port", type=int, default=2026)
    ap.add_argument("--wait", type=int, default=600, help="等待测试窗口秒数")
    args = ap.parse_args()

    base = f"http://127.0.0.1:{args.port}"
    os.makedirs("logs", exist_ok=True)
    stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    log = open(f"logs/robot_p{args.problem}_{stamp}.jsonl", "w", encoding="utf-8")

```

```
print(f"robot_id={args.team}, 问题{args.problem}, 接口 {base}")
print(f"日志: {log.name}")
print("等待测试窗口开启...(请在模拟器中确认开始演练测试)")

sim = Sim(base, args.team, log)
sim.enter(wait_s=args.wait)
Robot(sim, args.problem).run()

if __name__ == "__main__":
    main()
```